

MC ARTWORKS STUDIO

DOCUMENTACIÓN TÉCNICA

Proyecto WordPress y WooCommerce con sistema multimedia e interfaz dinámica

Desarrollado por · Gaston Rizzo

Índice

Sección	Pág.	Sección	Pág.
1. Descripción general	3	21. Página Cart y personalización	67
2. Funcionamiento general del proyecto	3	22. Sesión y conservación de datos	70
3. Tecnologías, plugins y dependencias	4	23. Checkout: formulario final de compra y validaciones	72
4. WordPress: qué es y cómo ejecuta MC Artworks Studio	8	24. Método de pago manual mediante PayPal	76
5. Base de datos y modelo de información	10	25. Creación del pedido	79
6. Estructura del proyecto	15	26. Logs: registro de pedidos y correos	81
7. Qué hace WordPress, Blocksyy WooCommerce	17	27. Correo al comprador	84
8. Secuencia inicial y animaciones	18	28. Correo al administrador	88
9. Introducción multimedia y HLS	23	29. Envío de correo y seguimiento de errores	91
10. Selección de facción	29	30. Pedido recibido (Order Received) y ejemplo completo	94
11. Cookies y apariencia NOD / GDI	32	31. Galería (Gallery)	99
12. Menú principal	36	32. Sección de servicios (Our Services)	109
13. Inicio (Home)	40	33. JavaScript y cambios dinámicos en la página	112
14. Cómo se gestionan los productos con WooCommerce	44	34. Adaptación entre navegadores	115
15. Campos propios con ACF	47	35. Recursos multimedia y servicios externos	118
16. Tienda y tarjetas de producto	48	36. Caché y rendimiento	120
17. Carga progresiva de videos y Fancybox	52	37. Tratamiento de datos y seguridad	123
18. Filtros de la Tienda con HUSKY y AJAX	56	38. Instalación, migración y verificación del proyecto	125
19. Bundles	59	39. Posibles líneas de evolución	130
20. Carrito lateral y límite de artículos	63	40. Alcance técnico y conclusión	133

PARTE I - PROYECTO Y ARQUITECTURA

1. Descripción general

MC Artworks Studio es un sitio WordPress orientado a la venta de artworks digitales y a ofrecer servicios de edición de imágenes, creación de artworks y fotomontajes realistas. A partir de WordPress y WooCommerce se desarrolló una interfaz audiovisual inspirada en la estética de un videojuego: el visitante visualiza una introducción, elige una facción y después recorre el sitio con la apariencia asociada a esa elección.

El tema hijo cambia gran parte de la presentación habitual de WordPress y WooCommerce. La tienda muestra videos como vista previa de los productos, incorpora filtros, bundles, un carrito lateral, opciones de personalización, un checkout reducido y un método de pago manual. Cuando se confirma un pedido, el proyecto guarda un registro en un archivo de log y prepara correos HTML diferentes para el comprador y para administración.

Alcance de esta documentación

Esta documentación describe la arquitectura, el funcionamiento y las principales decisiones técnicas de MC Artworks Studio. Explica cómo se integran WordPress, Blocksy, WooCommerce y Advanced Custom Fields, cómo se organizan los productos y bundles, cómo se gestionan los recursos multimedia, el carrito y el checkout, y cómo se almacenan y procesan los datos del sistema. También se detallan la estructura del proyecto, la base de datos, las dependencias utilizadas, el flujo de compra, la gestión de pedidos, el envío de correos y el registro de operaciones.

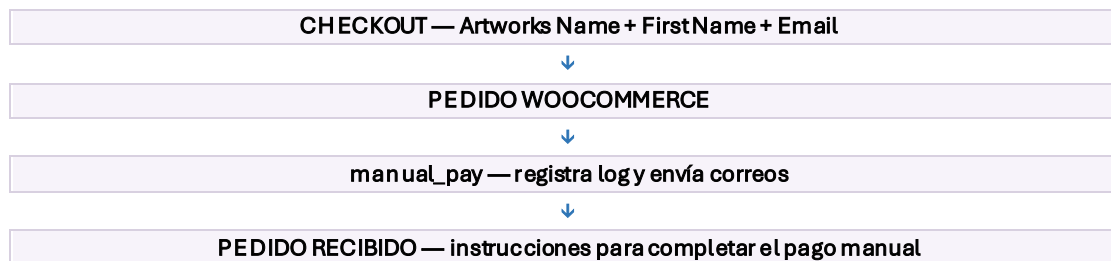
Qué vende y qué muestra

- Artworks digitales representados mediante productos de WooCommerce.
- Bundles (paquetes) que agrupan varios artworks relacionados.
- Servicios de edición de imágenes, creación de artworks y fotomontajes realistas.
- Una experiencia multimedia con videos, música, animaciones de entrada y dos apariencias visuales: NOD y GDI.

2. Funcionamiento general del proyecto

La forma más sencilla de entender el proyecto es seguir dos recorridos: una primera visita y una compra. WordPress recibe la dirección solicitada y carga el tema hijo. Cuando el visitante entra en la tienda, el carrito, el checkout o un pedido, WooCommerce gestiona los productos y los pasos de la compra.





Ejemplo que se reutiliza en el documento

Para explicar el recorrido se usará un caso ficticio: Alex compra “Cyberpunk Portrait” por USD 18, elige Frame Color = Blue, Comic Balloon Text = “Welcome back” y escribe Artworks Name = “Raven”. Los datos son de ejemplo; la estructura refleja la información que maneja el código.

3. Tecnologías, plugins y dependencias

MC Artworks Studio combina WordPress con un tema padre, un tema hijo propio, plugins de WordPress, librerías JavaScript y servicios externos. No todas estas piezas cumplen la misma función: algunas forman la plataforma base, otras agregan capacidades a WordPress y otras se cargan desde Internet cuando la interfaz las necesita. Separarlas por tipo ayuda a entender qué parte del sistema resuelve cada una y qué depende directamente del código de blocksy-child.

3.1 Plataforma, tecnologías y librerías base

Estas tecnologías sostienen el proyecto y sirven como base para el código propio. Aquí se resume el papel de cada una; las secciones posteriores explican su integración con más detalle.

Tecnología	Papel dentro de MC Artworks Studio
WordPress	Es el CMS sobre el que funciona el sitio. Administra páginas, carga temas y plugins y ofrece hooks y funciones como wp_mail() y wp_upload_dir().
PHP	Se ejecuta en el servidor. El tema hijo lo utiliza para consultar productos, decidir qué contenido cargar y generar HTML, CSS o JavaScript según la página y la facción.
Blocksy	Es el tema padre. Proporciona una base de WordPress que blocksy-child modifica sin editar directamente los archivos originales de Blocksy.
JavaScript	Se ejecuta en el navegador. Controla videos, audio, modales, animaciones, AJAX, observadores del DOM y buena parte de la interacción de la interfaz.
jQuery	Es una librería JavaScript utilizada principalmente para integrarse con WooCommerce y con plugins que actualizan elementos de forma dinámica.
MariaDB / MySQL	Es el sistema de base de datos. La instalación utiliza MariaDB 11.8.6 para conservar productos, configuración, sesiones y pedidos de WordPress y WooCommerce.

3.2 Plugins de WordPress utilizados

Un plugin de WordPress es una extensión instalada dentro de la plataforma para agregar una función que no pertenece al tema. MC Artworks Studio utiliza varios plugins como infraestructura y después adapta su apariencia o comportamiento desde blocksy-child. El tema hijo no sustituye esas herramientas: se apoya en ellas y agrega lógica propia encima.

3.2.1 WooCommerce

WooCommerce es el plugin de comercio electrónico que sostiene todo el flujo de venta. Administra los productos, precios, carrito, sesiones, checkout, pedidos y la estructura de los métodos de pago. Los artworks y bundles continúan siendo productos reales de WooCommerce aunque su presentación visual esté completamente personalizada por el tema hijo.

Por ejemplo, cuando el visitante pulsa ADD TO CART, blocksy-child puede validar el límite de artículos y cambiar el aspecto del botón, pero el producto termina dentro del carrito real de WooCommerce y más tarde forma parte de un WC_Order.

```

Artwork
↓
Producto WooCommerce
↓
ADD TO CART
↓
Carrito
↓
Checkout
↓
WC_Order

```

3.2.2 Advanced Custom Fields (ACF)

Advanced Custom Fields (ACF) es un plugin que permite agregar información personalizada a elementos administrados desde WordPress. En este proyecto complementa los productos de WooCommerce con datos que el producto estándar no incluye.

Los campos confirmados en la instalación son `video_destacado`, `video_workshop`, `productos_del_bundle` y `tiene_globo`. El tema hijo consulta esos valores para saber qué video mostrar, qué artworks forman un bundle o si un producto admite Comic Balloon Text. La sección 15 explica después cómo se leen estos datos mediante `get_field()` y cómo influyen en la interfaz.

```

Producto WooCommerce: Cyberpunk Portrait
↓
ACF
video_destacado = cyberpunk.mp4
tiene_globo = Sí
↓
Tema hijo
muestra el video y habilita Comic Balloon Text

```

3.2.3 HUSKY / WOOF

HUSKY / WOOF es el plugin utilizado como motor de filtros de productos dentro de la Tienda. Proporciona la infraestructura necesaria para aplicar criterios sobre el catálogo, como categorías, búsqueda por texto y rango de precios, sin que el tema hijo tenga que construir desde cero todo el sistema de filtrado.

MC Artworks Studio utiliza esa base y la adapta. El tema cambia textos, oculta categorías que no deben aparecer en Shop, agrega una lupa al buscador, exige un mínimo de tres caracteres, ajusta el rango de precios y vuelve a aplicar determinados comportamientos después de las actualizaciones AJAX. La sección 18 desarrolla ese funcionamiento con detalle.

```

Usuario modifica un filtro
↓
HUSKY prepara los criterios
↓
petición AJAX
↓
WooCommerce obtiene los productos compatibles
↓
HUSKY reemplaza la grilla
↓
el tema hijo vuelve a aplicar sus ajustes

```

3.2.4 Xootix Side Cart

Xootix Side Cart es un plugin para WooCommerce que presenta el contenido del carrito dentro de un panel lateral. Permite consultar o eliminar productos sin abrir inmediatamente la página Cart.

Xootix no mantiene un carrito independiente. WooCommerce conserva los productos y procesa las operaciones; Xootix representa ese mismo estado mediante una interfaz lateral que puede actualizarse dinámicamente. `blocksy-child` agrega sobre esa interfaz videos, loaders, el límite global de artículos y la sincronización entre ADD TO CART y VIEW CART.

```

ADD TO CART
↓
WooCommerce agrega el producto
↓
evento added_to_cart
↓
Xootix actualiza el carrito lateral

```

3.2.5 WP Mail SMTP

WP Mail SMTP es el plugin utilizado para configurar la salida de correo de WordPress mediante un servidor o servicio SMTP. No crea los correos del proyecto: el contenido HTML del comprador y de administración se construye desde blocksy-child y se entrega a WordPress mediante `wp_mail()`.

Después de esa llamada, WP Mail SMTP canaliza la solicitud hacia el servicio SMTP configurado. Por eso un pedido puede existir correctamente aunque exista un problema posterior de entrega de correo; las secciones 27 a 29 explican el contenido de los mensajes y el seguimiento de errores.

Tema hijo → `wp_mail()` → WordPress → WP Mail SMTP → Servidor / servicio SMTP → Destinatario

3.2.6 All-In-One Security (AIOSEC)

All-In-One Security (AIOSEC) es un plugin orientado a reforzar la seguridad de una instalación WordPress. Forma parte del entorno relevado de MC Artworks Studio, pero no participa directamente en la lógica que construye la Tienda, el carrito, el checkout o los pedidos.

La configuración concreta de sus módulos se administra fuera del tema hijo y no está definida dentro de blocksy-child. Por ese motivo esta documentación lo identifica como una herramienta auxiliar de seguridad sin atribuirle controles específicos que no hayan sido verificados en la configuración de WordPress.

3.3 Librerías y servicios externos

Además de los plugins instalados en WordPress, el proyecto utiliza librerías JavaScript y servicios externos. Estas dependencias no deben confundirse con un plugin: el tema puede cargarlas directamente mediante una URL o utilizarlas desde el navegador.

3.3.1 Fancybox4

Fancybox 4 es una librería JavaScript para mostrar contenido dentro de una ventana superpuesta, o modal, sin abandonar la página actual. En MC Artworks Studio se utiliza para ampliar los videos de los artworks desde la Tienda y permitir que el visitante recorra el video anterior o siguiente sin volver a la grilla.

Fancybox no está instalado como plugin de WordPress en este proyecto. blocksy-child carga su CSS y JavaScript desde jsDelivr cuando se visita Shop, una categoría o una etiqueta de producto, y después vincula las tarjetas con `Fancybox.bind()`. La sección 17 explica esa integración y el modal personalizado.

```

Usuario pulsa el video
↓
elemento data-fancybox
↓
Fancybox abre el modal
↓
reproductor HTML5
↓
Previous / Next / Close

```

3.3.2 HLS.js

HLS.js es una librería JavaScript que permite reproducir streams HLS en navegadores que no ofrecen el soporte necesario de forma nativa. Los streams HLS se describen mediante archivos `.m3u8` que apuntan a fragmentos de video.

MC Artworks Studio utiliza HLS en la introducción multimedia y en Home. Cuando el navegador puede reproducir HLS directamente se utiliza ese soporte; cuando no puede, HLS.js conecta el manifiesto con el elemento <video>. La sección 9 desarrolla la carga, el fallback y la sincronización de los feeds.

3.3.3 jsDelivr

jsDelivr es una CDN, es decir, una red de distribución utilizada para entregar archivos externos al navegador. En este proyecto se utiliza para obtener HLS.js y Fancybox. Esto evita tener que guardar necesariamente esas librerías dentro de blocksy-child, aunque implica que deben poder descargarse desde Internet cuando la página las necesita.

3.3.4 Google Fonts

Google Fonts es el servicio externo utilizado para cargar algunas tipografías de la interfaz, entre ellas Orbitron y Share Tech Mono. Estas fuentes forman parte de la presentación visual y no almacenan productos ni intervienen en el flujo de compra.

3.4 Qué es obligatorio y qué es auxiliar

Las dependencias no tienen el mismo nivel de importancia. Algunas son parte directa del funcionamiento comercial y otras complementan una función visual, de transporte de correo o de seguridad.

Componente	Papel dentro del proyecto	Tipo de dependencia
WordPress	Plataforma base sobre la que se ejecuta todo el proyecto.	Obligatoria
Blocksy	Tema padre necesario para que funcione blocksy-child tal como fue desarrollado.	Obligatoria
WooCommerce	Base del catálogo, carrito, checkout y pedidos.	Obligatoria
ACF	Aporta datos propios de los productos que el tema utiliza de forma directa.	Obligatoria en la implementación actual
HUSKY / WOOF	Motor de filtros utilizado por la Tienda.	Necesaria para los filtros actuales
XootixSide Cart	Interfaz lateral del carrito; WooCommerce conserva el carrito real.	Necesaria para el side cart actual
WP Mail SMTP	Canaliza los correos hacia el servicio SMTP configurado.	Necesaria para el SMTP documentado
AIOS	Herramienta auxiliar de seguridad; no forma parte del flujo comercial principal.	Auxiliar
Fancybox 4	Librería externa utilizada por la vista ampliada de videos de Shop.	Necesaria para el modal de videos
HLS.js	Librería externa utilizada para reproducción HLS cuando el navegador lo necesita.	Condicional
jsDelivr	CDN desde la que se cargan HLS.js y Fancybox.	Necesaria mientras se use como CDN
Google Fonts	Servicio externo de tipografías.	Visual / auxiliar
MariaDB	Almacenamiento de WordPress, WooCommerce y la configuración del sitio.	Obligatoria en el entorno actual

Distinción importante

Fancybox y HLS.js no son plugins de WordPress en MC Artworks Studio. Son librerías JavaScript que el tema carga como dependencias externas. Del mismo modo, Blocksy es un tema padre, MariaDB es la base de datos y jsDelivr es una CDN. Esta separación evita tratar todas las piezas del sistema como si fueran el mismo tipo de componente.

3.5 Servidor y navegador

PHP y WordPress trabajan en el servidor antes de que la página llegue al visitante. HTML, CSS y JavaScript se ejecutan o interpretan en el navegador. Esta diferencia es importante: por ejemplo, PHP decide qué CSS de facción debe cargarse, mientras JavaScript abre el menú, reproduce videos o detecta cuándo un elemento entra en la zona visible.

4. WordPress: qué es y cómo ejecuta MC Artworks Studio

Antes de estudiar Intro, Shop o Checkout conviene entender el entorno que ejecuta todo el código. WordPress no es una librería secundaria: es la plataforma que recibe la petición, carga temas y plugins, dispara hooks y termina produciendo la respuesta que recibe el navegador.

4.1 Qué ocurre cuando Alex abre una URL

Paso	Qué ocurre	Quién participa
1	Alex solicita /shop/ desde el navegador.	Navegador + servidor web
2	El servidor entrega la petición a WordPress.	WordPress Core
3	WordPress carga configuración, plugins activos y el tema.	Core + plugins + Blocksy + blocksy-child
4	WooCommerce reconoce el contexto comercial y prepara su consulta.	WooCommerce
5	Los hooks registrados por blocksy-child modifican consulta, HTML, CSS o JavaScript.	Código propio
6	PHP termina y el servidor envía HTML/CSS/JS al navegador.	Servidor
7	El navegador crea el DOM y ejecuta JavaScript, incluidos cambios AJAX posteriores.	Navegador + plugins JS

Idea clave: WordPress actúa antes de que Alex vea la pantalla. Después de recibir la respuesta, el navegador continúa el trabajo con JavaScript. Por eso una misma funcionalidad de MC Artworks puede comenzar en PHP y terminar en un listener del DOM.

4.2 WordPress Core, tema y plugins no son la misma cosa

Capa	Qué aporta en MC Artworks	Qué no debe atribuirse automáticamente a esa capa
WordPress Core	Carga páginas, temas y plugins; ofrece hooks, conditional tags, wp_mail(), wp_upload_dir() y admin-ajax.php.	No contiene la experiencia NOD/GDI ni la lógica comercial específica del proyecto.
Blocksy	Tema padre y estructura base de WordPress.	No contiene las personalizaciones propias del child theme.
blocksy-child	Código específico: HUD, facciones, videos, Shop personalizado, Cart, gateway, logs, Gallery y Services.	No reemplaza WordPress ni crea un WooCommerce paralelo.
WooCommerce	WC_Product, catálogo, carrito, sesión, Checkout, WC_Order y gateways.	No decide por sí solo los videos, posters o identidad NOD/GDI.
ACF / HUSKY / Xootix	Metadatos, filtros y side cart.	No son la fuente de verdad del producto o del carrito.

4.3 El ciclo de la petición y los hooks

Un hook es un momento específico del funcionamiento de WordPress o WooCommerce en el que puede ejecutarse código personalizado. El tema o plugin registra una función para ese hook y, cuando WordPress llega a ese momento, llama automáticamente a esa función.

En este ejemplo, `add_action()` registra la función `insertar_presentacion()` para el hook `template_redirect`:

```
add_action('template_redirect', 'insertar_presentacion');
```

Cuando WordPress carga `functions.php`, registra esa relación, pero todavía no ejecuta `insertar_presentacion()`.

Más adelante, cuando WordPress alcanza el hook `template_redirect`, llama automáticamente a la función `insertar_presentacion()`.

Hook/momento	Uso comprobado en MC Artworks
<code>wp_enqueue_scripts</code>	Carga CSS/JS como Fancybox y los estilos de facción.
<code>wp_head</code>	Activa piezas como <code>hud_intro()</code> y otros bloques que imprimen contenido/código.
<code>template_redirect</code>	Permite decidir la presentación inicial antes de cargar la plantilla final.
<code>pre_get_posts</code>	Modifica la consulta principal de Shop para excluir Bundles.
<code>wp_footer</code>	Lugar donde varios bloques imprimen JavaScript que se ejecutará en el navegador.
Hooks WooCommerce	Insertan o modifican datos durante Shop, Cart, Checkout y creación del pedido.

4.4 Actions y filters

Una action permite indicar qué función debe ejecutarse cuando WordPress o WooCommerce alcanza un momento determinado de su funcionamiento. Por ejemplo:

```
add_action('wp_footer', 'mi_script');
```

En este caso, `mi_script()` queda registrada para ejecutarse cuando WordPress llegue al hook `wp_footer`.

Un filter, en cambio, recibe un valor que WordPress o WooCommerce está procesando, permite modificarlo y debe devolver el resultado para que el sistema continúe trabajando con esa versión. Por ejemplo:

```
add_filter(
    'woocommerce_checkout_fields',
    'simplificar_checkout_nombre_y_correo'
);
```

4.5 Prioridad: varios callbacks pueden compartir el mismo hook

El tercer argumento de `add_action()`/`add_filter()` puede establecer una prioridad. Un número menor se ejecuta antes. Si dos callbacks tienen la misma prioridad, el orden de registro también importa. Este detalle es relevante en el proyecto porque existen dos callbacks de CSS de facción con prioridad 999 y el mismo handle; la sección de caché analiza ese caso sin asumir que el segundo reemplaza automáticamente al primero.

4.6 Conditional tags: saber qué pantalla se está procesando

Funciones como `is_shop()`, `is_cart()`, `is_checkout()` e `is_page()` son preguntas sobre el contexto actual. En un `functions.php` que concentra muchas funcionalidades, estas comprobaciones evitan que un bloque pensado para Shop altere Cart o Gallery.

```
if (!is_shop()) {
    return;
}
```

```
// A partir de aquí el bloque sólo continúa cuando WordPress/WooCommerce
// está procesando Shop.
```

4.7 PHP del servidor frente a JavaScript del navegador

MC Artworks mezcla PHP con fragmentos HTML, CSS y JavaScript. PHP se ejecuta en el servidor. Cuando `functions.php` hace echo de un `<script>`, PHP no ejecuta `document.addEventListener()`: sólo escribe ese texto en la respuesta. El navegador lo recibe después y recién entonces ejecuta JavaScript.

Servidor	Frontera HTTP	Navegador
PHP / WordPress decide y genera contenido.	HTML + CSS + JavaScript viajan en la respuesta.	Se crea el DOM y se ejecutan eventos, observers, audio, video y AJAX.

4.8 admin-ajax.php: infraestructura WordPress, callback propio

Cuando el child theme envía una petición AJAX a `admin-ajax.php`, la URL pertenece a WordPress. La acción y el callback que responden pueden pertenecer a MC Artworks. Esto se ve en la sincronización con Xootix: un clic generado por el plugin termina llamando infraestructura de WordPress y luego código propio.

4.9 El “ruido” de WordPress y los plugins

Al inspeccionar una página no todo el DOM, CSS o JavaScript visible proviene de `blocksy-child`. WordPress, Blocksy, WooCommerce, HUSKY y Xootix también generan elementos o los reemplazan dinámicamente. Este es un origen frecuente de confusión durante la depuración.

Elemento visible	Origen principal esperado
Estructura base del sitio	WordPress + Blocksy
Producto y precio	WooCommerce
Preview audiovisual de la tarjeta	blocksy-child + ACF
Controles de filtrado	HUSKY/WOOF + adaptaciones propias
Side cart	Xootix + WooCommerce + sincronización propia
Cart / Checkout	WooCommerce + hooks de blocksy-child
Intro / Faction / HUD	Principalmente blocksy-child

4.10 Cómo depurar mentalmente una pantalla

Ante un elemento inesperado conviene preguntar, en orden: ¿lo generó WordPress/Blocksy?, ¿lo generó WooCommerce?, ¿lo insertó un plugin?, ¿lo imprimió el child theme?, ¿apareció después por AJAX? Sólo después de ubicar al propietario conviene buscar el código. Esa regla reduce el “ruido” típico de un ecosistema WordPress con varios plugins activos.

Regla de lectura del código en este documento: cuando cambia el archivo, la función, el callback o la capa de ejecución, se indica la ruta y la pieza que contiene el fragmento, quién la activa y en qué momento interviene. “Código fuente” identifica la procedencia real; no implica que WordPress ejecute ese fragmento en el instante en que se carga `functions.php`.

5. Base de datos y modelo de información

WordPress no guarda los artworks dentro de los archivos PHP. El código del tema hijo define cómo se muestran y cómo se procesan, mientras que los productos creados desde el panel, sus campos personalizados, las sesiones del carrito y los pedidos quedan almacenados en la base de datos MariaDB de la instalación. Esta separación permite modificar el catálogo sin editar `functions.php` cada vez que cambia un producto.

5.1 Qué guarda la base de datos

La base contiene el contenido administrado por WordPress y WooCommerce y también parte de la configuración necesaria para interpretarlo. En MC Artworks Studio conserva productos, categorías, attachments, campos ACF, opciones del sitio, sesiones temporales del carrito y pedidos.

```
Código PHP del tema hijo
  ↓ define el comportamiento
WordPress + WooCommerce + ACF
  ↓ administran el contenido
MariaDB
  ↓ conserva productos, metadatos,
    sesiones, pedidos y configuración
```

5.2 Tablas relevantes de la instalación

Tabla / grupo	Función en MC Artworks Studio
wp_posts	Productos, páginas, attachments y definiciones internas de WordPress/ACF.
wp_postmeta	Metadatos del producto, referencias ACF y metadatos de attachments.
wp_terms · wp_term_taxonomy · wp_term_relationships	Categorías y relación entre cada producto y sus categorías.
wp_options	Configuración general de WordPress, WooCommerce, tema y plugins.
wp_woocommerce_sessions	Sesión temporal con la que WooCommerce reconstruye el carrito.
wp_wc_orders · wp_wc_order_addresses · wp_wc_orders_meta	Pedidos HPOS, datos del comprador y metadatos generales del pedido.
wp_woocommerce_order_items · wp_woocommerce_order_itemmeta	Líneas de pedido y metadatos por producto, como Frame Color y Comic Balloon Text.

La instalación utiliza el prefijo wp_. Otra instalación de WordPress puede utilizar un prefijo diferente. El tema no crea tablas propias para MC Artworks Studio y trabaja principalmente mediante las APIs de WordPress, WooCommerce y ACF.

5.3 Por qué no existen tablas artworks o bundles

Registrar y administrar un producto con WooCommerce no crea una tabla llamada artworks. Tampoco existe una tabla propia bundles. Los dos se representan como productos estándar de WooCommerce y se diferencian por sus categorías y metadatos.

```
NO existe una tabla propia:
artworks(id, nombre, precio, video, ...)

En cambio:
wp_posts
├── post_type = product
├── metadatos + taxonomías de WooCommerce/ACF
```

5.4 Cómo se representa un artwork

La base de datos permite seguir un producto real. Ejemplo: El registro 988 corresponde a un artwork publicado. Aunque su nombre contiene “VARIATION #1”, WooCommerce lo clasifica como producto simple: esa palabra forma parte del nombre comercial y no representa una variación técnica de WooCommerce.

```

wp_posts
ID           = 988
post_title   = Geralt #1 | The Witcher | VARIATION #1
post_type    = product
post_status  = publish

WooCommerce product type = simple

```

5.5 ACF y wp_postmeta

Advanced Custom Fields agrega datos propios sin crear columnas nuevas en wp_posts. Los valores se conservan como metadatos relacionados mediante post_id. La base real contiene cuatro campos ACF de producto: video_destacado, video_workshop, productos_del_bundle y tiene_globo.

Campo ACF	Qué representa	Ejemplo de uso
video_destacado	Video principal asociado al producto.	La tarjeta de Shop y Bundles lo usa como vista previa.
video_workshop	Video Workshop opcional.	Campo opcional disponible para almacenar un recurso Workshop asociado al producto.
productos_del_bundle	Relaciona un bundle con otros productos.	La tarjeta del bundle recorre los artworks asociados.
tiene_globo	Indica si el artwork admite texto de globo.	Si está activo, Cart muestra Comic Balloon Text.

Identificación de bundles

Los bundles se identifican mediante la categoría Bundles y utilizan productos_del_bundle para relacionar los artworks que forman cada paquete.

ACF utiliza además una clave interna que comienza con “_” para enlazar el valor con la definición del campo. Por ejemplo, video_destacado contiene el ID del archivo y _video_destacado apunta a la definición ACF correspondiente.

```

video_destacado = 989
_video_destacado = field_68422c183110a

tiene_globo     = 1
_tiene_globo     = field_6888eb6010b7a

```

5.6 Videos y archivos adjuntos

Los archivos MP4 no se guardan dentro de la fila del producto. video_destacado almacena una referencia a un attachment de WordPress. Ese attachment mantiene la ruta del archivo físico dentro de wp-content/uploads. El campo ACF está configurado con return_format = url, por lo que get_field() entrega al tema una URL utilizable.

```

PRODUCTO — wp_posts.ID = 988
↓
wp_postmeta: video_destacado = 989
↓
ATTACHMENT — wp_posts.ID = 989
post_type = attachment · mime = video/mp4
↓
_wp_attached_file / _wp_attachment_metadata
↓
archivo MP4 dentro de wp-content/uploads

```

```
$video_url = get_field(
    'video_destacado',
    $product->get_id()
);
```

5.7 Categorías y taxonomías

WooCommerce organiza el catálogo mediante las taxonomías de WordPress. Un producto puede pertenecer a varias categorías; por eso las cantidades siguientes representan relaciones y no deben sumarse para obtener el total del catálogo.

Categoría	Productos relacionados
Characters	307
Bundles	134
Fantasy	131
Cyberpunk	76
Universes	73
Anime	40

5.8 Cómo se representa un bundle

Un bundle no es un producto WooCommerce “grouped”. Los 134 bundles son productos simples pertenecientes a la categoría Bundles. El campo ACF productos_del_bundle guarda la relación con los artworks que se muestran dentro de su tarjeta.

```
PRODUCT SIMPLE
category = Bundles
↓
productos_del_bundle
├── Product #1086
├── Product #1089
└── Product #1092
```

Cada bundle tiene actualmente tres productos relacionados y las 402 referencias apuntan a productos existentes. Esto describe la composición actual del catálogo; la configuración ACF no establece un máximo técnico de tres.

5.9 Sesión y carrito

Antes de crear un pedido, WooCommerce mantiene el carrito dentro de su sistema de sesiones. El tema añade comic_balloon_text y color_frame a cada cart item. Estos datos son temporales: sobreviven al cambio de página, pero pasan a ser permanentes recién cuando se copian al pedido.

```
Cart item
├── product_id
├── comic_balloon_text
└── color_frame
    ↓
wp_woocommerce_sessions
    ↓
Checkout
```

5.10 Pedidos y HPOS

La instalación tiene activado High-Performance Order Storage (HPOS): woocommerce_custom_orders_table_enabled = yes. Los pedidos se almacenan en tablas específicas de WooCommerce. El tema usa objetos WC_Order y funciones como wc_get_order(), de modo que no necesita consultar esas tablas directamente.

```
$order = wc_get_order($order_id);
$artworks_name = $order->get_meta('Artworks Name');
```

Checkout



WC_Order / API CRUD de WooCommerce

```
├── wp_wc_orders
├── wp_wc_order_addresses
└── wp_wc_orders_meta
```



wp_woocommerce_order_items



wp_woocommerce_order_itemmeta

5.11 Dato general del pedido y datos por línea

Artworks Name pertenece al pedido completo. Frame Color y Comic Balloon Text pertenecen a una línea concreta, porque dos productos del mismo pedido pueden tener personalizaciones diferentes.

Dato	Nivel	Persistencia
Artworks Name	Pedido completo	wp_wc_orders_meta
First Name / Email	Datos de facturación	wp_wc_order_addresses / wp_wc_orders
Frame Color	Línea de producto	wp_woocommerce_order_itemmeta
Comic Balloon Text	Línea de producto	wp_woocommerce_order_itemmeta

5.12 Composición actual de la base de datos

Dato	Valor
Motor de base de datos	MariaDB 11.8.6
Tablas	74
Productos publicados	627
Tipo de producto	627 productos simples
Artworks fuera de Bundles	493
Bundles	134
tiene_globo = Sí / No	55 / 572
Attachments	544 (540 MP4, 3 PNG, 1 WebP)
HPOS	Activado
Pedidos registrados en HPOS	6
Método de pago	manual_pay
Estado de pedidos manuales	wc-pending
Moneda	USD

Dato	Valor
Datos de contenido Estas cifras describen el contenido actual de la instalación y no representan límites del sistema. El catálogo y la cantidad de pedidos pueden crecer sin modificar este modelo de datos.	

5.13 Cómo accede el tema a estos datos

A diferencia de un proyecto que consulta SQL manualmente, MC Artworks Studio utiliza las APIs de WooCommerce y ACF. Esto mantiene el código desacoplado del almacenamiento físico y es especialmente importante con HPOS.

```
$product->get_id();
$product->get_price();
get_field('video_destacado', $product->get_id());
wc_get_products([...]);
wc_get_order($order_id);
```

6. Estructura del proyecto

MC Artworks Studio se organiza sobre una instalación de WordPress. El código propio se concentra en el tema hijo blocksy-child, mientras WordPress, Blocksy, WooCommerce, los plugins y wp-content/uploads completan el entorno de ejecución del proyecto.

6.1 Organización general

```
public_html/
├── wp-content/
│   ├── themes/
│   │   ├── blocksy/           ← tema padre
│   │   └── blocksy-child/    ← código propio
│   ├── plugins/
│   │   ├── WooCommerce
│   │   ├── Advanced Custom Fields
│   │   ├── HUSKY / WOOF
│   │   ├── Xootix Side Cart
│   │   ├── WP Mail SMTP
│   │   ├── All-In-One Security (AIOS)
│   │   └── otros plugins
│   └── uploads/
│       ├── categories/
│       ├── images/
│       ├── videos/
│       ├── songs/
│       ├── sounds/
│       ├── logs/
│       ├── wc-logs/
│       ├── woocommerce_uploads/
│       └── otras carpetas de WordPress/plugins
```

Las carpetas images, videos, songs y sounds se encuentran físicamente dentro de wp-content/uploads. La carpeta categories organiza parte de los videos por categoría y personaje; por ejemplo, uploads/categories/characters/chainsaw-man/ contiene archivos MP4 asociados a ese grupo.

6.2 Tema hijo blocksy-child

Archivo	Para qué se usa
style.css	Declara el tema hijo y su relación con Blocksy.
functions.php	Concentra la mayor parte de la lógica: Inicio, Tienda, Bundles, carrito, checkout, pago manual, correos, logs, Galería y la sección de servicios.

Archivo	Para qué se usa
header.php	Construye el menú principal tipo HUD, sus videos, sonidos, navegación y persistencia del audio.
animations.php	Contiene ocho animaciones de presentación: cuatro GDI y cuatro NOD.
hud-intro.php	Construye la introducción multimedia con dos videos, música y controles.
faction-selection.php	Implementa la selección NOD/GDI y guarda la elección en cookies.
gdi-faction.css / gdi-faction-min.css	Estilos de la variante GDI.
nod-faction.css / nod-faction-min.css	Estilos de la variante NOD.
scrin-faction.css	Hoja de estilos presente en el tema, aunque Scrin no participa del flujo activo de selección.

6.3 Código, datos y recursos

Zona	Contenido
wp-content/themes/blocksy-child/	PHP y CSS desarrollado específicamente para MC Artworks Studio.
MariaDB	Productos, ACF, categorías, sesiones, configuración y pedidos HPOS.
wp-content/uploads/categories/	Recursos multimedia organizados por categorías/personajes.
wp-content/uploads/images/	Imágenes utilizadas por la interfaz.
wp-content/uploads/videos/	Videos generales del sitio.
wp-content/uploads/songs/	Música de la experiencia audiovisual.
wp-content/uploads/sounds/	Efectos de sonido.
wp-content/uploads/logs/	Logs propios generados por el gateway manual.

6.4 Concentración de la lógica en functions.php

MC Artworks Studio no separa cada pantalla en un archivo page-*.php. Gran parte del comportamiento se activa desde functions.php mediante hooks y comprobaciones de contexto como is_shop(), is_cart(), is_checkout() e is_page(). Por eso páginas como Bundles, Gallery u Our Services se construyen mediante esa lógica centralizada y no mediante archivos page-bundles.php, page-gallery.php o page-our-services.php independientes.

```

WordPress solicita una página
↓
functions.php comprueba el contexto
↓
is_shop() · is_cart() · is_checkout() · is_page(...)
↓
se ejecutan hooks, HTML, CSS y JavaScript
correspondientes a esa pantalla

```

6.5 Código propio, dependencias y recursos

Código y configuración del proyecto	Entorno y recursos relacionados
PHP y CSS del tema hijo	WordPress Core y Blocksy
Menú, intro y selector de facción	WooCommerce, ACF y demás plugins

Código y configuración del proyecto	Entorno y recursos relacionados
Lógica personalizada de tienda, carrito y checkout	Base MariaDB y contenido administrado
Estilos GDI/NOD/Scrin	wp-content/uploads con videos, imágenes, música y sonidos
Organización centralizada functions.php supera las 8.800 líneas y funciona como el principal punto de integración de la Tienda, Bundles, carrito, checkout, pago manual, correos, logs, Galería y servicios. Esta centralización forma parte de la arquitectura actual del proyecto.	

7. Qué hace WordPress, Blocksy y WooCommerce

WordPress recibe la dirección solicitada y carga el tema. Blocksy aporta la base sobre la que trabaja el sitio y blocksy-child contiene las modificaciones propias de MC Artworks Studio. WooCommerce se ocupa de los productos, precios, carrito, checkout y pedidos.

Qué significa tema padre y tema hijo

Un tema padre aporta una base reutilizable. Un tema hijo permite modificar esa base sin editar directamente los archivos del padre. En este proyecto Blocksy es el tema padre y blocksy-child contiene el código personalizado. Por ejemplo, el menú audiovisual vive en header.php del tema hijo: no es necesario modificar header.php de Blocksy para construirlo.

Qué es un hook

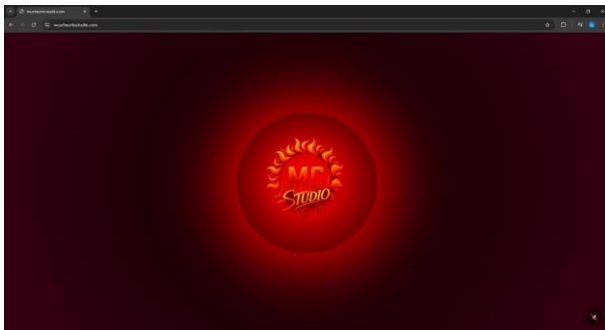
WordPress y WooCommerce ofrecen puntos concretos donde un tema o plugin puede ejecutar código. Esos puntos se llaman hooks. Permiten agregar o modificar comportamiento sin editar el núcleo de WordPress o WooCommerce. Por ejemplo, en la Tienda se utiliza un hook de WooCommerce para colocar el video de cada artwork antes del nombre del producto.

PARTE II — ENTRADA, FACCIÓNES Y NAVEGACIÓN

8. Secuencia inicial y animaciones

La secuencia inicial es una presentación previa a la Intro que se utiliza cuando el visitante todavía no tiene una facción elegida. No se trata de un único video ni de una animación cargada desde un archivo externo: `functions.php` decide si corresponde mostrarla y `animations.php` contiene ocho funciones PHP independientes que generan la experiencia visual en el navegador.

Para entenderla conviene separar dos responsabilidades. WordPress y PHP toman la decisión en el servidor; la función de animación elegida imprime HTML, CSS y JavaScript. Esa respuesta contiene temporizadores y redirecciones del lado del navegador, pero `insertar_presentacion()` también intenta un `wp_redirect()` del lado del servidor. Por eso la salida efectiva se analiza en 8.8 y no debe suponerse que todos los temporizadores llegan siempre a ejecutarse.



Variación roja



Variación violeta

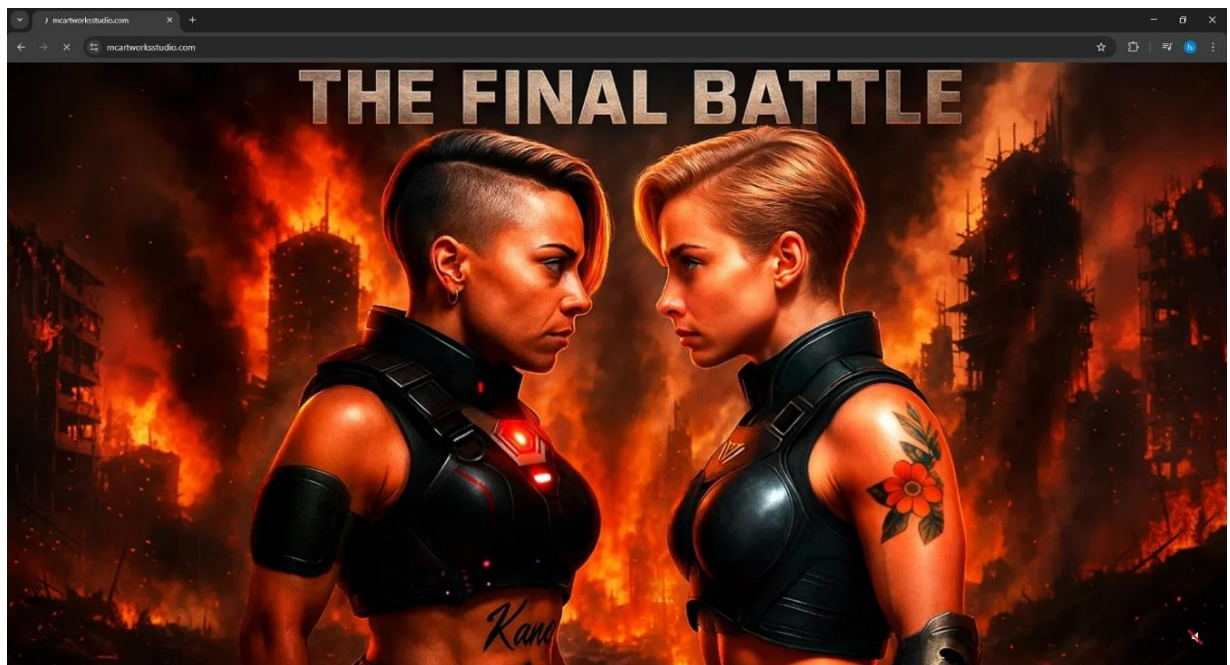


Imagen de presentación previa a la introducción multimedia.

8.1 Dónde se activa la secuencia

functions.php registra insertar_presentacion() en el hook template_redirect. Ese hook ocurre antes de que WordPress cargue la plantilla final, por lo que el código todavía puede decidir si deja continuar la petición normal o si interviene en el flujo de entrada. El fragmento siguiente está abreviado: conserva las condiciones reales de activación, pero omite el cuerpo completo donde se selecciona la variante, se imprime la cobertura y se intenta la salida hacia Intro.

```
add_action('template_redirect', 'insertar_presentacion');

function insertar_presentacion() {
    if (is_page(['intro', 'faction-selection'])) return;

    if (empty($_COOKIE['faction'])) {
        if (is_page() || is_single() || is_archive() ||
            (function_exists('is_woocommerce') && is_woocommerce()) ||
            is_shop() || is_product() || is_product_category() || is_product_tag()) {
            // Fragmento abreviado: aquí continúa la selección,
            // la salida visual y la redirección.
        }
    }
}
```

La ausencia de la cookie faction es necesaria, pero no es la única condición. La función no actúa dentro de /intro/ ni /faction-selection/ y sólo continúa en los contextos de frontend contemplados por el código: páginas, entradas, archivos y varias pantallas de WooCommerce.

Ubicación en el proyecto: wp-content/themes/blocksy-child/functions.php · hook template_redirect alrededor de la línea 220 · insertar_presentacion() aproximadamente entre las líneas 230 y 324.

8.2 Cómo functions.php hace disponibles las ocho animaciones

Antes de que insertar_presentacion() pueda elegir una variante, el tema hijo carga animations.php. require_once incorpora el archivo una sola vez durante esa petición y deja definidas sus funciones. Cargar el archivo no significa ejecutar todas las animaciones: sólo vuelve disponibles sus nombres para que una de ellas pueda invocarse más tarde.

```
if (file_exists(get_stylesheet_directory() . '/animations.php')) {
    require_once get_stylesheet_directory() . '/animations.php';
}
```

WordPress carga blocksy-child
 ↓
 functions.php ejecuta require_once animations.php
 ↓
 quedan definidas ocho funciones de animación
 ↓
 template_redirect llega a insertar_presentacion()
 ↓
 recién allí se elige una de esas funciones

8.3 Las ocho funciones disponibles

animations.php contiene cuatro variantes GDI y cuatro variantes NOD. Todas comparten la idea de una presentación fullscreen, pero cada función construye su propio conjunto de capas, efectos, sonidos y temporización.

Facción	Función PHP	Responsabilidad común
GDI	insertar_animacion_apocalipsis_GDI_UNO()	Genera una presentación fullscreen GDI y programa su salida.
GDI	insertar_animacion_apocalipsis_GDI_DOS()	Genera una segunda variante visual GDI.
GDI	insertar_animacion_apocalipsis_GDI_TRES()	Genera una tercera variante visual GDI.

GDI	insertar_animacion_apocalipsis_GDI_CUATRO()	Genera una cuarta variante visual GDI.
NOD	insertar_animacion_apocalipsis_NOD_UNO()	Genera una presentación fullscreen NOD y programa su salida.
NOD	insertar_animacion_apocalipsis_NOD_DOS()	Genera una segunda variante visual NOD.
NOD	insertar_animacion_apocalipsis_NOD_TRES()	Genera una tercera variante visual NOD.
NOD	insertar_animacion_apocalipsis_NOD_CUATRO()	Genera una cuarta variante visual NOD.

Ubicación en el proyecto: wp-content/themes/blocksy-child/animations.php · las ocho funciones ocupan prácticamente todo el archivo; la primera comienza al inicio y las restantes se distribuyen a lo largo de más de 2.300 líneas.

8.4 Cómo se elige una animación

La selección ocurre en dos pasos. Primero PHP decide una facción provisional con `rand(0,1)`. Después, dentro del grupo elegido, `array_rand()` selecciona uno de los cuatro índices disponibles. El resultado final no es todavía una animación ejecutada: es el nombre de una función guardado en la variable `$funcion`.

```
$faccion = rand(0,1) ? 'GDI' : 'NOD';

$animaciones = [
    'GDI' => [
        'insertar_animacion_apocalipsis_GDI_UNO',
        'insertar_animacion_apocalipsis_GDI_DOS',
        'insertar_animacion_apocalipsis_GDI_TRES',
        'insertar_animacion_apocalipsis_GDI_CUATRO',
    ],
    'NOD' => [
        'insertar_animacion_apocalipsis_NOD_UNO',
        'insertar_animacion_apocalipsis_NOD_DOS',
        'insertar_animacion_apocalipsis_NOD_TRES',
        'insertar_animacion_apocalipsis_NOD_CUATRO',
    ]
];

$funcion = $animaciones[$faccion][array_rand($animaciones[$faccion])];

if (function_exists($funcion)) {
    $funcion();
}
```

La llamada `$funcion()`; es una llamada dinámica: PHP toma el texto guardado en la variable y ejecuta la función que tiene exactamente ese nombre. `function_exists()` funciona como una comprobación previa para no intentar invocar una función inexistente.

```
rand(0,1)
↓
GDI o NOD
↓
array de 4 nombres de función
↓
array_rand() elige un índice
↓
$funcion contiene el nombre elegido
↓
function_exists($funcion)
↓
$funcion() ejecuta una sola variante
```

8.5 Cómo se genera realmente una animación

Cada variante de `animations.php` es una función PHP que imprime su propio bloque de HTML, CSS y JavaScript. Por eso la presentación no llega al navegador como un “archivo de animación” terminado. PHP arma la estructura y algunos valores; después el navegador aplica estilos, crea elementos adicionales y reproduce la secuencia.

La primera variante GDI permite ver esta mezcla con claridad. La función bloquea el scroll con `hud-lock`, crea un contenedor fullscreen, genera chispas desde PHP, define humo, ondas, fragmentos, flashes y shake mediante CSS, incluye audios y finalmente registra JavaScript que coordina la secuencia.

```
<?php for ($i = 0; $i < 60; $i++): ?>
    <div class="spark"
        style="top:<? rand(0,100) ?>%;
            left:<? rand(0,100) ?>%;
            animation-duration:<? rand(1,4) ?>s;">
    </div>
<?php endfor; ?>
```

En este fragmento PHP genera sesenta elementos `.spark` antes de enviar la respuesta. Cada uno recibe una posición y una duración distintas mediante `rand()`. El navegador recibe esas sesenta etiquetas ya construidas y luego CSS anima cada una.

```
const wave = document.createElement('div');
wave.className = 'wave';
wave.style.animationDuration = `${1.5 + Math.random()*0.5}s`;
compuerta.appendChild(wave);

const frag = document.createElement('div');
frag.className = 'fragment';
frag.style.setProperty('--x', `${(Math.random()-0.5)*2000}px`);
frag.style.setProperty('--y', `${(Math.random()-0.5)*1200}px`);
```

Aquí la aleatoriedad ya ocurre del lado del navegador. JavaScript crea ondas y fragmentos después de que la página fue recibida. Es decir: una misma función seleccionada puede producir pequeñas variaciones visuales entre ejecuciones porque parte de los valores se sortean en PHP y otra parte se calcula con `Math.random()` en JavaScript.

8.6 PHP y JavaScript no hacen la misma parte

Momento	Mecanismo	Ejemplo real
Servidor, antes de enviar la página	PHP	Selecciona GDI/NOD, elige la función, genera 60 sparks y escribe HTML/CSS/JS.
Navegador, después de recibir la respuesta	JavaScript	Espera <code>DOMContentLoaded</code> , crea waves/fragments, reproduce audio y ejecuta <code>setTimeout()</code> .
Presentación visual	CSS	Los <code>@keyframes</code> controlan pulse, waveExpand, shake, flash y otras transiciones.

Idea clave

PHP no “reproduce” la animación en el servidor. PHP construye una respuesta que contiene los elementos y scripts necesarios; la animación visible ocurre en el navegador.

8.7 Temporización, sonido y bloqueo de la interfaz

Las funciones utilizan `setTimeout()` para encadenar momentos concretos. En `GDI_UNO`, por ejemplo, el logo comienza a aparecer después de unos 400 ms; varias ondas se disparan desde aproximadamente 1,2 s; a los 4 s se activa una explosión con shake, sonido y fragmentos; y más adelante comienza el fade final. Otras variantes tienen sus propias secuencias.

La primera variante también crea un botón de sonido. El estado inicial `soundEnabled = false` evita reproducir los efectos hasta que el visitante active el audio. Al silenciar nuevamente, el código pausa y reinicia los elementos

<audio> presentes en el DOM. Mientras la presentación ocupa la pantalla, hud-lock bloquea el scroll y oculta temporalmente elementos del side cart para que no interfieran con la experiencia fullscreen.

8.8 Cobertura final y salida hacia Intro

Después de ejecutar la función elegida, insertar_presentacion() imprime una segunda capa general llamada #transition-cover. La imagen se construye a partir de uploads/images/image7.jpg. El JavaScript de esa capa programa su aparición a los 4 segundos y una navegación a /intro/ a los 6 segundos. Sin embargo, ese no es el único mecanismo de salida: inmediatamente después de imprimir el bloque, PHP intenta también un wp_redirect() y ejecuta exit. El fragmento siguiente reúne esos dos tramos del mismo código; se omiten el CSS y el HTML de la cobertura.

```
setTimeout(function() {
    const cover = document.getElementById('transition-cover');
    cover.classList.add('show');
}, 4000);

setTimeout(function() {
    window.location.href = "<?php echo home_url('/intro'); ?>";
}, 6000);

// Después de imprimir el overlay y el script:
wp_redirect(home_url('/intro'));
exit;
```

Las propias funciones de animations.php también contienen redirecciones temporizadas hacia /intro/. Por lo tanto, el código actual superpone tres estrategias: temporizadores de la variante, temporizadores de #transition-cover y un wp_redirect() del servidor. Si PHP todavía puede emitir la cabecera Location —por ejemplo, porque existe buffering y las cabeceras no fueron enviadas— el navegador puede recibir la redirección HTTP antes de llegar a ejecutar la presentación completa. Si las cabeceras ya no pueden modificarse por la salida previa, esa redirección del servidor puede no surtir efecto y adquieren relevancia las rutas JavaScript. No existe, entonces, una única secuencia temporal garantizada para todos los entornos.

Detalle de implementación actual

El comentario situado sobre insertar_presentacion() todavía menciona una cookie visited_intro, pero la condición ejecutada por el código comprueba realmente \$_COOKIE['faction']. Para describir el comportamiento se toma como fuente la condición efectiva, no el comentario desactualizado.

8.9 Recorrido técnico completo

```
WordPress recibe una petición de frontend
↓
functions.php ya cargó animations.php
↓
template_redirect ejecuta insertar_presentacion()
↓
se excluyen /intro/ y /faction-selection/
↓
¿existe cookie faction?
↓ no
se valida el contexto de página
↓
rand() elige GDI/NOD
↓
array_rand() elige una de 4 funciones
↓
$funcion() imprime HTML + CSS + JavaScript
↓
se imprimen #transition-cover + temporizadores
↓
PHP intenta wp_redirect('/intro') + exit
↓
¿la cabecera Location puede emitirse?
├ sí → el navegador puede ir a Intro antes de completar los timers
└ no → quedan las redirecciones JavaScript de la respuesta
↓
/intro/
```

8.10 Ejemplo guiado: primera visita de Alex

Ahora que el mecanismo está definido, se puede seguir una visita concreta. Alex abre /shop/ por primera vez y su navegador todavía no tiene la cookie faction.

1. WordPress carga blocksy-child. functions.php incorpora animations.php, pero todavía no muestra ninguna variante.
2. Al llegar a template_redirect, WordPress llama a insertar_presentacion(). La función verifica que Alex no esté en Intro ni en Faction Selection y confirma que Shop es un contexto admitido.
3. \$_COOKIE['faction'] está vacío. PHP ejecuta rand(0,1). Supongamos que el resultado elige GDI.
4. Dentro del array GDI, array_rand() devuelve uno de los cuatro índices. Supongamos que queda seleccionada insertar_animacion_apocalipsis_GDI_UNO().
5. function_exists() confirma que la función está cargada y \$funcion() la ejecuta. PHP genera el contenedor fullscreen, las chispas, los audios y los scripts de la variante.
6. La función elegida imprime el HTML, CSS y JavaScript de la variante; insertar_presentacion() imprime además #transition-cover y sus temporizadores. Antes de terminar la ejecución PHP intenta wp_redirect('/intro/') y llama a exit.
7. Si la cabecera HTTP Location puede enviarse, el navegador de Alex puede saltar a /intro/ antes de reproducir por completo los temporizadores del cuerpo. Si esa redirección del servidor no puede aplicarse porque la salida previa ya comprometió las cabeceras, el navegador puede ejecutar la presentación y las redirecciones JavaScript. En ambos caminos el destino pretendido es /intro/, pero el código actual no garantiza que la animación visible dure siempre la misma cantidad de tiempo.

8.11 Qué no debe confundirse con esta elección

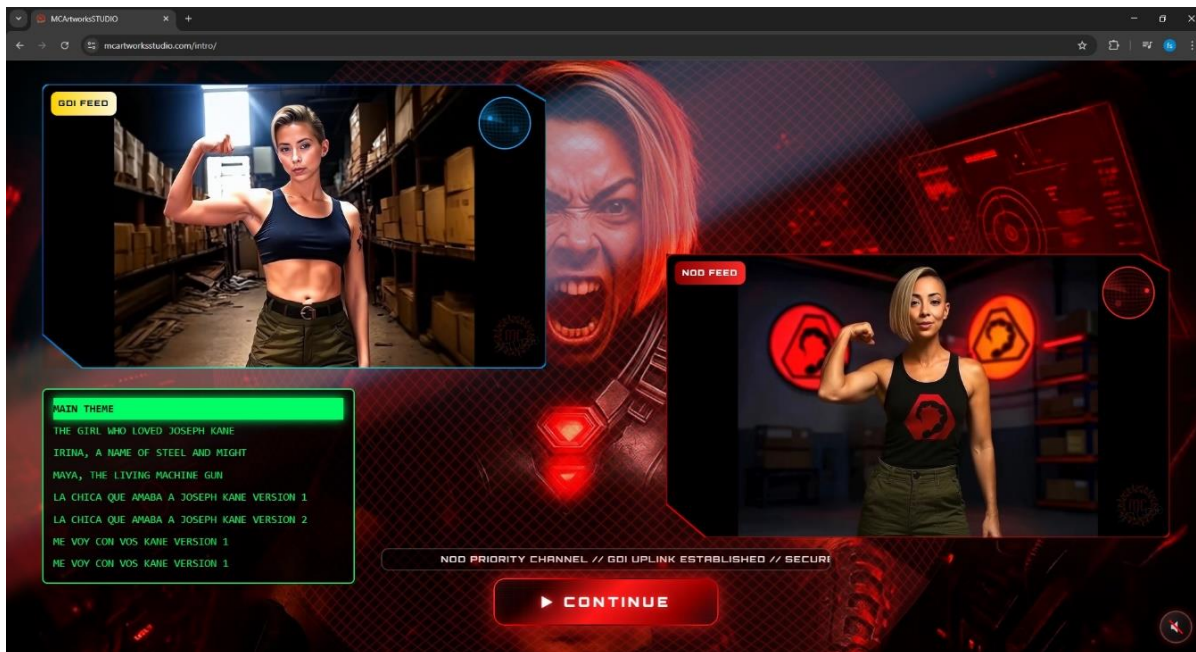
Importante

Que la presentación haya sido GDI o NOD no fija la identidad del visitante. Esta elección sólo decide qué animación se muestra en la entrada. La facción persistente se crea más adelante, cuando el visitante confirma YES dentro de faction-selection.php y se escribe la cookie faction.

9. Introducción multimedia y HLS

La página intro carga hud-intro.php, un módulo audiovisual de más de 1.500 líneas que construye la transición entre la animación inicial y la selección de facción. Combina PHP, HTML, CSS y JavaScript con dos feeds HLS, videos MP4, música, efectos de sonido, compuertas, radares, grillas generadas dinámicamente y un flujo propio de navegación.

Ubicación en el proyecto: wp-content/themes/blocksy-child/hud-intro.php · WordPress lo incorpora desde functions.php y el archivo genera la interfaz de Intro, los dos feeds, el reproductor musical y la lógica JavaScript de inicialización. El punto de entrada está registrado desde functions.php mediante wp_head.



Introducción multimedia: dos videos, paneles de información, música y botón para continuar.

9.1 Integración con WordPress

La introducción se carga de forma condicional. `functions.php` comprueba que la página actual sea `intro`; recién entonces incorpora `hud-intro.php` y ejecuta `nod_gdi_hud_intro()`. De esta manera el código, los estilos y la interfaz del HUD no se generan en el resto del sitio.

```
function hud_intro() {
    $hud_intro = get_page_by_path('intro');
    if (!$hud_intro || !is_page($hud_intro->ID)) return;
    require_once get_stylesheet_directory() . '/hud-intro.php';
    if (function_exists('nod_gdi_hud_intro')) nod_gdi_hud_intro();
}
```

```
WordPress carga una página
↓
hud_intro() comprueba el slug
↓
¿es /intro/?
↓ sí
require_once hud-intro.php
↓
nod_gdi_hud_intro()
```

9.2 Composición de la pantalla

`hud-intro.php` construye un HUD fijo que ocupa toda la ventana y bloquea el desplazamiento mientras está activo. La pantalla está compuesta por varias capas independientes: siete fondos animados, lente y grillas curvas, dos feeds de video, compuertas GDI/NOD, radares, un panel musical, una inicialización de 0 a 100 %, ticker, sonidos y el botón CONTINUE.

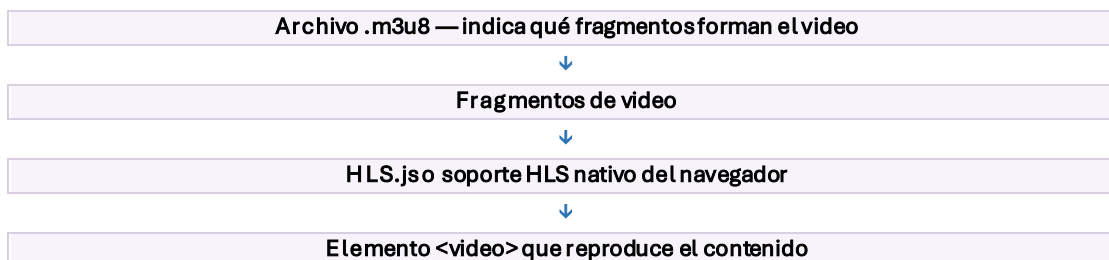
```

HUD INTRO
├─ fondos image1.jpg ... image7.jpg
├─ lente y grillas curvas
├─ GDI FEED → HLS + MP4 de respaldo
├─ NOD FEED → HLS + MP4 de respaldo
├─ compuertas y radares
├─ selector musical + hud-audio
├─ inicialización 0-100 %
├─ ticker
└─ CONTINUE + control global de sonido

```

9.3 Qué es HLS

HLS significa HTTP Live Streaming. En lugar de entregar un video como un único archivo grande, utiliza una lista .m3u8 que apunta a fragmentos más pequeños. El reproductor solicita esos fragmentos a medida que los necesita. En este proyecto se usa HLS tanto en la introducción como en la página de Inicio.



9.4 Carga de video según el navegador

Código fuente: wp-content/themes/blocks-child/hud-intro.php → nod_gdi_hud_intro() → bloque JavaScript ejecutado en DOMContentLoaded alrededor de la línea 1032. Ese bloque detecta Firefox/Chromium, define las rutas GDI/NOD y deja preparados los elementos que usará loadVideo().

hud-intro.php distingue Firefox de navegadores basados en Chromium, como Chrome y Opera, y ajusta las rutas y la configuración de HLS.js. También intenta mantener sincronizados los dos videos: si la diferencia de tiempo supera un pequeño margen, lleva ambos al tiempo más bajo para evitar que uno avance claramente más que el otro.

Ejemplo

Si GDIvideo va por 12,40 segundos y NODvideo por 12,30, la diferencia es pequeña. Si se separan más de 0,05 s, el código puede llevar ambos al punto común para evitar que una pantalla avance claramente más que la otra.

```

const isFirefox = typeof InstallTrigger !== 'undefined';
const isChromium = !!window.chrome;

const videos = isFirefox
  ? { GDIvideo: '/videos/gdi-intro-firefox/GDI-INTRO.m3u8',
      NODvideo: '/videos/nod-intro-firefox/NOD-INTRO.m3u8' }
  : { GDIvideo: '/wp-content/uploads/videos/GDI-INTRO/GDI-INTRO.m3u8',
      NODvideo: '/wp-content/uploads/videos/NOD-INTRO/NOD-INTRO.m3u8' };

```

9.5 Carga con HLS.js y preparación simultánea

Código fuente: hud-intro.php → mismo callback DOMContentLoaded de nod_gdi_hud_intro() (aprox. línea 1032) → loadVideo(), Hls.loadSource()/attachMedia() y Promise.all(). Se ejecuta en el navegador cuando el DOM de Intro ya existe; PHP sólo imprimió el script durante la respuesta.

La función loadVideo() conecta cada archivo .m3u8 con su elemento <video>. Cuando HLS.js está disponible utiliza loadSource() y attachMedia(); si el navegador admite HLS de forma nativa, asigna el archivo directamente al atributo src. La configuración de HLS.js también cambia entre Firefox y Chromium.

```
const hls = new Hls({
  enableWorker: false,
  lowLatencyMode: false,
  backBufferLength: 1,
  maxBufferLength: 10,
  maxMaxBufferLength: 20
});
hls.loadSource(src);
hls.attachMedia(vid);
```

Los dos feeds se preparan con `Promise.all()`. Después ambos vuelven al segundo 0, se espera un ciclo de renderizado mediante `requestAnimationFrame()` y se intenta iniciar su reproducción. Así GDI y NOD arrancan a partir de un estado coordinado.

```
await Promise.all(
  Object.entries(videos).map(([id, src]) => loadVideo(id, src))
);
Object.values(elements).forEach(v => { v.currentTime = 0; v.pause(); });
await new Promise(r => requestAnimationFrame(r));
Object.values(elements).forEach(v => v.play().catch(() => {}));
```

9.6 Sincronización de GDI y NOD

Código fuente: hud-intro.php → mismo callback `DOMContentLoaded` (aprox. línea 1032) → `setInterval` de sincronización. Recibe `GDIvideo` y `NODvideo` ya preparados y corrige `currentTime` cuando la diferencia absoluta supera 0,05 s.

Aunque ambos videos se inicien juntos, el buffering y la decodificación pueden provocar una pequeña diferencia de tiempo. El código comprueba periódicamente los dos `currentTime` y, si la separación supera 0,05 segundos, lleva ambos al menor tiempo observado. En Chromium la comprobación se realiza aproximadamente cada 1,5 segundos y en los demás casos cada 2 segundos.

```
const diff = v1.currentTime - v2.currentTime;
if (Math.abs(diff) > 0.05) {
  const target = Math.min(v1.currentTime, v2.currentTime);
  v1.currentTime = target;
  v2.currentTime = target;
}
```

Ejemplo técnico

Si GDI está en 12,40 s y NOD en 12,31 s, la diferencia es 0,09 s. El código toma 12,31 s como punto común para que ninguno de los dos feeds continúe claramente adelantado.

9.7 Video de respaldo y MutationObserver

Código fuente: hud-intro.php → callback `DOMContentLoaded` aproximadamente entre las líneas 1472-1507 → listener `canplay` de cada feed + `MutationObserver` sobre la compuerta correspondiente. El observer reacciona a cambios de clase del DOM; no consulta el servidor por sí mismo.

Cada feed HLS tiene también un MP4 de respaldo. El fallback permanece disponible mientras el stream principal todavía no puede reproducirse. Cuando el evento `canplay` confirma que HLS está preparado, el MP4 se oculta. Un `MutationObserver` vigila además la clase de la compuerta: si ésta se abre antes de que el stream tenga datos suficientes, mantiene visible el video alternativo en lugar de mostrar un espacio vacío.

```
video.addEventListener('canplay', () => {
  fallback.style.display = 'none';
});

const observer = new MutationObserver(() => {
  if (door.classList.contains('open') && video.readyState < 3)
    fallback.style.display = 'block';
});
```

9.8 Inicialización, compuertas y música

Código fuente: hud-intro.php → JavaScript principal de `nod_gdi_hud_intro()` → IIFE `init(ms = 3000)`, aproximadamente desde la línea 1177. El temporizador actualiza porcentaje/barra y, al llegar a 100 %, habilita el tramo de compuertas, audio y CONTINUE.

La barra 0–100 % es una inicialización visual temporizada de aproximadamente tres segundos. Cada 40 ms se recalcula el porcentaje. Al llegar a 100 % se detiene el beep de carga, se abren las compuertas NOD/GDI, se reproduce el efecto de puerta, se aplica un breve shake a los paneles, se habilita CONTINUE y se activa el panel de música.

```
const pct = Math.min(100, Math.round((elapsed / ms) * 100));
percent.textContent = pct + '%';
bar.style.width = pct + '%';

if (pct >= 100) {
  nodDoor.classList.add('open');
  gdiDoor.classList.add('open');
  btn.classList.add('ready');
}
```

El reproductor musical utiliza un único elemento `hud-audio` y varias opciones con `data-src`. El objeto `trackPositions` recuerda el `currentTime` de cada pista durante la estancia en Intro, de modo que al cambiar de tema y volver al anterior puede continuar desde el punto guardado. Los cambios se acompañan con un fade de volumen.

9.8.1 Cómo se habilita y utiliza el selector musical

Código fuente: hud-intro.php → HTML del panel `#hud-music-panel` y `<audio id="hud-audio">` alrededor de las líneas 827-837; sus listeners JavaScript comienzan en el callback `DOMContentLoaded` cercano a la línea 855. Las pistas visibles son controles que cambian la fuente de un único elemento `<audio>`.

El panel musical no está activo desde el primer instante. Al comenzar Intro, JavaScript deshabilita sus clics con `pointerEvents = none` y reduce su opacidad. Cuando la inicialización llega al 100 %, las compuertas se abren y, unos instantes después, el panel vuelve a aceptar interacción.

Las ocho opciones visibles no son ocho elementos `<audio>`. Cada opción es un `<div class="track">` con una ruta `data-src`. Todas comparten un único reproductor, `<audio id="hud-audio">`. Cambiar de canción significa cambiar la fuente utilizada por ese mismo elemento.

```
<div class="track" data-src="/songs/Main_Theme">MAIN THEME</div>
<div class="track" data-src="/songs/The_Girl_Who_Loved_Joseph_Kane">THE GIRL WHO LOVED JOSEPH KANE</div>
...
<audio id="hud-audio" preload="none" loop playsinline></audio>
```

MAIN THEME se prepara como pista inicial y queda marcada como active. Ese bloque asigna su `data-src` a `hud-audio`, configura `loop`, deja `audio.muted = true` y llama a `load()`. También hace un intento de `play()` seguido inmediatamente por `pause()` y `currentTime = 0` para forzar una preparación inicial del recurso. Esto no equivale a dejar la canción reproduciéndose de forma continua.

Detalle de implementación actual

El control global de sonido declara `let isMuted = true` dentro de un callback `DOMContentLoaded` y expone `window.setGlobalMute()` / `window.setMainReady()`. La inicialización 0–100 %, en cambio, vive en otro IIFE. Al llegar al 100 %, ese segundo bloque ejecuta más adelante `audio.muted = isMuted`, pero `isMuted` no está definido en ese ámbito. En el código fuente actual esa línea puede producir `ReferenceError: isMuted is not defined`. Antes de alcanzarla, la inicialización ya comprueba `mainAudio.muted`: si está desmuteado intenta reproducirlo; si sigue muteado lo pausa y vuelve a `currentTime = 0`. Por eso la reproducción automática de MAIN THEME al 100 % no debe documentarse como una secuencia garantizada.

Detalle de la lista de pistas

La última opción visible contiene `data-src="/songs/Me_Voy_con_Vos_Kane_VERSION_2"`, pero su texto vuelve a decir ME VOY CON VOS KANE VERSION 1. Es una inconsistencia literal del código fuente actual: las rutas `VERSION_1` y `VERSION_2` son distintas aunque las dos últimas etiquetas visibles muestran VERSION 1.

Los listeners de clic de las pistas y el objeto `trackPositions` se registran en otro callback `DOMContentLoaded`. Por eso el mecanismo manual de selección y recuperación de posición explicado a continuación es independiente de esa referencia fuera de alcance; la inconsistencia afecta específicamente al tramo automático de inicialización del MAIN THEME.

9.8.2 Cómo recuerda la posición de cada canción

Código fuente: hud-intro.php → callback DOMContentLoaded aproximadamente entre las líneas 855-929 → trackPositions, evento timeupdate, listeners click de .track y evento loadedmetadata. Ese estado vive en memoria del navegador durante la estancia actual en Intro.

trackPositions es un objeto JavaScript que asocia cada ruta de canción con su currentTime. El evento timeupdate actualiza continuamente la posición del tema activo. Cuando el visitante elige otra pista, antes de cambiar la fuente se vuelve a guardar la posición de la anterior.

```
const trackPositions = {};

audio.addEventListener('timeupdate', () => {
  if (audio.dataset.currentTrack) {
    trackPositions[audio.dataset.currentTrack] = audio.currentTime;
  }
});

// al cambiar de pista
trackPositions[audio.dataset.currentTrack] = audio.currentTime;
audio.dataset.currentTrack = newSrc;
audio.src = newSrc;
```

Después de loadedmetadata, el código busca trackPositions[newSrc]. Si existe una posición guardada, asigna ese valor a currentTime y continúa desde allí. fadeIn() lleva el volumen gradualmente desde 0 hasta el objetivo, evitando un cambio brusco entre canciones.

No confundir

trackPositions sólo vive en memoria JavaScript durante esa estancia en Intro. No es la cookie faction, no es sessionStorage y no es la sesión de WooCommerce. Al abandonar o recargar la página, ese objeto se vuelve a crear.

9.8.3 Ejemplo guiado: Alex cambia de música

Alex llega a Intro. Mientras la barra todavía está entre 0 y 100 %, el panel musical aparece deshabilitado. Al completarse la inicialización se abren las compuertas, CONTINUE queda disponible y el panel acepta clics.

1. MAIN THEME ya está preparado en hud-audio, pero inicialmente permanece muteado. Al llegar al 100 %, si mainAudio sigue muteado el código lo pausa y lo devuelve a 0; si Alex había desmuteado antes, intenta reproducirlo. Más adelante aparece la referencia fuera de alcance a isMuted explicada en 9.8.1. Alex puede utilizar el control global de sonido: setGlobalMute(false), con mainReady ya activo, intenta reproducir el hud-audio actual.

2. Con el sonido activado y MAIN THEME efectivamente reproduciéndose, Alex lo escucha durante unos 35 segundos y después pulsa THE GIRL WHO LOVED JOSEPH KANE. Antes del cambio, trackPositions guarda aproximadamente 35 como posición de MAIN THEME.

3. JavaScript marca visualmente la nueva opción, pausa hud-audio, cambia audio.src y espera loadedmetadata. Cuando la pista está lista, recupera su posición previa si ya había sido usada y aplica el fade.

4. Más tarde Alex vuelve a MAIN THEME. Como trackPositions conserva el segundo anterior, hud-audio puede continuar aproximadamente desde donde había quedado en vez de empezar obligatoriamente desde 0.

9.9 CONTINUE y origen de la navegación

Código fuente: hud-intro.php → el script principal lee y elimina sessionStorage.fromMenu alrededor de las líneas 1158-1161; freezeVideo() se declara cerca de la línea 1295 y se utiliza antes de abandonar Intro. La decisión de destino ocurre en JavaScript del navegador.

La introducción puede abrirse durante la primera visita o desde VIEW INTRO en el menú principal. El menú guarda sessionStorage.fromMenu = true. Intro lee ese valor y lo elimina; al pulsar CONTINUE decide el destino según ese origen.

```
const fromMenu = sessionStorage.getItem('fromMenu') === 'true';
sessionStorage.removeItem('fromMenu');

if (fromMenu) {
  window.location.href = 'https://mcartworksstudio.com';
} else {
  window.location.href = 'https://mcartworksstudio.com/faction-selection';
}
```

En Chromium, antes de salir, `freezeVideo()` captura el fotograma visible de los dos feeds mediante Canvas y lo deja superpuesto mientras se realiza la navegación. Esto evita un corte a negro durante la transición.

```
/intro/
↓
carga GDI + NOD
↓
Promise.all() + reproducción
↓
sincronización periódica
↓
0-100 % → compuertas → música → CONTINUE
↓
¿fromMenu?
├ sí → Home
└ no → Faction Selection
```

9.10 Ejemplo guiado: Alex atraviesa Intro de principio a fin

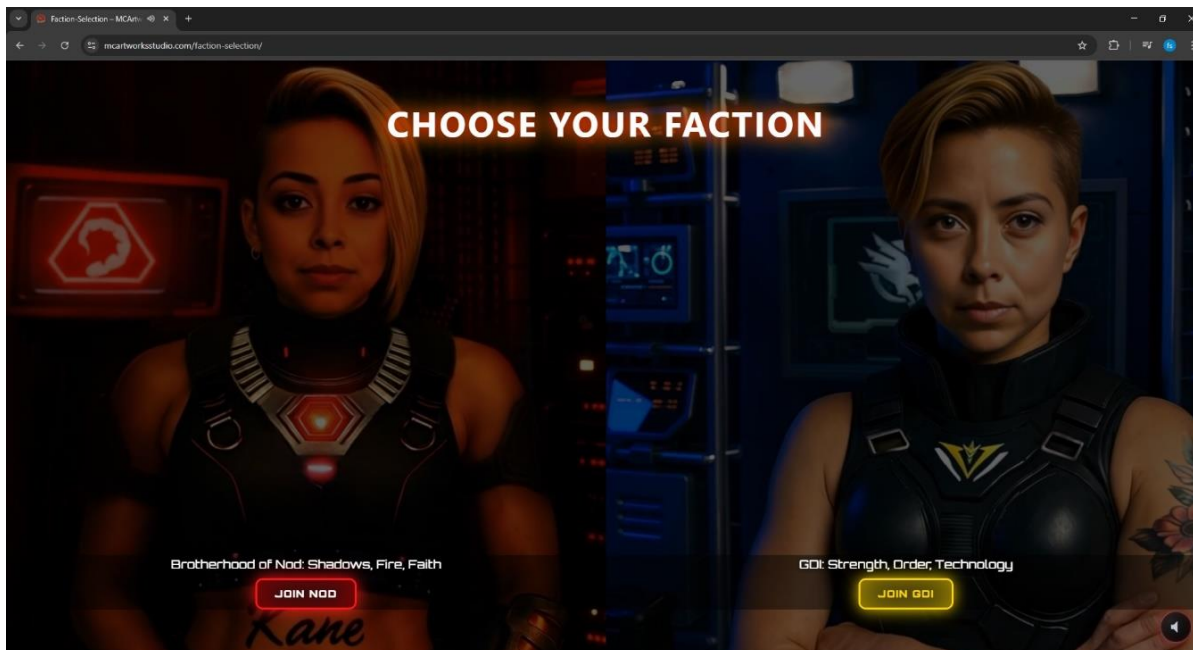
Alex llega a `/intro/` desde la presentación inicial. WordPress confirma el slug, incluye `hud-intro.php` y ejecuta `nod_gdi_hud_intro()`. El navegador recibe los dos feeds GDI/NOD, los MP4 de respaldo, el HUD, los controles y el código JavaScript.

```
Alex entra en /intro/
↓
se preparan GDI + NOD con Promise.all()
↓
ambos feeds comienzan coordinados
↓
la sincronización corrige diferencias mayores a 0,05 s
↓
la barra visual llega a 100 %
↓
se abren las compuertas y se habilitan música + CONTINUE
↓
Alex puede elegir/cambiar una pista
↓
Alex pulsa CONTINUE
↓
¿sessionStorage.fromMenu era true al entrar?
├ sí → Home
└ no → Faction Selection
```

La decisión final no depende de la música elegida ni de una comprobación directa de “primera visita”. El selector musical afecta únicamente la experiencia de Intro; CONTINUE decide el destino sólo a partir del valor `fromMenu` leído desde `sessionStorage` al entrar, tal como se explicó en 9.9.

10. Selección de facción

Después de la introducción el visitante llega a `faction-selection.php`. Este archivo, de más de 1.000 líneas, construye una pantalla fullscreen con loading propio, dos paneles audiovisuales, videos de espera y confirmación, música, efectos de sonido, partículas Canvas y la lógica que vuelve persistente la elección NOD/GDI.



Pantalla de selección: NOD a la izquierda y GDI a la derecha.

10.1 Carga de la pantalla de selección

faction-selection.php se conecta a WordPress mediante wp_body_open y sólo actúa en la página faction-selection. El orden está definido: primero se genera el loading general y después la pantalla de selección. Mientras el overlay está activo se aplica loading-active para bloquear el scroll y mantener el contenido principal oculto.

```
add_action('wp_body_open', function() {
    if (is_page('faction-selection')) {
        mostrar_loading_general_overlay();
        pantalla_faccion_epica();
    }
});
```

JavaScript espera el evento animationend de la barra de carga para hacer fade out del overlay y mostrar la pantalla NOD/GDI. Existe además un fallback de 3,5 segundos que dispara el mismo final si el evento no se produce.

10.2 Dos paneles con dos estados de video

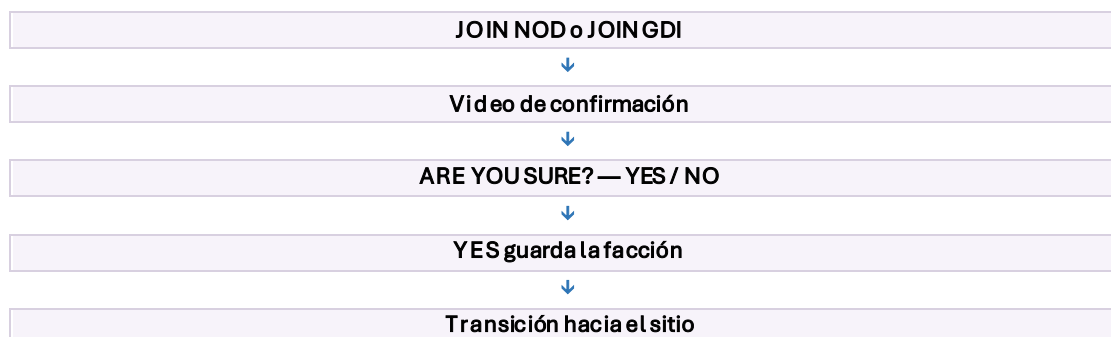
Cada facción se representa con un contenedor .faccion y un atributo data-faccion que vale nod o gdi. Dentro hay un video-default en loop para el estado de espera y un video-confirm que se utiliza después de pulsar JOIN.

```
<div class="faccion nod" data-faccion="nod">
  <video class="video-default" autoplay muted loop playsinline></video>
  <video class="video-confirm" playsinline style="display:none;"></video>
</div>
```

```
ESTADO INICIAL
video-default visible
overlay-default visible
  ↓ JOIN
video-default oculto
video-confirm visible
overlay-confirm visible
  ↓
ARE YOU SURE? → YES / NO
```

10.3 Vista previa, JOIN y confirmación

Cada facción tiene un video de vista previa y otro de confirmación. Al pasar el cursor por JOIN se activa la vista previa correspondiente; al hacer clic se reproduce el video de confirmación y aparece la pregunta YES / NO. La facción se guarda recién cuando el visitante confirma YES. El código también crea una cookie temporal para indicar que acaba de salir de esta pantalla.



10.4 Por qué existe la confirmación

La confirmación evita que un simple movimiento del cursor cambie toda la apariencia del sitio. La elección se guarda únicamente cuando el visitante confirma la facción.

10.5 Control de estados y botón NO

Al iniciar una confirmación se comprueba si la otra facción ya tiene un video-confirm visible. Si existe, se reinicia antes de activar la nueva elección. Esto garantiza que NOD y GDI no permanezcan simultáneamente en estado de confirmación.

```

function resetThisFaction() {
  videoConfirm.pause();
  videoConfirm.currentTime = 0;
  videoConfirm.style.display = 'none';
  overlayConfirm.style.display = 'none';
  videoDefault.style.display = 'block';
  overlayDefault.style.display = 'flex';
}
  
```

El botón NO ejecuta esta función y vuelve al estado inicial sin guardar ninguna cookie. Cuando el video de confirmación termina, queda pausado en su último fotograma mientras el visitante decide.

10.6 Confirmación YES y persistencia de la elección

YES obtiene la facción desde el atributo data-faccion del contenedor seleccionado. Recién en ese momento se crea la cookie faction por aproximadamente un año y la cookie temporal from_faction_selection por diez segundos. Después de la transición, el sitio vuelve a Home.

```

const faction = btn.closest('.faccion');

document.cookie = 'faction=' + faction.dataset.faction
  + '; path=/; max-age=' + (60 * 60 * 24 * 365);
document.cookie = 'from_faction_selection=1; path=/; max-age=10';

setTimeout(() => { window.location.href = '/'; }, 1000);
  
```

Diferencia entre las dos cookies

faction conserva la identidad elegida y condiciona la apariencia posterior del sitio. from_faction_selection es sólo una señal transitoria que permite reconocer que la navegación acaba de salir del selector.

10.7 Transición audiovisual y Canvas

Después de YES se crea una capa fullscreen roja y se aplica zoom, brillo y blur al contenido de confirmación. Chrome y Opera reciben un tratamiento específico: el código captura el último fotograma del video en un Canvas, lo convierte en una imagen JPEG temporal y anima esa imagen antes de redirigir. En otros navegadores el efecto se aplica directamente al video.

```
const canvas = document.createElement('canvas');
canvas.width = videoConfirm.videoWidth || 1920;
canvas.height = videoConfirm.videoHeight || 1080;
const ctx = canvas.getContext('2d');
ctx.drawImage(videoConfirm, 0, 0, canvas.width, canvas.height);
img.src = canvas.toDataURL('image/jpeg', 0.9);
```

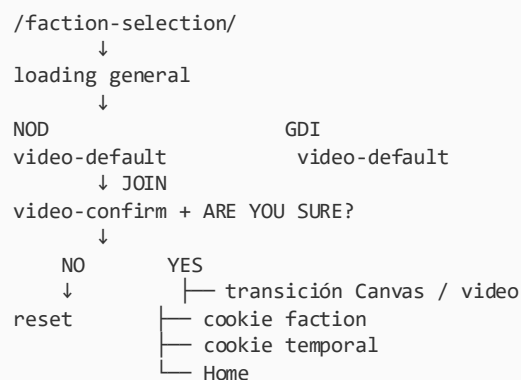
10.8 Sonido global y partículas

La pantalla dispone de música ambiental y efectos diferentes para hover, JOIN, YES y NO. Un único estado isMuted controla tanto esos audios como los videos. Al activar el sonido por primera vez, la interacción del usuario permite iniciar la música sin infringir las restricciones habituales de autoplay.

El fondo utiliza un elemento Canvas independiente que genera 80 partículas con posición, radio y velocidad aleatorios. animateParticles() actualiza su posición y utiliza requestAnimationFrame() para mantener el movimiento sincronizado con el renderizado del navegador.

```
for (let i = 0; i < 80; i++) {
  particles.push({
    x: Math.random() * canvas.width,
    y: Math.random() * canvas.height,
    r: Math.random() * 2 + 1,
    dy: Math.random() * -1 - 0.2
  });
}
requestAnimationFrame(animateParticles);
```

10.9 Flujo completo de selección



11. Cookies y apariencia NOD / GDI

La cookie faction es el dato que permite que una elección hecha en una página del navegador siga afectando páginas posteriores. Sin esa persistencia, Home, el menú, Gallery y el resto de la interfaz no sabrían si deben utilizar recursos GDI o NOD en cada nueva petición.

Para entender esta parte hay que separar dos momentos: JavaScript escribe la cookie en el navegador cuando el visitante confirma una facción; después, en la siguiente petición HTTP, PHP recibe ese valor dentro de \$_COOKIE y genera la respuesta correspondiente.

11.1 Dónde nace la cookie faction

La cookie definitiva se crea en `faction-selection.php`, dentro del evento click de los botones YES. El valor no se toma del texto visible del botón, sino del atributo `data-faccion` del contenedor `.faccion` seleccionado.

```
const faction = btn.closest('.faccion');

document.cookie =
  'faction=' + faction.dataset.faction
  + '; path=/; max-age=' + (60 * 60 * 24 * 365);

document.cookie =
  'from_faction_selection=1; path=/; max-age=10';

setTimeout(() => { window.location.href = '/'; }, 1000);
```

Ubicación en el proyecto: `wp-content/themes/blocksy-child/faction-selection.php` · bloque del botón YES alrededor de las líneas 894-977; las cookies se escriben aproximadamente en las líneas 969-971.

11.2 Qué significa cada parte de la cookie

Parte	Ejemplo	Qué significa
Nombre	<code>faction</code>	Identifica el dato que luego leerá PHP.
Valor	<code>nod / gdi</code>	Es la facción confirmada por el visitante.
path	<code>/</code>	Hace que la cookie se envíe en las distintas rutas del mismo sitio.
max-age	<code>60 × 60 × 24 × 365</code>	Mantiene faction durante aproximadamente un año.
Cookie temporal	<code>from_faction_selection=1</code>	Indica durante unos segundos que la navegación acaba de salir del selector.

La duración larga pertenece a `faction`. `from_faction_selection` tiene otro propósito y dura sólo diez segundos: no representa identidad, sino el origen inmediato de una navegación.

11.3 JavaScript escribe; PHP la recibe en la petición siguiente

Cuando Alex pulsa YES, la página actual ya fue generada por PHP. Escribir `document.cookie` no hace que el PHP de esa misma respuesta vuelva a ejecutarse. Lo que ocurre es que el navegador conserva el nuevo dato y lo adjunta automáticamente en la siguiente petición al mismo sitio.

```
JavaScript ejecuta document.cookie
↓
el navegador guarda faction=nod o faction=gdi
↓
window.location.href abre otra URL
↓
empieza una nueva petición HTTP
↓
el navegador envía la cookie al servidor
↓
PHP recibe $_COOKIE['faction']
↓
el tema decide recursos y estilos antes de responder
```

Idea clave

La cookie funciona como puente entre dos peticiones distintas. JavaScript la escribe en el cliente; PHP la consume después desde el servidor.

11.4 Cómo PHP lee la facción

```
$faction = $_COOKIE['faction'] ?? 'gdi';
```

El operador ?? utiliza el valor de la cookie si existe; si no existe, el código mostrado adopta gdi como valor por defecto. Esta lectura aparece en distintos módulos porque la facción afecta más que una única hoja de estilo.

11.5 Cómo se encola el CSS GDI o NOD

functions.php registra callbacks en wp_enqueue_scripts. El bloque de producción lee faction, elimina handles previos y encola gdi-faction-min.css o nod-faction-min.css. La versión se calcula con filemtime() para que cambie cuando cambia el archivo.

```
$faction = $_COOKIE['faction'] ?? 'gdi';

wp_dequeue_style('gdi-faction');
wp_dequeue_style('nod-faction');

if ($faction === 'gdi') {
    $css_file = get_stylesheet_directory() . '/gdi-faction-min.css';
    $version = file_exists($css_file) ? filemtime($css_file) : '1.0';
    wp_enqueue_style('gdi-faction', get_stylesheet_directory_uri() . '/gdi-faction-min.css', [], $version, 'all');
} elseif ($faction === 'nod') {
    $css_file = get_stylesheet_directory() . '/nod-faction-min.css';
    $version = file_exists($css_file) ? filemtime($css_file) : '1.0';
    wp_enqueue_style('nod-faction', get_stylesheet_directory_uri() . '/nod-faction-min.css', [], $version, 'all');
}
```

El código actual contiene además un segundo callback activo con la misma prioridad 999 y los mismos handles, pero usa time() para forzar una URL diferente en cada carga. La sección 36 analiza ese solapamiento desde el punto de vista de caché y rendimiento; aquí interesa que ambos bloques consultan la misma cookie faction.

Ubicación en el proyecto: functions.php · callbacks de CSS de facción aproximadamente entre las líneas 77 y 185 · ambos registrados en wp_enqueue_scripts con prioridad 999.

11.6 Qué partes del sitio dependen de faction

Zona	Cómo utiliza faction
CSS global	Elige gdi-faction-min.css o nod-faction-min.css.
Home	Selecciona título, HLS y recursos visuales de la facción.
header.php / menú	Elige el conjunto de videos y música del HUD.
Gallery	Selecciona el array GDI/NOD y su identidad visual.
Our Services y otras pantallas	Reciben la apariencia general aportada por la hoja de estilo activa.

Por eso faction no equivale a “un color guardado”. Es un dato de contexto que varias piezas leen para decidir recursos y apariencia.

11.7 Para qué existe from_faction_selection

from_faction_selection dura sólo diez segundos y sirve para reconocer la navegación inmediatamente posterior al selector. Home la comprueba en mostrar_overlay_faccion_inicio(); si existe, lee faction, muestra el overlay GDI o NOD correspondiente y luego la elimina para que una simple recarga no repita esa transición.

```
if (isset($_COOKIE['from_faction_selection'])) {
    $faction = $_COOKIE['faction'] ?? 'gdi';

    if ($faction === 'gdi') {
        mostrar_loading_GDI_overlay();
    } elseif ($faction === 'nod') {
        mostrar_loading_NOD_overlay();
    }
}
```

```

    }

    setcookie('from_faction_selection', '', time() - 3600, '/');
}

```

```

YES en Faction Selection
↓
faction = nod/gdi por ~1 año
+
from_faction_selection = 1 por 10 s
↓
Home recibe ambas cookies
↓
faction decide identidad
↓
from_faction_selection dispara overlay de acceso
↓
PHP elimina la cookie temporal

```

11.8 Qué ocurre si faction no existe

La ausencia de faction vuelve a ser significativa para la entrada del sitio: insertar_presentacion() puede activar la secuencia aleatoria explicada en la sección 8 cuando la petición pertenece a uno de los contextos admitidos. En otras partes del código existen valores GDI por defecto para evitar que ciertos módulos queden sin recursos mientras no haya una elección persistente.

Esto no significa que GDI quede guardado automáticamente. Un valor por defecto usado por PHP para construir una respuesta no es lo mismo que escribir document.cookie. La identidad persistente aparece recién cuando Faction Selection confirma YES.

11.9 No confundir cookie, sessionStorage y sesión WooCommerce

Mecanismo	Qué conserva	Uso en MC Artworks
cookie faction	Identidad NOD/GDI durante un período largo.	Apariencia y recursos de distintas páginas.
cookie from_faction_selection	Señal temporal de unos segundos.	Mostrar el overlay al regresar a Home desde el selector.
sessionStorage.fromMenu	Estado de esa pestaña/sesión de navegación.	Intro sabe si CONTINUE debe volver a Home.
Sesión WooCommerce	Estado comercial temporal.	Carrito y datos asociados a sus items.

11.10 Ejemplo guiado: Alex elige NOD y navega por el sitio

Una vez explicado el mecanismo, se puede seguir la experiencia completa. Alex llega a Faction Selection, pulsa JOIN NOD y finalmente confirma YES.

- JavaScript obtiene el contenedor .faccion de NOD y lee data-faccion="nod".
- El navegador guarda faction=nod con path=/ y una duración aproximada de un año. También guarda from_faction_selection=1 durante diez segundos.
- Un segundo después, JavaScript navega a /. Esa navegación crea una petición nueva; el navegador envía las cookies al servidor.
- PHP recibe \$_COOKIE['faction'] = "nod". Los callbacks de estilos encolan nod-faction-min.css y Home selecciona sus recursos NOD.
- Como from_faction_selection todavía existe, Home muestra el overlay NOD de acceso y después elimina esa cookie temporal.

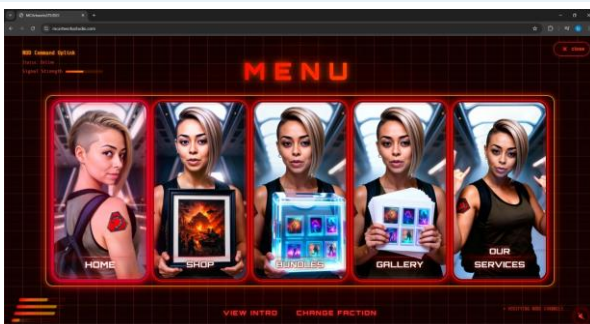
6. Alex abre Shop y más tarde Gallery. En esas nuevas peticiones el navegador sigue enviando `faction=nod`, por lo que el sitio mantiene la identidad NOD sin pedirle que elija otra vez.

7. Si Alex vuelve otro día y la cookie no expiró ni fue borrada, el navegador continúa enviándola. Si desaparece, la lógica de entrada vuelve a tratarlo como una visita sin facción persistida.

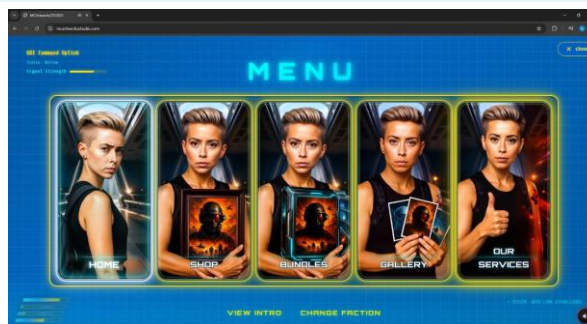
12. Menú principal

`header.php` construye uno de los componentes centrales de MC Artworks Studio: un menú HUD de pantalla completa de más de 560 líneas. No funciona como una barra de navegación tradicional; combina PHP, HTML y JavaScript para generar tarjetas con video, recursos diferentes por facción, música persistente, efectos de sonido, estados accesibles, transiciones con Canvas y coordinación con Gallery e Intro.

Ubicación en el proyecto: `wp-content/themes/blocksy-child/header.php` - este archivo construye el HUD del menú, selecciona recursos según la cookie `faction` y contiene la mayor parte de los listeners JavaScript de apertura, cierre, audio y navegación. `functions.php` interviene en piezas globales del sitio, pero el menú audiovisual se concentra en `header.php`.



Menú NOD: rojo y oscuro.



Menú GDI: azul y amarillo.

12.1 Integración con Blocksy

Código fuente: `wp-content/themes/blocksy-child/header.php` → PHP del template de cabecera, líneas aproximadas 17-19. `ob_start()` captura la salida de `blocksy_output_header()`; `ob_get_clean()` la guarda en `$global_header` para reinsertarla después junto al HUD propio.

`header.php` sigue formando parte del sistema de cabecera de Blocksy. Antes de construir el HUD captura la salida generada por `blocksy_output_header()` mediante un buffer. Después inserta el menú personalizado y vuelve a imprimir la cabecera del tema padre dentro del flujo normal de WordPress.

```
ob_start();
blocksy_output_header();
$global_header = ob_get_clean();

...

do_action('blocksy:header:before');
echo $global_header;
do_action('blocksy:header:after');
```

```
header.php
↓
Blocksy genera su header
↓
ob_start() / ob_get_clean() lo guardan
↓
se construye el HUD propio
↓
se reinserta el header de Blocksy
```

12.2 Recursos según la facción

Código fuente: header.php → PHP alrededor de las líneas 21-42. La cookie faction selecciona uno de los arrays de videos antes de que el servidor genere las tarjetas del menú.

PHP lee la cookie faction y selecciona un conjunto diferente de cinco videos. La estructura HTML del menú es la misma para NOD y GDI; lo que cambia son los recursos audiovisuales y la hoja de estilo de facción.

```
$faction = $_COOKIE['faction'] ?? 'gdi';

$videos = $videos_by_faction[$faction];
```

12.3 Construcción de las tarjetas

Código fuente: header.php → PHP/HTML aproximadamente entre las líneas 68-109. foreach(\$videos as \$index=>\$video_url) combina videos, enlaces y etiquetas; este bloque se ejecuta en el servidor y entrega las cinco tarjetas ya construidas al navegador.

Los destinos, etiquetas y videos se conectan por índice. foreach() genera las cinco tarjetas, marca la actual con is-current y crea un enlace que cubre toda la superficie de cada tarjeta. Los valores se imprimen mediante esc_url(), esc_html() y esc_attr().

```
foreach ($videos as $index => $video_url) {
    $link_url = $links[$index];
    $is_current = rtrim($link_url, '/') === rtrim($current_url, '/');
    // tarjeta + video + enlace + etiqueta
}
```

Además del carril de tarjetas, el overlay incluye capas data-stream, scanlines, sweep-beam, radar-pulse, barras HUD, información de estado y una terminal ficticia. El título NOD Command Uplink o GDI Command Uplink se genera desde PHP según la facción.

12.4 Apertura, cierre y entrada escalonada

Código fuente: header.php → JavaScript del HUD; openOverlay() comienza alrededor de la línea 225 y es llamado por los listeners del botón #zajMenu. Aquí ya estamos en el navegador: se modifican clases, atributos ARIA, scroll y retrasos de animación.

El botón #zajMenu controla el overlay #zajOverlay. openOverlay() agrega is-open, actualiza aria-expanded/aria-hidden, bloquea el scroll y vuelve a disparar la animación de cada tarjeta. El retraso index * 0.07 s crea una entrada escalonada. El menú también puede cerrarse con Escape o pulsando el fondo del overlay.

```
function openOverlay() {
    btn.classList.add('is-open');
    overlay.classList.add('is-open');
    btn.setAttribute('aria-expanded', 'true');
    overlay.setAttribute('aria-hidden', 'false');

    cards.forEach((card, index) => {
        card.style.animationDelay = `${index * 0.07}s`;
        card.classList.add('animate-in');
    });
}
```

12.5 Efectos de sonido de las tarjetas

JavaScript crea tres objetos Audio: hoverSound, clickSound y currentClickSound. El hover sólo se reproduce sobre tarjetas que no representan la página actual; el clic utiliza un efecto distinto si la tarjeta tiene is-current. Antes de cada reproducción se reinicia currentTime a 0 para permitir interacciones repetidas.

12.6 Audio que continúa donde quedó

El menú guarda en una cookie el segundo exacto en que quedó la música. El nombre de esa cookie depende de la facción: por ejemplo, gdi_menu_audio_time o nod_menu_audio_time. Cuando el menú vuelve a abrirse, JavaScript recupera ese valor y continúa desde allí.

Ejemplo

Si el visitante cierra el menú cuando la pista NOD está en 42,6 segundos, el código guarda ese tiempo. Al volver a abrir el menú NOD intenta iniciar la reproducción desde aproximadamente 42,6 s en lugar de empezar siempre desde cero.

12.7 Música, fade y control de mute

Código fuente: header.php → JavaScript del HUD → initMenuAudio() alrededor de la línea 301. La función crea/configura el único Audio del menú, restaura el segundo guardado en la cookie de facción y mantiene esa posición mientras la pista se reproduce.

initMenuAudio() construye la ruta /songs/nod-menu-theme o /songs/gdi-menu-theme, configura loop y preload y restaura el segundo guardado en la cookie. Mientras la pista está activa, un intervalo actualiza esa posición cada dos segundos.

```
const cookieName = `${faction}_menu_audio_time`;
const savedTime = parseFloat(getCookie(cookieName));
if (!isNaN(savedTime)) menuAudio.currentTime = savedTime;

setInterval(() => {
  if (menuAudio && !menuAudio.paused)
    setCookie(cookieName, menuAudio.currentTime);
}, 2000);
```

La apertura realiza un fade hasta volumen 0.4 en unos 800 ms; el cierre reduce el volumen a 0 y luego pausa la pista. Si el menú fue abierto automáticamente, comienza silenciado y muestra un control que permite al usuario activar posteriormente música y efectos.

12.8 Navegación con transición

Código fuente: header.php → listeners de .zaj-card-link alrededor de las líneas 194-218 → showLoadingAndRedirect() desde aproximadamente la línea 432. El listener cancela la navegación inmediata y delega la transición a esa función.

Al elegir una tarjeta, el cambio de página no ocurre de golpe. El menú muestra una breve pantalla de carga y después abre el destino. VIEW INTRO permite volver a la introducción y CHANGE FACTION lleva nuevamente al selector de facción.

La transición se implementa en showLoadingAndRedirect(). El clic normal del enlace se cancela con preventDefault(); antes de cambiar window.location.href se congela el último fotograma de las cinco vistas previas, se muestra ESTABLISHING UPLINK y se anima una barra de progreso.

```
link.addEventListener('click', e => {
  e.preventDefault();
  showLoadingAndRedirect(link.getAttribute('href'));
});
```

12.9 Congelación del último fotograma

Para evitar un corte visual, showLoadingAndRedirect() crea un Canvas por cada .zaj-card-video, dibuja el frame visible y lo convierte en una imagen JPEG que se superpone temporalmente al video. Después activa el loader y realiza la navegación tras unos 800 ms.

```
const canvas = document.createElement('canvas');
canvas.width = v.videoWidth;
canvas.height = v.videoHeight;
const ctx = canvas.getContext('2d');
ctx.drawImage(v, 0, 0, canvas.width, canvas.height);
snapshot.style.backgroundImage = `url(${canvas.toDataURL('image/jpeg')})`;
```

12.10 VIEW INTRO, CHANGE FACTION y Gallery

Código fuente: header.php → sessionStorage.fromMenu se escribe cerca de la línea 424; stopAllOtherAudio()/restorePreviousAudio() comienzan aproximadamente en las líneas 514 y 533. Son piezas distintas del mismo script del menú: una conserva el origen de Intro y las otras coordinan audio/video con Gallery.

VIEW INTRO guarda sessionStorage.fromMenu = true antes de abrir /intro, permitiendo que CONTINUE regrese luego a Home. CHANGE FACTION utiliza el mismo loader para abrir /faction-selection. En Gallery existe además una integración específica: al abrir el menú se pausa el audio ajeno al overlay y se silencian los videos externos; al cerrarlo se restaura el estado previo.

```
sessionStorage.setItem('fromMenu', 'true');
showLoadingAndRedirect('https://mcartworksstudio.com/intro');
```

```
usuario abre MENU
↓
openOverlay()
├─ bloquea scroll
├─ anima cinco tarjetas
├─ restaura música desde cookie
└─ habilita sonidos
↓
usuario elige destino
↓
showLoadingAndRedirect()
├─ captura frames con Canvas
├─ muestra ESTABLISHING UPLINK
├─ anima progreso
└─ cambia de página
```

12.11 Ejemplo guiado: Alex abre el menú y entra en Gallery

Las subsecciones anteriores describieron cada pieza por separado. En este ejemplo se combinan para ver qué ocurre cuando una persona usa el HUD de navegación.

Alex está en Home con faction=nod. PHP ya generó header.php con los cinco videos y recursos correspondientes a NOD. Cuando Alex pulsa MENU, JavaScript ejecuta openOverlay(): agrega is-open, actualiza aria-expanded/aria-hidden, bloquea el scroll y aplica una entrada escalonada a las tarjetas.

El menú inicializa la pista NOD y consulta la cookie nod_menu_audio_time. Si Alex había cerrado el HUD anteriormente en el segundo 42,6, el reproductor intenta retomar aproximadamente desde ese punto. Los sonidos de hover y click siguen reaccionando a las tarjetas según el estado actual.

Alex coloca el cursor sobre Gallery y después hace clic. El listener cancela la navegación inmediata con preventDefault() y llama a showLoadingAndRedirect(). Antes de cambiar de página, la función toma el fotograma visible de cada preview con Canvas, lo deja como snapshot, muestra ESTABLISHING UPLINK y ejecuta la transición de progreso. Recién entonces cambia window.location.href hacia Gallery.

```
Alex pulsa MENU
↓
openOverlay() abre el HUD
↓
se restauran música y sonidos de NOD
↓
Alex elige Gallery
↓
preventDefault() evita el salto inmediato
↓
showLoadingAndRedirect()
↓
Canvas congela los previews
↓
ESTABLISHING UPLINK + progreso
↓
navegación a Gallery
```

12.12 Ejemplo guiado: Alex vuelve a Intro desde el menú

VIEW INTRO usa el mismo sistema de transición, pero antes guarda `sessionStorage.fromMenu = true`. Cuando Intro se abre, lee ese valor y lo elimina. Por eso, si Alex pulsa CONTINUE después de haber entrado desde el menú, regresa a Home en lugar de pasar nuevamente por Faction Selection.

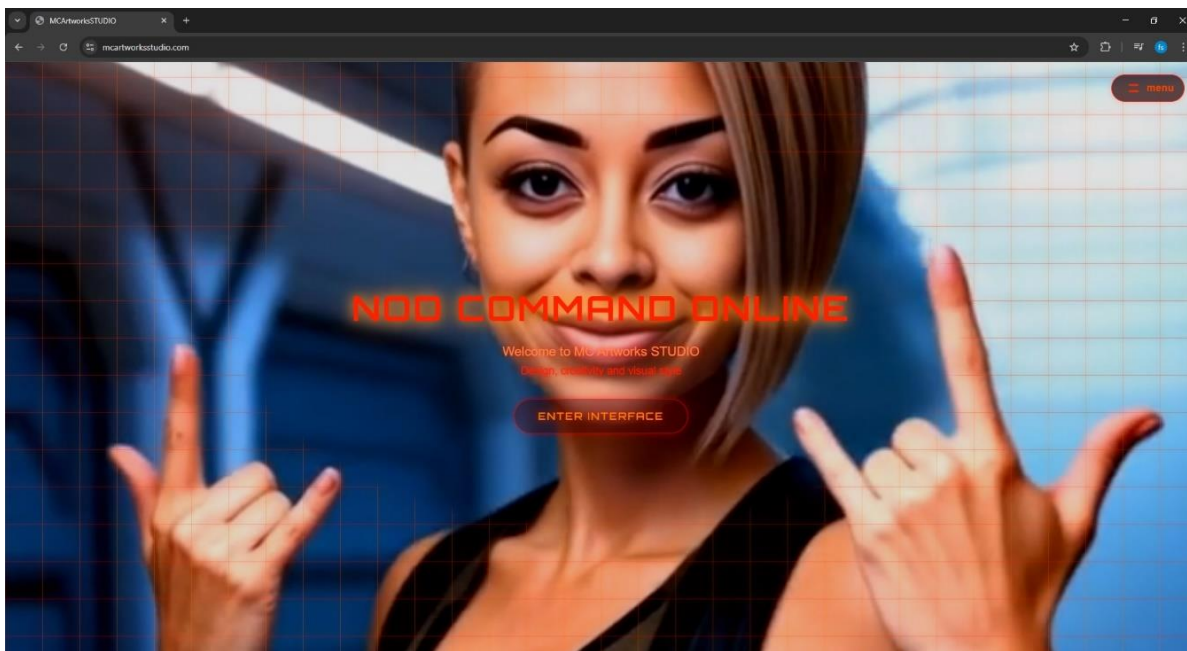
```
Alex abre MENU
↓
VIEW INTRO
↓
sessionStorage.fromMenu = true
↓
showLoadingAndRedirect(/intro/)
↓
Intro lee y elimina fromMenu
↓
Alex pulsa CONTINUE
↓
Home
```

13. Inicio (Home)

La portada cambia su video según la facción guardada. GDI utiliza un video HLS distinto de NOD. Ese video ocupa el fondo principal y el botón ENTER INTERFACE abre una vista ampliada del contenido.

Ubicación en el proyecto: `wp-content/themes/blocksy-child/functions.php · add_action('wp_head', 'home')` alrededor de la línea 381 activa `home()`; `mostrar_overlay_faccion_inicio()` se registra también en `wp_head` alrededor de la línea 667. Ambas piezas leen la facción antes de imprimir HTML/CSS/JavaScript de Home.

En diseño web se suele llamar Hero al bloque principal que ocupa la zona más visible de una página. En MC Artworks Studio ese bloque no es una imagen fija: combina video, texto y controles superpuestos.



Inicio NOD con contenido sobre el video principal y acceso ENTER INTERFACE.

13.1 Cuándo se activa la portada

Código fuente: `wp-content/themes/blocksy-child/functions.php → add_action('wp_head', 'home')` línea aproximada 381 → `home()` desde la 391. WordPress llama `home()` al procesar `wp_head`; la función retorna inmediatamente cuando la petición no es la portada.

La función `home()` está conectada al hook `wp_head`, pero antes de generar la interfaz comprueba que WordPress esté mostrando la portada. Si la petición corresponde a otra página, la función termina inmediatamente. Esto evita cargar el video HLS, el modal y sus scripts en zonas donde no son necesarios.

```
add_action('wp_head', 'home');

function home() {
    if (!is_front_page() && !is_home()) return;
    // ... interfaz Home
}
```

Ejemplo claro. Si el visitante entra en /gallery/, la condición falla y Home no genera su reproductor. Si entra en la portada, la función continúa y construye la interfaz audiovisual.

13.2 Selección del contenido según la facción

Código fuente: functions.php → home() alrededor de las líneas 395-400. PHP lee \$_COOKIE['faction'] en el servidor y, antes de imprimir la respuesta, calcula \$title_text y \$video_src.

Home lee la cookie faction y utiliza ese valor para decidir el título y el video principal. La elección no cambia el producto de WooCommerce ni una entrada de la base de datos: cambia la experiencia visual que se genera para esa visita.

```
$faction = $_COOKIE['faction'] ?? 'gdi';
$title_text = ($faction === 'gdi') ? 'GDI' : 'NOD';
$video_src = ($faction === 'gdi')
    ? '/videos/gdi-home/gdi-home.m3u8'
    : '/videos/nod-home/nod-home.m3u8';

faction = gdi
↓
título GDI + video HLS GDI

faction = nod
↓
título NOD + video HLS NOD
```

13.3 Cómo se arma el Hero

Código fuente: functions.php → home() → HTML impreso aproximadamente entre las líneas 403-432. El Hero y el modal nacen en la respuesta del servidor; los eventos que los vuelven interactivos se conectan después en el JavaScript de la misma función.

El bloque principal combina tres capas: un contenedor ovalado que contiene el video, un texto de inicialización y el contenido final con el título, el subtítulo y ENTER INTERFACE. El video empieza silenciado para facilitar el autoplay permitido por los navegadores.

```
<section class="hero-video-section">
  <div class="oval-video-wrapper">
    <div class="home-loading-text">INITIALIZING ...</div>
    <video autoplay muted loop playsinline id="header-video"></video>
  </div>
  <div class="hero-content" id="hero-content">
    <h1>GDI / NOD COMMAND ONLINE</h1>
    <a class="hero-hud-btn">ENTER INTERFACE</a>
  </div>
</section>
```

13.4 Precarga del primer fragmento

Código fuente: functions.php → home() → <script> ejecutado en DOMContentLoaded desde aproximadamente la línea 437; la precarga manual del manifiesto/primer segmento está alrededor de las líneas 444-454. PHP imprime el script y el navegador realiza los fetch().

El código intenta tener datos disponibles antes de iniciar la reproducción para reducir la espera hasta el primer fotograma. En HLS, cada fragmento es una pequeña parte del video. Tener el primero listo permite comenzar sin esperar a descargar el contenido completo.

En la implementación real, primero se descarga el manifiesto HLS como texto. Una expresión regular localiza el primer archivo .ts, se reconstruye su ruta a partir del mismo directorio del .m3u8 y se realiza un segundo fetch para adelantar esa descarga.

```

fetch(hlsSource)
  .then(response => response.text())
  .then(manifest => {
    const firstSegment = manifest.match(/^(?!#).*\.ts/m);
    if (firstSegment && firstSegment[0]) {
      const basePath = hlsSource.substring(0, hlsSource.lastIndexOf('/') + 1);
      fetch(basePath + firstSegment[0]);
    }
  })
  .catch(() => {});

```

13.5 Inicialización con HLS.js y respaldo nativo

Código fuente: functions.php → home() → función JavaScript initHLS(videoElement), aproximadamente líneas 457-471. Recibe cada <video>, decide entre HLS.js y soporte HLS nativo y conecta la misma fuente HLS al Hero y al modal.

Después de obtener la ruta .m3u8, JavaScript intenta reproducirla con HLS.js. Si el navegador ya entiende HLS de forma nativa, asigna la lista directamente al elemento video. Son dos caminos distintos hacia el mismo resultado: reproducir el contenido HLS sin descargar un MP4 completo de una sola vez.

```

if (Hls.isSupported()) {
  const hls = new Hls({ maxBufferLength: 10, startLevel: 0 });
  hls.loadSource(hlsSource);
  hls.attachMedia(videoElement);
} else if (videoElement.canPlayType('application/vnd.apple.mpegurl')) {
  videoElement.src = hlsSource;
}

```

13.6 Cuándo desaparece la pantalla de carga

Código fuente: functions.php → home() → JavaScript aproximadamente entre las líneas 473-520. startIntroSequence() se dispara desde canplay/loadeddata/play; esos eventos pertenecen al elemento <video> ya cargado en el navegador.

La portada no expande el video apenas se crea el elemento HTML. Espera a que el navegador tenga datos suficientes para dibujar un fotograma. Los eventos canplay y loadeddata activan startIntroSequence(), ocultan el texto de inicialización, expanden el óvalo y muestran el contenido del Hero.

```

HLS comienza a cargar
↓
canplay / loadeddata
↓
startIntroSequence()
↓
texto de carga desaparece
↓
óvalo se expande
↓
aparece ENTER INTERFACE

```

13.7 Qué hace ENTER INTERFACE

Código fuente: functions.php → home() → listener click de .hero-hud-btn alrededor de las líneas 522-554. El clic no llama a WordPress ni cambia de URL: modifica el DOM, bloquea MENU y reproduce fullVideo dentro del modal actual.

El botón no navega a otra página. Abre un modal a pantalla completa con el mismo contenido HLS, bloquea temporalmente la interacción con el resto de la página y deshabilita el botón MENU mientras el modal está activo.

```

ENTER INTERFACE
↓
gdiModal recibe class="active"
↓
body recibe no-interaction
↓
MENU queda deshabilitado
↓
fullVideo vuelve a 0 y comienza a reproducir

```

13.8 Cierre del modal y comportamiento NOD

Código fuente: functions.php → home() → temporizadores NOD desde aproximadamente la línea 573 y listener de cierre #closeModal alrededor de la línea 605. Todo ocurre en el mismo callback DOMContentLoaded que abrió el modal; al cerrar se limpian clases, reproducción, temporizadores e imagen temporal.

Al pulsar X, el modal se cierra, el video se pausa y MENU vuelve a estar disponible. En NOD existe además una imagen temporal (wuhrrer-kucan.jpg) que aparece durante la reproducción. El código conserva temporizadores propios para mostrarla, ocultarla y eliminarla sin dejar elementos duplicados. El cierre no se limita a ocultar el modal: también restablece la interacción general, vuelve a habilitar MENU y limpia los temporizadores y la imagen temporal de NOD para que no queden efectos pendientes después de cerrar la vista fullscreen.

```
closeBtn.addEventListener('click', () => {
  modal.classList.remove('active');
  document.body.classList.remove('no-interaction');
  fullVideo.pause();

  if (menuBtn) {
    menuBtn.disabled = false;
    menuBtn.classList.remove('zaj-menu--disabled');
  }

  clearTimeout(nodShowTimeout);
  clearTimeout(nodHideTimeout);
  clearTimeout(nodRemoveTimeout);
  document.querySelectorAll('.nod-fade-image')
    .forEach(img => img.remove());
});
```

En GDI el video fullscreen se reinicia al finalizar. En NOD también existe un control de reinicio para poder volver a programar la imagen temporal cuando el video comienza otro ciclo.

```
nodShowTimeout = setTimeout(() => {
  nodImage.classList.add('visible');

  nodHideTimeout = setTimeout(() => {
    nodImage.classList.remove('visible');
    nodRemoveTimeout = setTimeout(() => nodImage.remove(), 1500);
  }, 6000);
}, 16500);
```

13.9 Entrada desde la selección de facción

Código fuente: functions.php → add_action('wp_head', 'mostrar_overlay_faccion_inicio') alrededor de la línea 667 → mostrar_overlay_faccion_inicio() desde aproximadamente la 679. Esta pieza corre en una carga nueva de Home, lee las cookies y elimina from_faction_selection después de decidir el overlay.

Cuando Home se abre justo después de confirmar GDI o NOD, la cookie temporal from_faction_selection permite mostrar un overlay específico de la facción antes de revelar la portada. Después se elimina esa cookie para que el overlay no vuelva a aparecer por una simple recarga.

```
YES en Faction Selection
↓
cookie faction + cookie temporal
↓
Home
↓
overlay GDI o NOD
↓
se borra from_faction_selection
```

PARTE III — TIENDA, PRODUCTOS Y BUNDLES

14. Cómo se gestionan los productos con WooCommerce

WooCommerce es la fuente del producto comercial: mantiene su identificador, nombre, precio, estado y relación con el carrito y los pedidos. El tema hijo modifica la presentación y agrega comportamientos propios, pero no crea un segundo catálogo paralelo.

Para entender el flujo conviene seguir cómo cambia la representación del mismo producto. En Shop el código trabaja con un objeto `WC_Product`; después de ADD TO CART aparece un cart item temporal; y al confirmar Checkout WooCommerce crea una línea persistente dentro del `WC_Order`.

14.1 Qué es un `WC_Product`

`WC_Product` es la clase que WooCommerce utiliza para representar un producto desde PHP. En lugar de consultar manualmente las tablas de MariaDB para buscar nombre, precio o ID, el tema recibe un objeto y llama a sus métodos. Esto también desacopla el código de detalles internos del almacenamiento.

```
global $product;

$product_id = $product->get_id();
$title      = $product->get_name();
$price      = $product->get_price();
```

El objeto no es la tarjeta HTML. La tarjeta es una representación visual generada después. `WC_Product` es el objeto de dominio que identifica al producto que WooCommerce está procesando.

14.2 De dónde sale global `$product` en Shop

WooCommerce ejecuta un loop para mostrar los productos del catálogo. Durante cada vuelta deja disponible el producto actual en la variable global `$product` y dispara hooks alrededor de su tarjeta. El tema hijo aprovecha uno de esos hooks, `woocommerce_before_shop_loop_item_title`, para ejecutar `mostrar_videos_shop()` una vez por producto visible. El bloque siguiente es un fragmento abreviado del código fuente: conserva el hook, la condición principal y la lectura real de ACF, pero omite el resto del HTML y JavaScript que construyen el preview.

```
add_action(
    'woocommerce_before_shop_loop_item_title',
    'mostrar_videos_shop',
    10
);

function mostrar_videos_shop() {
    if (!is_shop()) return;

    global $product;
    $video_url = get_field('video_destacado', $product->get_id());
    if (!$video_url) return;

    // Fragmento abreviado: continúa la construcción del preview.
}
```

La consecuencia es importante: `mostrar_videos_shop()` no busca “qué producto corresponde” mediante texto o URL. WooCommerce ya estableció el contexto. La función toma el ID del objeto actual y usa ese mismo ID para consultar ACF.

Ubicación en el proyecto: `functions.php` · hook `woocommerce_before_shop_loop_item_title` alrededor de la línea 2878 · `mostrar_videos_shop()` comienza aproximadamente en la línea 2897.

14.3 Qué aporta WooCommerce y qué agrega ACF

Dato	Origen principal	Uso
ID del producto	WooCommerce / WC_Product	Relaciona catálogo, ACF, carrito y pedido.
Nombre	WooCommerce / WC_Product	Texto comercial y referencias del pedido.
Precio	WooCommerce / WC_Product	Cálculo comercial y total.
Estado/publicación	WordPress + WooCommerce	Determina si el producto participa del catálogo.
video_destacado	ACF asociado al product_id	Preview audiovisual de Shop/Bundles.
tiene_globo	ACF asociado al product_id	Decide si Cart ofrece Comic Balloon Text.

ACF complementa al producto, no lo reemplaza. Si Cyberpunk Portrait tiene `product_id = X`, `get_field('video_destacado', X)` consulta el metadato de ese mismo producto.

14.4 El product_id es el hilo que conecta las etapas

El identificador permite reconocer el mismo producto aunque cambie la estructura que lo representa. En Shop se obtiene desde `WC_Product`; en el carrito el cart item conserva `product_id`; y al crear el pedido WooCommerce genera una línea asociada a ese producto.

```
WC_Product
product_id = X
↓
Shop genera la tarjeta
↓
ADD TO CART envía/reutiliza X
↓
cart item contiene product_id = X
↓
Checkout
↓
WC_Order
↓
WC_Order_Item_Product asociado al mismo producto
```

14.5 Qué significa “vendido individualmente”

El tema registra el filtro `woocommerce_is_sold_individually` y la implementación actual devuelve `true` para todos los productos. WooCommerce interpreta ese resultado como cantidad individual: no se permite manejar libremente una cantidad mayor dentro de la misma línea del carrito.

```
add_filter(
    'woocommerce_is_sold_individually',
    'forzar_artworks_individuales',
    10,
    2
);

function forzar_artworks_individuales($individually, $product) {
    return true;
}
```

No confundir

“Sold individually” no significa que exista una sola copia total del artwork en la base de datos. Significa que, dentro del flujo de compra, WooCommerce fuerza `cantidad = 1` para esa línea y evita el selector normal de cantidades.

Ubicación en el proyecto: `functions.php` · filtro `woocommerce_is_sold_individually` aproximadamente en las líneas 6062-6075.

14.6 Producto, cart item y línea de pedido no son lo mismo

Etapa	Estructura	Qué representa
Catálogo	WC_Product	Producto administrado por WooCommerce: ID, nombre, precio y estado.
Carrito	cart item	Selección temporal que referencia al producto y puede incorporar datos como color_frame o comic_balloon_text.
Pedido	WC_Order_Item_Product	Línea persistida dentro del WC_Order con el producto comprado y sus metadatos copiados.

Esta diferencia explica por qué Frame Color y Comic Balloon Text no deben guardarse cambiando el WC_Product original. Son decisiones de una compra concreta y por eso viajan primero con el cart item y luego se copian a la línea del pedido.

14.7 Recorrido técnico de un producto

```

WooCommerce obtiene el producto
↓
crea/expone WC_Product
↓
hooks del loop permiten al child theme personalizar la tarjeta
↓
ACF agrega datos visuales mediante product_id
↓
usuario pulsa ADD TO CART
↓
WooCommerce crea cart item
↓
el tema agrega personalizaciones al cart item
↓
Checkout valida
↓
WooCommerce crea WC_Order + líneas
↓
hooks copian personalizaciones a WC_Order_Item_Product

```

El tema trabaja principalmente mediante APIs y hooks de WooCommerce en vez de construir SQL manual. Esa decisión es especialmente importante porque la instalación utiliza HPOS para los pedidos, tal como se explicó en la sección 5.

14.8 Ejemplo guiado: Alex compra Cyberpunk Portrait

Supongamos que WooCommerce administra Cyberpunk Portrait a USD 18. El producto tiene un product_id concreto y ACF le asocia un video_destacado y tiene_globo = Sí.

1. Alex abre Shop. Durante el loop WooCommerce coloca Cyberpunk Portrait como \$product actual. mostrar_videos_shop() obtiene su ID y consulta video_destacado con get_field().
2. El tema genera la tarjeta audiovisual, pero el nombre, el precio y el identificador siguen perteneciendo a WC_Product. La presentación cambió; el producto comercial no.
3. Alex pulsa ADD TO CART. WooCommerce agrega el product_id al carrito y crea un cart item. Como el producto se vende individualmente, la línea trabaja con cantidad 1.
4. En Cart, Alex selecciona Frame Color = Blue y escribe Comic Balloon Text = “Welcome back”. Esos valores se asocian a ese cart item; no modifican el producto maestro Cyberpunk Portrait.
5. Alex avanza a Checkout. WooCommerce reconstruye el carrito desde su sesión y las personalizaciones continúan vinculadas a la línea correspondiente.
6. Al confirmar, WooCommerce crea el WC_Order y su WC_Order_Item_Product. Los hooks del tema copian Frame Color y Comic Balloon Text a la línea del pedido para que dejen de depender de la sesión temporal.

7. Más adelante, los logs y los correos pueden recorrer el pedido y mostrar Cyberpunk Portrait junto con Blue y “Welcome back”. El mismo producto atravesó todo el flujo, pero cada etapa utilizó la estructura apropiada.

14.9 Qué no crea el tema hijo

Idea central

blocksy-child no mantiene un catálogo propio paralelo a WooCommerce. Personaliza cómo se consulta, se muestra y se acompaña al producto durante la compra, pero la fuente comercial sigue siendo WooCommerce.

15. Campos propios con ACF

Advanced Custom Fields (ACF) permite agregar datos propios a los productos. La sección 5 explicó cómo se almacenan; aquí interesa cómo los usa el tema hijo para decidir qué video mostrar, si habilitar Comic Balloon Text y qué productos componen un bundle.

Origen del código que se muestra en esta sección: los valores ACF no se leen desde una única clase propia. El tema los consulta desde functions.php en los flujos que los necesitan: Shop usa video_destacado dentro de mostrar_videos_shop(); Bundles usa productos_del_bundle y video_destacado; Cart/Emails consultan tiene_globo. Cada subsección debe leerse como una llamada get_field() realizada desde el flujo técnico indicado, no como un proceso independiente de ACF.

Campo ACF	Qué representa	Ejemplo de uso
video_destacado	Video principal asociado al producto.	La tarjeta de Shop y Bundles lo usa como vista previa.
video_workshop	Video Workshop opcional.	Campo opcional disponible para almacenar un recurso Workshop asociado al producto.
productos_del_bundle	Relaciona un bundle con otros productos.	La tarjeta del bundle recorre los artworks asociados.
tiene_globo	Indica si el artwork admite texto de globo.	Si está activo, Cart muestra Comic Balloon Text.

Qué significa metadato

Un metadato es información adicional asociada a otro dato principal. En este caso el producto de WooCommerce es el elemento principal y “tiene globo” o “video destacado” son datos adicionales que describen cómo debe comportarse o mostrarse.

Ejemplo

Producto: Cyberpunk Portrait → video_destacado = /uploads/.../cyberpunk.mp4 → tiene_globo = Sí. El mismo producto puede tener comportamiento audiovisual y opciones propias sin dejar de ser un producto simple de WooCommerce.

15.1 Lectura de video_destacado

La Tienda obtiene el video principal con get_field(). Si el campo no tiene valor, mostrar_videos_shop() termina y no intenta crear un reproductor vacío.

```
$video_url = get_field('video_destacado', $product->get_id());
if (!$video_url) return;
```

15.2 Campo tiene_globo

tiene_globo funciona como una decisión de interfaz. Cuando está activado, el producto admite Comic Balloon Text en Cart y ese dato puede pasar al pedido. Cuando está desactivado, el texto de globo se oculta en los resúmenes donde no corresponde.

```
$tiene_globo = (bool) get_field('tiene_globo', $product_id);

if ($tiene_globo) {
```

```
// mostrar o conservar Comic Balloon Text
}
```

15.3 Relación productos_del_bundle

Un bundle utiliza `productos_del_bundle` para conocer qué artworks debe representar dentro de su tarjeta. El bundle sigue siendo el producto que se compra; los productos relacionados sirven para mostrar visualmente su contenido.

```
$productos_in_bundle = get_field('productos_del_bundle', $bundle_id);

foreach ($productos_in_bundle as $prod) {
    $video_url = get_field('video_destacado', $prod_id);
}
```

15.4 Campo video_workshop

`video_workshop` está disponible como campo adicional para asociar un recurso Workshop a un producto. En la versión actual no participa del flujo principal de Shop, Cart o Checkout, por lo que se documenta como dato disponible y no como requisito para comprar.

Diferencia entre el código y la configuración ACF

El filtro `wpo_filtrar_meta_en_pedido()` espera un campo ACF llamado `es_bundle` y el propio comentario de `functions.php` lo presenta como una dependencia junto con `tiene_globo`. Sin embargo, el relevamiento actual de la instalación sólo identifica cuatro campos ACF de producto y `es_bundle` no aparece entre ellos. Si ese campo no existe, `get_field('es_bundle', $product_id)` devuelve un valor vacío y esta función deja de reconocer bundles por esa vía; el filtrado de Comic Balloon Text depende entonces de `tiene_globo`. En otras partes del proyecto los bundles se identifican por categoría o por nombre. Esta diferencia forma parte del estado actual del proyecto.

16. Tienda y tarjetas de producto

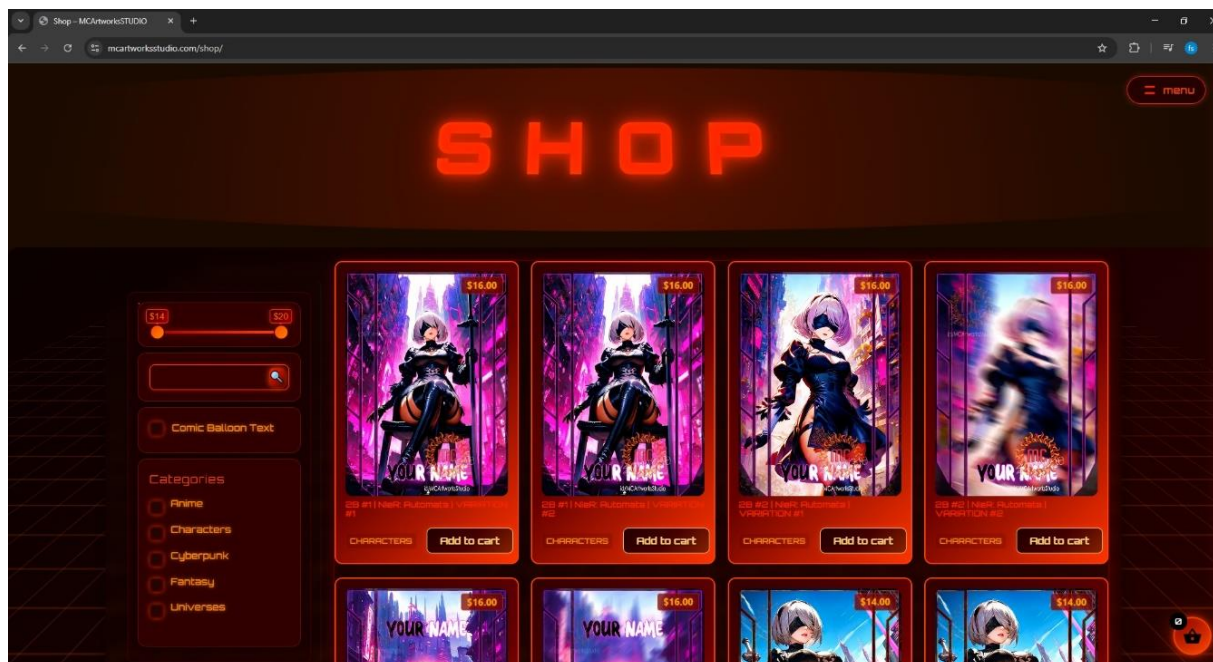
La Tienda utiliza el catálogo y el loop estándar de WooCommerce como base, pero MC Artworks Studio modifica qué productos participan, cuántos se muestran por página y cómo se representa visualmente cada artwork.

WooCommerce mantiene la consulta, los objetos de producto y los botones comerciales; el tema hijo agrega la grilla, los videos, los posters, el precio superpuesto, las animaciones y el comportamiento audiovisual.

Ubicación en el proyecto: `wp-content/themes/blocksy-child/functions.php` · `ocultar_bundles_de_shop()` se registra en `pre_get_posts` alrededor de la línea 2844; `mostrar_videos_shop()` se conecta a `woocommerce_before_shop_loop_item_title` alrededor de la línea 2878 y comienza cerca de la línea 2897; `productos_por_pagina()` se registra en `loop_shop_per_page` alrededor de las líneas 3315-3320. La animación de entrada de Shop se activa desde `wp_footer` alrededor de la línea 2021.

Cómo leer los fragmentos de esta sección. WooCommerce crea la consulta y el loop; los hooks de `functions.php` intervienen en momentos concretos de ese recorrido. Por eso un `add_action()` o `add_filter()` mostrado aquí registra un callback: no significa que ese código se ejecute en el instante en que WordPress lee la línea.

```
WooCommerce consulta productos
↓
se excluye la categoría Bundles
↓
WooCommerce genera el loop
↓
<ul class="products">
↓
<li class="product"> por cada artwork
↓
el tema hijo agrega video + precio + efectos
```



Tienda NOD: sidebar de filtros y grilla de artworks con vistas previas en video.

16.1 Qué productos participan en la Tienda

Antes de ejecutar la consulta principal, el tema usa el hook `pre_get_posts` para excluir los productos pertenecientes a la categoría Bundles. De esta forma los bundles continúan siendo productos de WooCommerce, pero se reservan para su página específica.

```
add_action('pre_get_posts', 'ocultar_bundles_de_shop');

$tax_query[] = [
    'taxonomy' => 'product_cat',
    'field'    => 'slug',
    'terms'    => ['bundles'],
    'operator' => 'NOT IN',
];
```

La condición `NOT IN` significa que los productos asociados a la categoría Bundles quedan fuera del loop principal de la Tienda.

La función completa no sólo agrega `NOT IN`. Primero limita el cambio a la consulta principal de Shop y conserva cualquier `tax_query` que WordPress o WooCommerce ya hayan agregado. Esto evita modificar consultas secundarias o pantallas del administrador.

```
function ocultar_bundles_de_shop($query) {
    if (!is_admin() && $query->is_main_query() && is_shop()) {
        $tax_query = (array) $query->get('tax_query');
        $tax_query[] = [
            'taxonomy' => 'product_cat',
            'field'    => 'slug',
            'terms'    => ['bundles'],
            'operator' => 'NOT IN',
        ];
        $query->set('tax_query', $tax_query);
    }
}
```

Cómo se lee el ejemplo. `!is_admin()` evita tocar el panel; `is_main_query()` limita el cambio al loop principal; `is_shop()` lo restringe a la Tienda; `get()` conserva condiciones existentes; `NOT IN` excluye Bundles; `set()` devuelve la consulta modificada a WordPress.

```

WordPress prepara consulta Shop
↓
pre_get_posts
↓
¿frontend + consulta principal + Shop?
↓ sí
conserva tax_query existente
↓
agrega NOT IN bundles
↓
WooCommerce ejecuta la consulta

```

16.2 Cantidad de productos por página

WooCommerce controla la paginación del catálogo mediante el filtro `loop_shop_per_page`. MC Artworks Studio fija el valor en 28 productos por página y utiliza una prioridad alta para que esa cantidad prevalezca frente a otras modificaciones del mismo filtro.

```

add_filter('loop_shop_per_page', 'productos_por_pagina', 999);

function productos_por_pagina($cols) {
    return 28;
}

```

16.3 WooCommerce construye el loop; CSS construye la grilla

WooCommerce genera una lista `ul.products` y un `li.product` por cada registro. El tema hijo no reemplaza ese mecanismo: aplica CSS Grid sobre la estructura existente para convertirla en una grilla adaptable.

```

body.woocommerce-shop ul.products {
    display: grid;
    grid-template-columns:
        repeat(auto-fill, minmax(220px, 1fr));
    gap: 20px;
}

```

`repeat(auto-fill, minmax(220px, 1fr))` permite que el navegador calcule cuántas columnas entran según el ancho disponible. En pantallas reducidas la grilla pasa a una sola columna. El sidebar de filtros ocupa el sector izquierdo y puede mantenerse visible mediante `position: sticky` mientras el usuario recorre los resultados.

16.4 Construcción de una tarjeta audiovisual

WooCommerce ejecuta el hook `woocommerce_before_shop_loop_item_title` antes del título de cada producto. El tema conecta allí `mostrar_videos_shop()`, por lo que la función se ejecuta una vez por cada artwork visible en el loop.

```

add_action(
    'woocommerce_before_shop_loop_item_title',
    'mostrar_videos_shop',
    10
);

```

La función obtiene el producto actual, consulta el campo ACF `video_destacado` y, si existe, genera un enlace que funciona al mismo tiempo como vista previa de la tarjeta y como origen del video que Fancybox podrá ampliar.

```

global $product;

$video_url = get_field(
    'video_destacado',
    $product->get_id()
);

if (!$video_url) return;

```

16.5 Poster según la facción

Antes de que el MP4 esté listo se muestra un poster. La ruta se decide con la cookie faction: GDI utiliza irina_poster.jpg y NOD utiliza poster.jpg, ambos dentro de wp-content/uploads/images. Así la misma tarjeta conserva la identidad visual seleccionada incluso durante la carga inicial del video.

```
$faction = $_COOKIE['faction'] ?? 'gdi';

if ($faction == 'gdi')
    $poster_path = $uploads_dir['baseurl'] . '/images/irina_poster.jpg';
else
    $poster_path = $uploads_dir['baseurl'] . '/images/poster.jpg';
```

16.6 HTML generado para video, precio y Fancybox

```
<a href="VIDEO.mp4"
  data-fancybox="videos-tienda"
  class="video-loop-link video-loop-wrapper">

  <div class="video-container">
    <video
      poster="POSTER.jpg"
      data-src="VIDEO.mp4"
      preload="metadata"
      muted autoplay playsinline
      class="video-loop">
    </video>

    <span class="price precio-video">
      PRECIO
    </span>
  </div>
</a>
```

La URL del MP4 se utiliza de dos maneras: href permite abrir el artwork en Fancybox y data-src conserva la fuente de la vista previa sin obligar al navegador a activarla inmediatamente. El precio se obtiene desde WooCommerce con get_price() y se imprime mediante wc_price().

16.7 Título y botones comerciales

El título del producto se imprime con la clase estándar woocommerce-loop-product__title, pero sin convertirlo en enlace. Los botones Add to Cart siguen siendo botones WooCommerce, lo que permite conservar AJAX, validaciones, eventos added_to_cart y la integración con Xootix.

```
echo '<h2 class="woocommerce-loop-product__title">'
    . esc_html($product->get_title())
    . '</h2>';
```

Los botones comerciales no son un sistema de carrito nuevo. Siguen siendo controles de WooCommerce. El tema cambia su estado visual según si el producto ya está dentro de WC()->cart y escucha los eventos AJAX para actualizar la tarjeta sin recargar la página.

Producto fuera del carrito
→ ADD TO CART

```
click
↓
WooCommerce AJAX
↓
producto agregado
↓
evento added_to_cart
↓
ADD TO CART se oculta
↓
aparece VIEW CART
```

Ejemplo claro. Alex agrega Cyberpunk Portrait. WooCommerce conserva el producto en su carrito real. Xootix abre el panel lateral y la misma tarjeta de Shop deja de ofrecer ADD TO CART para mostrar VIEW CART. Si Alex elimina el producto desde Xootix, la tarjeta vuelve a habilitar ADD TO CART.

16.8 Categoría mostrada junto al producto

El tema puede recuperar las categorías asociadas al producto para mostrarlas junto a los controles comerciales. La información sale de las taxonomías de WooCommerce; no se duplica en un campo propio del tema.

```
$terms = get_the_terms($product_id, 'product_cat');
// Se utilizan las categorías devueltas por WooCommerce para la interfaz.
```

16.9 Clase visual cuando existe video

El filtro `post_class` puede agregar la clase `tiene-video` a la tarjeta cuando existe `video_destacado`. Esa clase permite que CSS distinga una tarjeta audiovisual de una tarjeta que dependería de la imagen estándar de WooCommerce.

```
producto
↓
{video_destacado tiene valor?
├ sí → class="tiene-video" → se prioriza la vista previa en video
└ no → se conserva el comportamiento visual disponible
```

16.10 Animaciones aleatorias de entrada

La Tienda dispone de cinco funciones de animación. Al final de la página, `animacion_random_shop()` verifica que realmente existan productos y selecciona una variante al azar. Las animaciones controlan el sidebar y la aparición de las tarjetas, incluyendo retardos progresivos que producen un efecto de cascada.

```
$animaciones = [
    'animacion_shop_uno',
    'animacion_shop_dos',
    'animacion_shop_tres',
    'animacion_shop_cuatro',
    'animacion_shop_cinco'
];

$seleccion = $animaciones[array_rand($animaciones)];
```

17. Carga progresiva de videos y Fancybox

Una página de catálogo puede contener decenas de MP4. Para no descargar todos al mismo tiempo, la Tienda usa carga progresiva: el HTML conserva la URL en `data-src` y JavaScript decide cuándo convertirla en una fuente multimedia real. El mismo video puede abrirse después en un modal Fancybox sin abandonar la grilla.

Ubicación en el proyecto: `wp-content/themes/blocksy-child/functions.php` · `cargar_fancybox_tienda()` se registra en `wp_enqueue_scripts` cerca del inicio del archivo; `video_lazy_loop_shop()` se registra en `wp_footer` alrededor de la línea 2940; `fix_videos_chrome_shop_universal()` alrededor de la 3039; `custom_fancybox_video_script()` alrededor de la 3972; `cerrar_fancybox_con_jquery()` alrededor de la 4142 y `cf_fix_cart_over_fancybox()` alrededor de la 4173.

17.1 Por qué se utiliza data-src

```
<video
  data-src="artwork.mp4"
  preload="metadata"
  muted
  playsinline>
</video>
```

Mientras la dirección permanezca en `data-src`, el tema puede decidir cuándo activarla. El navegador no recibe un `src` directo desde el primer momento para todas las tarjetas.

```

HTML inicial
  data-src = artwork.mp4
  src      = vacío
    ↓
JavaScript detecta proximidad
    ↓
  src = data-src
    ↓
comienza la carga real

```

17.2 Videos visibles al abrir la Tienda

Al producirse `DOMContentLoaded` se buscan todos los `video[data-src]`. Para cada elemento se calcula su posición mediante `getBoundingClientRect()`. Si ya está dentro del viewport, `reproducirVideo(video, true)` lo prepara con un retraso inicial de aproximadamente 850 ms.

```

const rect = video.getBoundingClientRect();
const inViewport =
  rect.top < window.innerHeight
  && rect.bottom > 0;

if (inViewport) {
  reproducirVideo(video, true);
}

```

Ese pequeño retraso mantiene el poster visible mientras el navegador termina de preparar los primeros recursos y evita una entrada brusca de varios videos simultáneos.

17.3 IntersectionObserver para los videos que todavía no se ven

Los videos situados más abajo se entregan a `IntersectionObserver`. La opción `rootMargin: "200px"` amplía la zona de detección, de modo que el recurso puede empezar a cargarse antes de entrar físicamente en pantalla.

```

const observer = new IntersectionObserver(
  (entries, obs) => {
    entries.forEach(entry => {
      if (entry.isIntersecting) {
        reproducirVideo(entry.target, false);
        obs.unobserve(entry.target);
      }
    });
  },
  { rootMargin: "200px" }
);

```

Una vez activado el video deja de observarse porque ya no necesita volver a pasar por el proceso de lazy loading.

17.4 Activación del MP4

```

if (!video.src) {
  video.src = video.getAttribute('data-src');
}

video.preload = 'auto';
video.currentTime = 0.01;
video.play();

```

El paso `data-src` → `src` es el momento concreto en que el navegador recibe la fuente real. La reproducción comienza cerca del primer fotograma y la clase `active` produce un fade de opacidad de aproximadamente 0,35 segundos.

17.5 Loop manual

El código también controla el final de cada preview. Cuando faltan aproximadamente 0,05 segundos para terminar, pausa, vuelve a 0,01 s y solicita la siguiente reproducción mediante `requestAnimationFrame()`. Con esto se busca que el salto entre ciclos sea más estable.

```
video.addEventListener('timeupdate', () => {
  if (video.currentTime >= video.duration - 0.05) {
    video.pause();
    video.currentTime = 0.01;
    requestAnimationFrame(() => video.play());
  }
});
```

17.6 Tratamiento de salida en Chrome y Opera

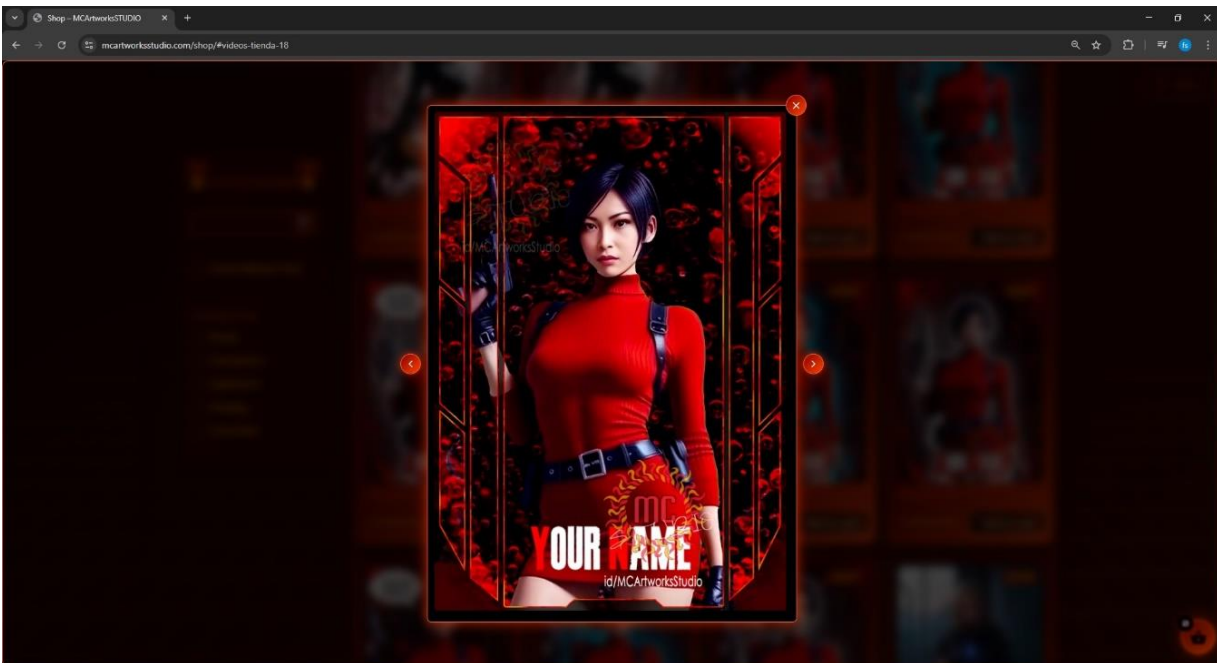
La función `fix_videos_chrome_shop_universal()` actúa en la Tienda para evitar cuadros negros al abandonar la página en Chrome u Opera. Antes de navegar intenta capturar el fotograma visible en un canvas; si no puede hacerlo, utiliza el poster como respaldo. También contempla por separado los videos que Xootix puede tener dentro del carrito lateral.

```
video visible
↓
canvas.drawImage(video, ...)
↓
fotograma queda superpuesto
↓
se inicia la navegación
↓
no aparece un cuadro negro
```

17.7 Carga condicional de Fancybox

Fancybox se registra mediante `wp_enqueue_scripts` únicamente cuando WordPress está mostrando la Tienda, una categoría de producto o una etiqueta de producto. La librería se obtiene desde jsDelivr.

```
if (
  is_shop()
  || is_product_category()
  || is_product_tag()
) {
  wp_enqueue_style('fancybox-css', ...);
  wp_enqueue_script('fancybox-js', ...);
}
```



Vista ampliada de un artwork mediante el modal de video de Fancybox.

17.8 Cómo se abre una tarjeta

Cada tarjeta contiene `data-fancybox="videos-tienda"`. El script enlaza esos elementos con `Fancybox.bind()` y, cuando detecta un MP4, reemplaza el contenido del slide por un reproductor HTML5 propio.

```
Fancybox.bind(
  '[data-fancybox="videos-tienda"]',
  {
    dragToClose: false,
    width: '90vw',
    height: '80vh'
  }
);
```

17.9 Modal personalizado y navegación

El modal agrega su propio botón de cierre y dos botones laterales. Las acciones llaman a `fancybox.prev()` y `fancybox.next()`; también se aceptan `ArrowLeft` y `ArrowRight` desde el teclado. Esto permite recorrer los videos de la Tienda sin cerrar la ventana ampliada entre un artwork y otro.

```
Artwork anterior ← VIDEO AMPLIADO → Artwork siguiente

ArrowLeft → prev()
ArrowRight → next()
```

17.10 Bloqueo del fondo e integración con Xootix

Al revelar Fancybox se añade la clase `fancybox-lock` al body y una capa transparente bloquea la interacción con el fondo. Al cerrar, ambos elementos se retiran. Otra función, `cf_fix_cart_over_fancybox()`, permite que el carrito lateral de Xootix pueda aparecer por encima del modal sin reinicializar Fancybox.

```
Fancybox abierto
↓
usuario abre carrito
↓
Xootix recibe prioridad visual
↓
el carrito sigue siendo interactivo
↓
Fancybox permanece abierto detrás
```

17.11 Ejemplo guiado: Alex baja por Shop y abre un artwork

Alex entra en Shop. PHP ya generó las tarjetas con `data-src`, pero muchos MP4 todavía no tienen `src` activo. Los videos que están visibles se preparan al cargar la página y los que quedan más abajo permanecen bajo `IntersectionObserver`.

Cuando Alex hace scroll, una tarjeta entra en la zona ampliada por `rootMargin`. JavaScript copia `data-src` a `src`, llama `load()/play()` y deja de observar ese elemento. Si Alex pulsa la vista previa, el enlace `data-fancybox` entrega la URL del mismo MP4 a Fancybox, que crea el reproductor ampliado sin sacar a Alex de Shop.

```
Alex abre Shop
↓
PHP entrega tarjetas con data-src
↓
Alex hace scroll
↓
IntersectionObserver detecta proximidad
↓
data-src pasa a src
↓
se reproduce la vista previa
↓
Alex pulsa el video
↓
Fancybox.bind() abre el modal
↓
Alex navega Previous / Next o cierra
```

18. Filtros de la Tienda con HUSKY y AJAX

HUSKY/WOOF aporta el motor de filtros para WooCommerce. MC Artworks Studio utiliza esa infraestructura y agrega lógica propia para textos, categorías visibles, búsqueda por texto, bloqueo de interfaz, rango de precios y restauración de elementos después de las actualizaciones dinámicas.

Ubicación en el proyecto: wp-content/themes/blocksy-child/functions.php · las adaptaciones de HUSKY se distribuyen principalmente entre las líneas aproximadas 1282 y 1955 mediante callbacks de wp_footer/wp_head; el estado personalizado de “sin resultados” se registra posteriormente desde init alrededor de la línea 2646. El plugin HUSKY genera sus propios controles y AJAX; functions.php modifica su interfaz y reengancha comportamientos cuando el plugin reemplaza el DOM.

18.1 Categorías visibles

El tema cambia el encabezado del filtro a Categories y elimina del listado visual las opciones Uncategorized y Bundles. Bundles no se ofrece aquí porque posee su propia página y su propio sistema de filtros.

```
const categoriasOcultas = [
  'Uncategorized',
  'Bundles'
];
```

18.2 Comic Balloon Text dentro de HUSKY

El filtro asociado al campo tiene_globo puede aparecer después de que HUSKY reconstruye parte del DOM. Por eso un MutationObserver espera el label correspondiente y reemplaza su texto por Comic Balloon Text. Una vez realizada la modificación, el observer se desconecta.

```
const label = document.querySelector(
  'label[for="woof_meta_checkbox_tiene_globo"]'
);

label.textContent = 'Comic Balloon Text';
```

18.3 Buscador y botón de lupa

El tema localiza .woof_husky_txt-input y agrega dinámicamente un botón de búsqueda. Como HUSKY puede recrear ese input después de aplicar filtros, otro MutationObserver detecta la nueva versión y vuelve a colocar la lupa sin duplicarla.

El botón no se agrega una sola vez al cargar la página. HUSKY puede reemplazar el input durante una actualización AJAX, por eso el código observa nodos nuevos y vuelve a enlazar la lupa cuando detecta una nueva instancia del buscador.

```
function findInput() {
  return document.querySelector('.woof_husky_txt-input');
}

function agregarLupaSiInputListoEl(inputEl) {
  if (!inputEl) return;
  if (inputEl.parentElement?.querySelector('#btn_search')) return;

  const btn = document.createElement('button');
  btn.id = 'btn_search';
  btn.type = 'button';
  btn.textContent = '🔍';
  inputEl.parentElement.appendChild(btn);
}

const observer = new MutationObserver(mutations => {
  for (const m of mutations) {
    for (const node of m.addedNodes) {
      if (!(node instanceof HTMLElement)) continue;
      const found = node.matches?.('.woof_husky_txt-input')
        ? node
        : node.querySelector?.('.woof_husky_txt-input');
```

```

    if (found) agregarLupaSiInputListoEl(found);
  }
}
});

observer.observe(document.body, { childList: true, subtree: true });

```

18.4 Mínimo de tres caracteres

La búsqueda textual exige al menos tres caracteres tanto al pulsar la lupa como al presionar Enter. Si el valor es demasiado corto, la ejecución se detiene y se abre la alerta personalizada.

```

const val = (input.value || '').trim();

if (val.length < 3) {
  mostrarAlertaPersonalizada(
    'Please enter at least 3 characters.'
  );
  return false;
}

```

Ejemplo: “cy” se bloquea por tener dos caracteres; “cyr” puede continuar hacia el flujo de HUSKY.

18.5 Intercepción de woof_submit_link()

HUSKY puede ejecutar directamente su función global woof_submit_link(). Para asegurarse de que la validación también se respete en ese camino, el proyecto conserva la función original y coloca una función propia delante. Si el texto es válido se llama a original.call(this, link); si no, se cancela el filtro.

```

const original = window.woof_submit_link;

window.woof_submit_link = function(link) {
  ... validación ...
  return original.call(this, link);
};

```

18.6 Qué aporta AJAX

Cuando HUSKY aplica un filtro puede actualizar la zona de resultados sin recargar toda la página. WooCommerce vuelve a obtener los productos compatibles y el plugin reemplaza la parte correspondiente de la grilla. El menú, el fondo y el resto de la interfaz permanecen en la misma página.

```

usuario cambia filtro
↓
HUSKY prepara parámetros
↓
petición AJAX
↓
WooCommerce devuelve resultados
↓
HUSKY reemplaza la grilla

```

18.7 Bloqueo visual mientras se filtra

Al interactuar con checkboxes, radios, precio o el botón de filtros se añade loading-active al body. Un screen-blocker evita nuevas interacciones mientras el cambio está siendo procesado. También se aplica bloqueo durante beforeunload para cubrir los casos en que la operación termina en una navegación completa.

El bloqueo se activa únicamente cuando la interacción pertenece a un control de HUSKY que puede iniciar una actualización. El mismo estado se fuerza en beforeunload para cubrir los casos en que el filtro termina provocando una navegación completa.

```

const blocker = document.createElement('div');
blocker.className = 'screen-blocker';
document.body.appendChild(blocker);

```

```
function activarBloqueo() {
  if (!document.body.classList.contains('loading-active')) {
    document.body.classList.add('loading-active');
  }
}

const validSelectors = [
  '.woof_checkbox',
  '.woof_radio',
  '.woof_price_filter input',
  '.woof_submit_search_form'
].join(',');

document.addEventListener('click', e => {
  if (e.target.closest(validSelectors)) activarBloqueo();
});

document.addEventListener('change', e => {
  if (e.target.closest(validSelectors)) activarBloqueo();
});

window.addEventListener('beforeunload', () => {
  document.body.classList.add('loading-active');
});
```

18.8 Rango de precios

La Tienda fuerza el slider de HUSKY a un rango general de USD 14–20. Si la URL ya contiene un rango, por ejemplo price-14-to-18, JavaScript recupera esos valores y actualiza ionRangeSlider. Después del evento woof_ajax_done el rango se vuelve a aplicar, porque HUSKY puede reconstruir el control.

```
slider.setAttribute('data-min', '14');
slider.setAttribute('data-max', '20');

jQuery(document).on(
  'woof_ajax_done',
  () => setTimeout(forzarRangoWoof, 300)
);
```

18.9 Sin resultados

Cuando no quedan productos, el mensaje estándar de WooCommerce se reemplaza por una presentación propia. La imagen cambia según la facción y el resultado se trata como un estado normal de búsqueda, no como un error del sitio.

NOD → imagen de “sin resultados” NOD
GDI → imagen de “sin resultados” GDI

18.10 Ejemplo guiado: Alex utiliza la búsqueda de HUSKY

Alex escribe “cy” en el buscador. El listener propio detecta dos caracteres, detiene el envío y muestra la alerta del proyecto. Después escribe “cyr”. La validación permite continuar y HUSKY ejecuta su flujo de filtrado.

Si HUSKY actualiza la grilla mediante AJAX, puede reemplazar nodos completos. Por eso los MutationObserver y el evento woof_ajax_done vuelven a aplicar piezas como la lupa, el texto Comic Balloon Text y el rango de precio después de la actualización.

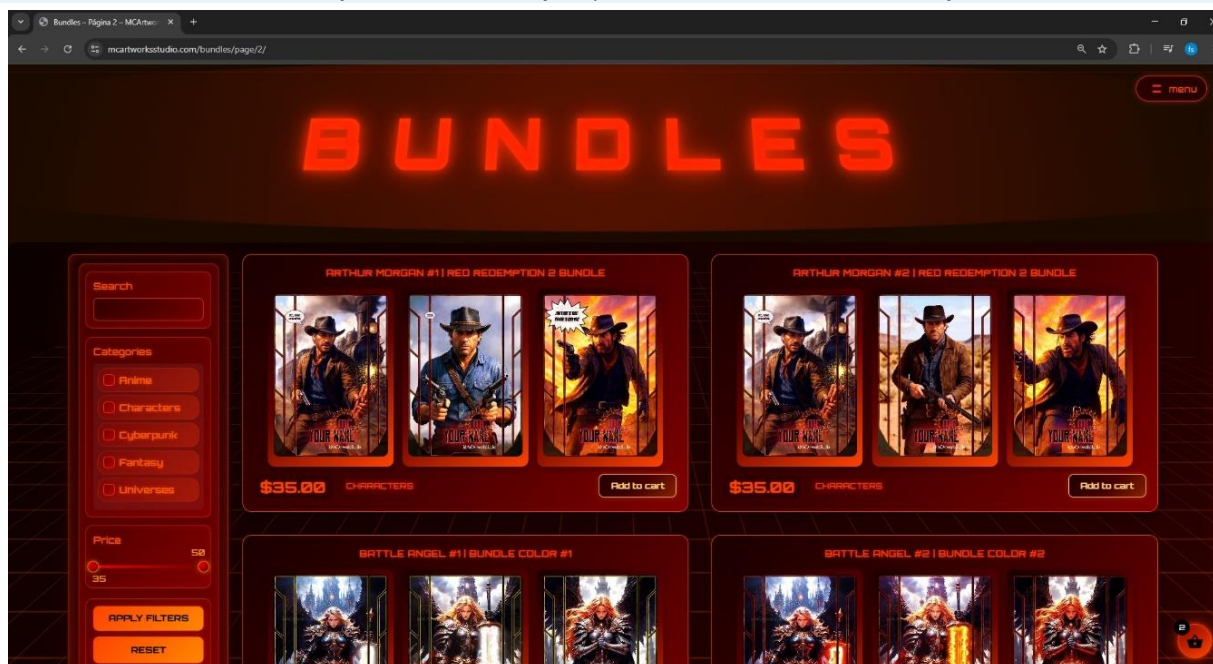
```
Alex escribe “cyr”
↓
validación propia: 3 caracteres o más
↓
HUSKY prepara filtros
↓
petición AJAX
↓
WooCommerce obtiene productos compatibles
```

↓
 HUSKY reemplaza la grilla
 ↓
 callbacks/observers del tema vuelven a aplicar ajustes

19. Bundles

La página Bundles utiliza productos WooCommerce de la categoría Bundles, pero no reutiliza visualmente el loop estándar de la Tienda. La función `mostrar_bundles_filtrados_rapido()` reemplaza el contenido de esa página con un sidebar propio, consulta los bundles mediante `wc_get_products()`, aplica filtros, pagina los resultados y construye manualmente cada bloque con sus artworks relacionados.

Ubicación en el proyecto: `wp-content/themes/blocksy-child/functions.php · add_filter('the_content', 'mostrar_bundles_filtrados_rapido', 21)` aparece alrededor de la línea 4454 y la función comienza cerca de la 4466. El lazy loading de sus videos se conecta desde `wp_footer` alrededor de la línea 4828; otras sincronizaciones visuales y de carrito se distribuyen aproximadamente entre las líneas 5401 y 5795.



Bundles NOD: filtros propios y bloques con varios artworks relacionados.

19.1 Parámetros de filtro

La página recibe búsqueda, precio y categorías mediante parámetros GET. Los valores se sanitizan antes de usarse. Las categorías seleccionadas se comparan con los slugs reales de `product_cat` para descartar valores que no pertenezcan a la taxonomía.

```
$q          = sanitize_text_field(...);
$min_price = floatval(...);
$max_price = floatval(...);
$cats_raw  = (array) $_GET['cat'];
```

19.2 Consulta de productos

`wc_get_products()` obtiene todos los productos publicados que pertenecen a Bundles. Si el usuario selecciona categorías adicionales, `tax_query` utiliza `relation = AND` para exigir que el bundle pertenezca también a esas categorías.

```
$all_bundles = wc_get_products([
    'status'    => 'publish',
    'limit'     => -1,
    'orderby'   => 'title',
    'order'     => 'ASC',
    'tax_query' => $tax_query,
]);
```

19.3 Filtrado manual por texto y precio

Una vez recuperados los bundles, PHP recorre los objetos y conserva únicamente los que están dentro del rango de precio y cuyo nombre coincide con la búsqueda, si se escribió alguna.

```
if (
    $price >= $min_price
    && $price <= $max_price
    && ($q === '' || stripos($name, $q) !== false)
) {
    $filtered_products[] = $bundle;
}
```

19.4 Paginación de 12 bundles

Después del filtrado se calcula la página actual, el desplazamiento y el bloque que debe mostrarse. La página contiene hasta 12 bundles.

```
$per_page = 12;
$offset = ($paged - 1) * $per_page;

$filtered_products = array_slice(
    $filtered_products,
    $offset,
    $per_page
);
```

Si existen 30 resultados compatibles, las páginas quedan distribuidas como 1–12, 13–24 y 25–30. Los enlaces de paginación conservan los parámetros de filtro para que cambiar de página no borre la selección actual.

19.5 Sidebar propio

A diferencia de la Tienda, Bundles no depende de HUSKY para su interfaz de filtros. El proyecto construye directamente un formulario con Search, Categories, Price, Apply Filters y Reset. El rango general de precio se trabaja entre USD 35 y USD 50 y los valores finales se envían mediante inputs ocultos.

El sidebar es un formulario GET propio. Search viaja en q, las categorías en cat[] y los límites de precio en campos ocultos. Apply Filters vuelve a solicitar la página con esos parámetros; Reset abre nuevamente /bundles/ sin filtros.

```
<form method="get" action="<?php echo esc_url($bundles_url); ?>">
    <label for="q">Search</label>
    <input id="q" name="q" type="text"
        value="<?php echo esc_attr($q); ?>">

    <?php foreach ($cats_list as $term): ?>
        <label>
            <input type="checkbox" name="cat[]"
                value="<?php echo esc_attr($term->slug); ?>"
                <?php checked(in_array($term->slug, $cats, true)); ?>>
            <span><?php echo esc_html($term->name); ?></span>
        </label>
    <?php endforeach; ?>

    <input type="hidden" name="min_price" id="min_price_input"
        value="<?php echo esc_attr($min_price); ?>">
    <input type="hidden" name="max_price" id="max_price_input"
        value="<?php echo esc_attr($max_price); ?>">
```

```
<button type="submit" class="button">Apply Filters</button>
<button type="button" class="button"
  onclick="window.location.href='https://mcartworksstudio.com/bundles/';">
  Reset
</button>
</form>
```

19.6 Relación productos_del_bundle

Cada bundle recupera el campo ACF productos_del_bundle. La relación devuelve los artworks asociados; para cada elemento se obtiene el producto WooCommerce y su video_destacado.

```
$productos_in_bundle = get_field(
  'productos_del_bundle',
  $bundle_id
);

foreach ($productos_in_bundle as $prod) {
  $video_url = get_field(
    'video_destacado',
    $prod_id
  );
}
```

El producto que se agrega al carrito es el bundle. Los artworks relacionados sirven para representar visualmente su contenido dentro de la tarjeta.

19.7 Lazy loading de los videos internos

Los videos de Bundles utilizan una variante del lazy loading de la Tienda. La URL queda en <source data-src="..."> y el elemento <video> se crea con preload="none" y un poster de facción.

```
<video class="lazy-video"
  autoplay muted loop playsinline
  preload="none"
  poster="POSTER">
  <source data-src="VIDEO.mp4"
    type="video/mp4">
</video>
```

Cuando IntersectionObserver detecta el video, JavaScript copia data-src a source.src, llama a video.load() y espera loadeddata para ejecutar play(). El observer usa un margen de 200 px y threshold 0.25.

19.8 MutationObserver para contenido dinámico

Además de observar los videos existentes al cargar la página, Bundles crea un MutationObserver sobre el body. Si aparece un nuevo .lazy-video dentro del DOM, también se registra en IntersectionObserver. Esto permite mantener el mismo comportamiento aunque se inserten nodos posteriormente.

El MutationObserver no carga los MP4 directamente. Su tarea es registrar cualquier .lazy-video nuevo en el IntersectionObserver ya existente; ese segundo observer será quien decida cuándo asignar data-src a src y comenzar la descarga.

```
const mutationObserver = new MutationObserver(mutations => {
  mutations.forEach(mutation => {
    mutation.addedNodes.forEach(node => {
      if (node.nodeType !== 1) return;

      if (node.matches('.lazy-video')) {
        observer.observe(node);
      } else {
        node.querySelectorAll?('.lazy-video')
          .forEach(v => observer.observe(v));
      }
    });
  });
});
```

```
});

mutationObserver.observe(document.body, {
  childList: true,
  subtree: true
});
```

19.9 Animaciones y botón Add to Cart

Si hay resultados, se elige al azar una de cinco animaciones de Bundles. Cada bloque genera además un botón WooCommerce con las clases `add_to_cart_button` y `ajax_add_to_cart`, de modo que el producto participa en el mismo flujo AJAX, límite global y carrito lateral que los artworks individuales.

```
class="button add_to_cart_button
      ajax_add_to_cart product_type_simple"
```

19.10 Ejemplo guiado: Alex filtra Bundles y agrega uno

Alex abre `/bundles/` y escribe un texto o modifica el rango de precio. El formulario GET vuelve a solicitar la página con esos parámetros. WordPress ejecuta el filtro `the_content` y `mostrar_bundles_filtrados_rapido()` lee y sanitiza los valores, obtiene los productos de la categoría Bundles y aplica el filtrado y la paginación.

Cuando Alex pulsa ADD TO CART sobre un bundle, el botón conserva las clases AJAX de WooCommerce. El producto que entra al carrito es el `WC_Product` del bundle; `productos_del_bundle` sólo se utiliza para mostrar los artworks relacionados dentro de la tarjeta.

```
Alex aplica filtros en /bundles/
↓
GET q / cat[] / precio
↓
mostrar_bundles_filtrados_rapido()
↓
wc_get_products()
↓
filtrado + paginación
↓
PHP genera tarjetas y videos lazy
↓
Alex pulsa ADD TO CART
↓
WooCommerce agrega el producto bundle
```

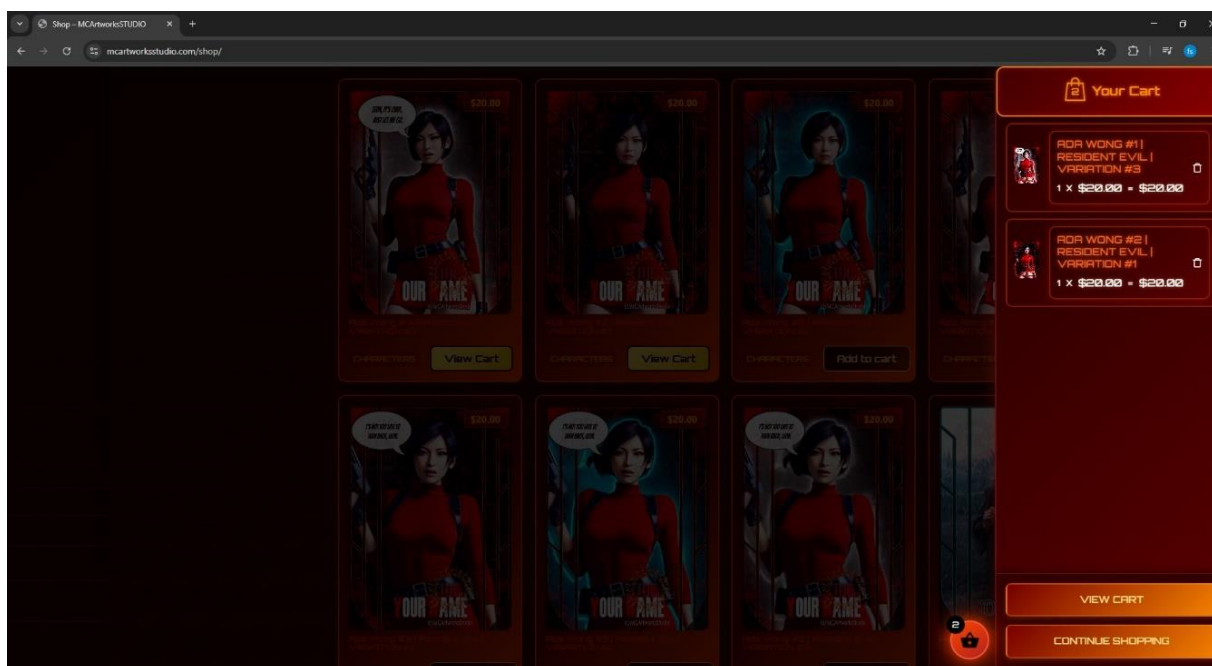
PARTE IV — CARRITO, CHECKOUT Y PEDIDOS

20. Carrito lateral y límite de artículos

Xootix Side Cart muestra el carrito como un panel lateral actualizado dinámicamente. WooCommerce conserva el carrito real y procesa las operaciones; Xootix representa ese estado sin obligar a recargar la página; el tema hijo agrega validaciones, estados de botones, multimedia, loaders y sincronización con la Tienda y Bundles.

Ubicación en el proyecto: wp-content/themes/blocksy-child/functions.php · la sección cruza varios bloques del archivo. El límite PHP se registra en woocommerce_add_to_cart_validation alrededor de la línea 3391; la comprobación JavaScript del contador Xootix alrededor de la 3510; la sincronización al eliminar usa admin-ajax.php con wp_ajax_obtener_id_desde_slug y obtener_id_desde_slug_callback() alrededor de las líneas 3693-3707; el filtro de miniatura mc_cart_thumbnail_media() se registra alrededor de la 6091; el loader de eliminación se activa desde wp_footer alrededor de la 6145.

Qué significa esta distribución. Xootix no reemplaza el carrito de WooCommerce. Algunas piezas viven en JavaScript porque reaccionan a la interfaz lateral; otras viven en PHP porque deben validar o consultar el carrito real. Los diagramas de esta sección unen ambos lados de la misma operación.



Carrito lateral Xootix abierto sobre la Tienda.

20.1 Qué ocurre al agregar un artwork

Cuando el usuario pulsa Add to Cart, el botón WooCommerce contiene ajax_add_to_cart y data-product_id. El tema realiza primero sus comprobaciones; si la operación está permitida, WooCommerce ejecuta el agregado mediante AJAX. Cuando termina correctamente dispara el evento added_to_cart y Xootix actualiza el panel lateral.

```

Usuario pulsa ADD TO CART
↓
JavaScript comprueba límite
↓
botón entra en loading
↓
WooCommerce ejecuta AJAX
↓
PHP valida el carrito
↓
producto agregado
↓
added_to_cart
↓
Xootix actualiza sidecart

```

20.2 Productos vendidos de a uno

El filtro `woocommerce_is_sold_individually` devuelve `true` para todos los productos. Esto fuerza cantidad 1 y elimina el selector de cantidades: cada artwork o bundle ocupa una línea individual dentro del carrito.

```

add_filter(
    'woocommerce_is_sold_individually',
    'forzar_artworks_individuales',
    10,
    2
);

function forzar_artworks_individuales(...) {
    return true;
}

```

20.3 Límite de diez: primera comprobación en el navegador

Antes del agregado, `verificar_limite_productos()` lee el contador de Xootix. Si ya existen diez elementos, cancela el clic y utiliza la alerta propia del proyecto.

```

const limite = 10;
const cantidadActual = parseInt(
    $('#xoo-wsc-items-count').text().trim() || '0'
);

if (cantidadActual >= limite) {
    e.preventDefault();
    mostrarAlertaPersonalizada(
        'Only up to 10 artworks are allowed per order.'
    );
}

```

20.4 Segunda comprobación en PHP

La validación definitiva ocurre en el servidor mediante `woocommerce_add_to_cart_validation`. Por eso el máximo no depende únicamente del JavaScript del navegador.

```

$cantidad_actual =
    WC()->cart->get_cart_contents_count();

$nueva_cantidad_total =
    $cantidad_actual + $quantity;

if ($nueva_cantidad_total > 10) {
    return false;
}

```

Ejemplo: si existen nueve productos y el usuario intenta agregar otro, $9 + 1 = 10$ y la operación puede continuar. Con diez elementos, una nueva incorporación queda bloqueada.

20.5 Estado loading del botón

Al iniciar un Add to Cart AJAX, el botón recibe las clases disabled y loading, aria-disabled=true y pointer-events:none. Esto evita clics duplicados mientras WooCommerce todavía está procesando la misma acción. added_to_cart revierte esas propiedades al finalizar.

```
$btn
    .addClass('disabled loading')
    .attr('aria-disabled', 'true')
    .css('pointer-events', 'none');
```

20.6 Add to Cart y View Cart

Al renderizar la Tienda, el filtro woocommerce_loop_add_to_cart_link revisa si el producto ya está en WC() -> cart. Si ya existe, oculta Add to Cart y agrega View Cart. Después de un agregado AJAX, otro listener de added_to_cart realiza el mismo cambio inmediatamente sin esperar una recarga.

Producto fuera del carrito
→ ADD TO CART

Producto agregado
→ botón original se oculta
→ aparece VIEW CART

20.7 Eliminación desde Xootix y sincronización con la Tienda

Cuando el usuario elimina una línea desde .xoo-wsc-smr-del, JavaScript obtiene el slug desde el enlace del producto y realiza una petición a admin-ajax.php. La acción obtener_id_desde_slug busca el producto con get_page_by_path() y devuelve su ID. Con ese dato se actualiza únicamente la tarjeta correspondiente de la Tienda: View Cart desaparece y Add to Cart vuelve a estar disponible.

```
$.post(
    admin_url,
    {
        action: 'obtener_id_desde_slug',
        slug: slug
    }
);
```

```

Xootix: REMOVE
  ↓
slug del producto
  ↓
AJAX → ID WooCommerce
  ↓
se localiza la tarjeta
  ↓
VIEW CART se elimina
ADD TO CART reaparece
```

20.8 Multimedia dentro del sidecart

El filtro woocommerce_cart_item_thumbnail sustituye la miniatura estándar. Los artworks pueden mostrar video_destacado; los bundles utilizan un poster propio que cambia con NOD/GDI. De esta forma el panel lateral conserva la identidad audiovisual de la Tienda.

El mismo filtro recibe cada línea del carrito y decide qué contenido devolver. Los bundles usan un poster dependiente de la facción; los artworks con video_destacado reciben un <video>; si ninguna condición aplica, WooCommerce conserva la miniatura original.

```
add_filter(
    'woocommerce_cart_item_thumbnail',
    'mc_cart_thumbnail_media',
    10,
    3
);
```

```

);

function mc_cart_thumbnail_media($thumbnail, $cart_item, $cart_item_key) {
    $product = $cart_item['data'];
    $product_name = (string) $product->get_name();

    if (stripos($product_name, 'bundle') !== false) {
        $faction = $_COOKIE['faction'] ?? 'gdi';
        $uploads_dir = wp_upload_dir();
        $poster_url = $faction === 'gdi'
            ? trailingslashit($uploads_dir['baseurl']) . 'images/poster_bundle.png'
            : trailingslashit($uploads_dir['baseurl']) . 'images/poster_bundle_nod.jpg';

        return '';
    }

    $video_url = get_field('video_destacado', $product->get_id());
    if ($video_url) {
        return '<video class="xoo-wsc-thumb-video" autoplay muted loop playsinline src="'
            . esc_url($video_url) . '"></video>';
    }

    return $thumbnail;
}

```

20.9 Loader al eliminar

El tema escucha los clics sobre los controles de eliminación de Xootix y coloca un estado de carga sobre la fila mientras el plugin procesa el cambio. Los eventos `xoo_wsc_removed_item` y `xoo_wsc_cart_updated` se utilizan para limpiar el loader una vez actualizada la información.

El loader se adjunta a la fila concreta que el usuario está eliminando. No se retira por tiempo fijo como mecanismo principal: los eventos de Xootix confirman que el ítem fue eliminado o que el carrito terminó de actualizarse y entonces se quita el overlay.

```

document.addEventListener('click', function(e) {
    const btn = e.target.closest(
        '.xoo-wsc-smr-del, .xoo-wsc-remove, .xoo-wsc-smr .remove'
    );
    if (!btn) return;

    const productRow = btn.closest(
        '.xoo-wsc-smr, .xoo-wsc-product, .xoo-wsc-cart-item'
    );
    if (!productRow) return;

    const overlay = document.createElement('div');
    overlay.className = 'xoo-wsc-loading-overlay';
    overlay.innerHTML =
        '<div class="xoo-wsc-spinner"></div><div>Loading...</div>';
    productRow.appendChild(overlay);

    function limpiar() {
        if (overlay.parentNode) overlay.remove();
    }

    document.addEventListener('xoo_wsc_removed_item', limpiar, { once: true });
    document.addEventListener('xoo_wsc_cart_updated', limpiar, { once: true });
});

```

20.10 El sidecart no sustituye la página Cart

En las páginas donde se aplica, un `MutationObserver` detecta el botón de Checkout generado dinámicamente por Xootix y lo elimina. El recorrido esperado pasa por la página Cart, porque allí se configuran Comic Balloon Text y Frame Color antes de avanzar al checkout.

Como Xootix puede insertar sus botones después de cargar la página, no alcanza con eliminar Checkout una sola vez. El observer permanece atento al DOM y lo vuelve a retirar cuando el plugin lo genera dinámicamente.

```
document.addEventListener('DOMContentLoaded', () => {
  const observer = new MutationObserver(() => {
    const checkoutBtn = document.querySelector(
      '.xoo-wsc-ft-btn-checkout'
    );

    if (checkoutBtn) checkoutBtn.remove();
  });

  observer.observe(document.body, {
    childList: true,
    subtree: true
  });
});
```

Tienda / Bundles



Xootix Side Cart



VIEW CART



Página Cart



personalización

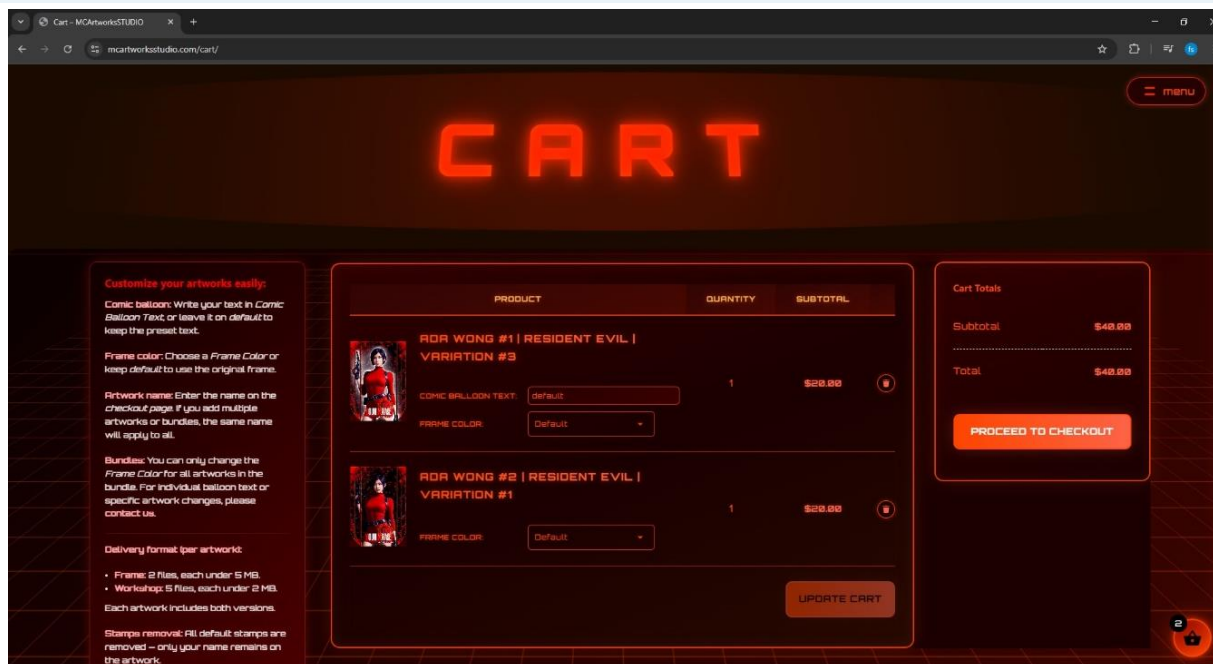


Checkout

21. Página Cart y personalización

La página Cart es la etapa donde WooCommerce muestra las líneas del carrito y MC Artworks Studio agrega información propia a cada una. Además de reemplazar la miniatura, el tema elimina algunos comportamientos estándar, incorpora Comic Balloon Text y Frame Color y guarda esos valores utilizando la clave única de cada item del carrito.

Ubicación en el proyecto: wp-content/themes/blocksy-child/functions.php · mc_cart_thumbnail_media() comienza alrededor de la línea 6099; mostrar_campos_personalizados_en_carrito() se conecta a woocommerce_after_cart_item_name alrededor de la 6248 y comienza cerca de la 6261; mcart_guardar_campos_personalizados_en_carrito() se registra alrededor de la 6383 y comienza cerca de la 6391; la restauración desde sesión se registra alrededor de la 6438 y forzar_valores_por_defecto_en_carrito() alrededor de la 6471.



Página Cart NOD con productos, personalización y resumen del pedido.

21.1 Miniatura audiovisual

`woocommerce_cart_item_thumbnail` se utiliza para reemplazar la miniatura convencional. Cart reutiliza la misma función `mc_cart_thumbnail_media()` documentada en 20.8: los bundles reciben un poster NOD/GDI, los artworks pueden mostrar `video_destacado` y, si no aplica ninguna rama personalizada, se devuelve la miniatura original de WooCommerce.

Cart y Xootix reutilizan el mismo filtro `woocommerce_cart_item_thumbnail`. Por eso la decisión entre poster de bundle, video ACF o miniatura estándar permanece coherente en ambas vistas; la implementación completa se muestra en 20.8.

```
add_filter(
    'woocommerce_cart_item_thumbnail',
    'mc_cart_thumbnail_media',
    10,
    3
);
```

21.2 Campos debajo de cada producto

WooCommerce ejecuta `woocommerce_after_cart_item_name` después del nombre de cada línea. El tema conecta allí `mostrar_campos_personalizados_en_carrito($cart_item, $cart_item_key)`. `cart_item_key` es la clave que distingue una línea concreta del resto del carrito.

```
add_action(
    'woocommerce_after_cart_item_name',
    'mostrar_campos_personalizados_en_carrito',
    10,
    2
);
```

21.3 Comic Balloon Text

El campo sólo se muestra cuando ACF tiene `globo` está activado para ese producto. El input incluye `maxlength=50`, placeholder y un contador visible de caracteres.

```
$tiene_globo = get_field(
    'tiene_globo',
    $product_id
);

if ($tiene_globo) {
    // imprimir input Comic Balloon Text
}
```

El nombre del input incorpora `cart_item_key`, por ejemplo `comic_balloon_text[abc123]`. Así el texto queda asociado a esa línea y no a todo el carrito.

21.4 Frame Color

Cada línea contiene un select `color_frame[cart_item_key]`. Las opciones disponibles son `default`, `blue`, `green`, `pink`, `red`, `violet`, `white` y `yellow`. No representan productos diferentes: son datos de personalización del mismo item.

```
default · blue · green · pink · red · violet · white · yellow
```

21.5 Por qué se usa `cart_item_key`

```
abc123 → Cyberpunk Portrait
      comic_balloon_text[abc123]
      color_frame[abc123]
```

```
def456 → Fantasy Portrait
      comic_balloon_text[def456]
      color_frame[def456]
```

La clave permite que dos productos del mismo carrito mantengan personalizaciones independientes.

21.6 Qué ocurre al actualizar el carrito

El hook `woocommerce_update_cart_action_cart_updated` ejecuta `mcart_guardar_campos_personalizados_en_carrito()`. La función recorre `WC()->cart->get_cart()`, busca los valores enviados para cada `cart_item_key`, los sanitiza y los guarda dentro de `cart_contents`.

```
$texto = sanitize_text_field(
    $_POST['comic_balloon_text'][$cart_item_key]
);

$color = sanitize_text_field(
    $_POST['color_frame'][$cart_item_key]
);
```

Después se ejecuta `WC()->cart->set_session()`, de modo que WooCommerce actualiza su representación persistente del carrito.

21.7 Restauración desde la sesión

Cuando WooCommerce reconstruye el carrito después de una recarga, el filtro `woocommerce_get_cart_item_from_session` restaura `comic_balloon_text` y `color_frame` desde los valores almacenados. Esto permite que la selección continúe visible después de actualizar o volver a abrir la página.

```
if (isset($values['color_frame'])) {
    $cart_item['color_frame'] =
        $values['color_frame'];
}
```

21.8 Valores predeterminados

`woocommerce_before_calculate_totals` ejecuta `forzar_valores_por_defecto_en_carrito()`. Si un item todavía no posee alguno de los campos, se establece default y se vuelve a guardar la sesión. De esta forma el sistema trabaja con un valor definido incluso si el usuario nunca modificó los controles.

```
comic_balloon_text = 'default'
color_frame        = 'default'
```

21.9 Ajustes de la interfaz de Cart

El nombre del producto deja de funcionar como enlace mediante `woocommerce_cart_item_name`. Los cupones se desactivan con `woocommerce_coupons_enabled` y se eliminan varios avisos estándar de WooCommerce relacionados con agregar, eliminar o restaurar productos, manteniendo una interfaz más controlada por el tema.

Estos filtros modifican la interfaz sin alterar el `WC_Product` ni el contenido real del carrito: se conserva el texto del nombre pero se elimina su enlace, los cupones se deshabilitan y algunos avisos estándar dejan de imprimirse.

```
add_filter(
    'woocommerce_cart_item_name',
    'mca_cart_item_name_no_link',
    10,
    3
);

function mca_cart_item_name_no_link($product_name_html) {
    if (false === strpos($product_name_html, '<a')) {
        return $product_name_html;
    }

    $without_link = preg_replace(
        '#<a[^\>]*>(.*?)</a>#is',
        '$1',
        $product_name_html
    );
```

```

);

return '<span class="product-title">' . trim($without_link) . '</span>';
}

add_filter('woocommerce_coupons_enabled', '__return_false');
add_filter('wc_add_to_cart_message_html', '__return_empty_string');
add_filter('woocommerce_cart_undo_item_notice', '__return_empty_string');

```

21.10 Ejemplo completo: desde la Tienda hasta la personalización

```

Cyberpunk Portrait · USD 18
↓
ADD TO CART
↓
WooCommerce AJAX
↓
Xootix muestra el artwork
↓
VIEW CART
↓
Cart detecta tiene_globo = true
↓
Comic Balloon Text = "Welcome back"
Frame Color = Blue
↓
UPDATE CART
↓
PHP sanitiza los valores
↓
cart_item_key conserva la asociación
↓
WooCommerce guarda la sesión
↓
recarga de Cart
↓
los valores siguen seleccionados
↓
Checkout

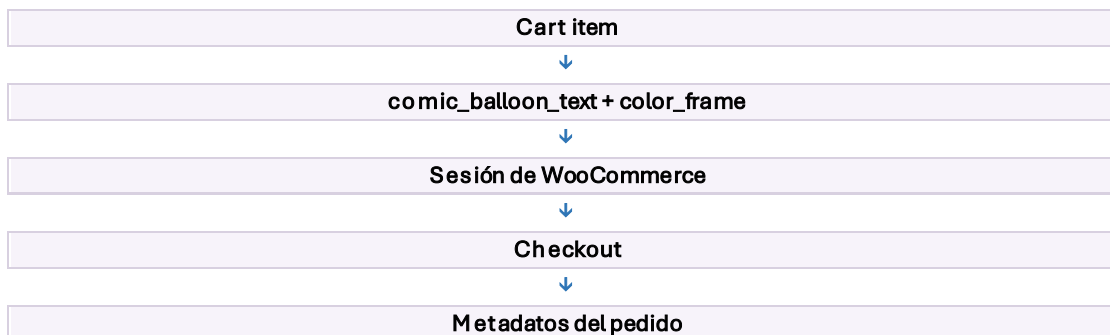
```

22. Sesión y conservación de datos

La siguiente sección explica con más detalle cómo WooCommerce conserva esos datos entre peticiones y cómo pasan posteriormente desde la sesión del carrito hacia el pedido.

Ubicación en el proyecto: wp-content/themes/blocksy-child/functions.php · esta sección profundiza tres callbacks ya usados por Cart: `woocommerce_update_cart_action_cart_updated` → `mcart_guardar_campos_personalizados_en_carrito()`; `woocommerce_get_cart_item_from_session` → `mcart_restaurar_campos_personalizados_desde_sesion()`; y `woocommerce_before_calculate_totals` → `forzar_valores_por_defecto_en_carrito()`. Se encuentran aproximadamente entre las líneas 6381 y 6495.

Una sesión es información temporal que el servidor conserva mientras el visitante continúa la compra. WooCommerce utiliza su propia sesión y la tabla `wp_woocommerce_sessions` para mantener el carrito entre páginas. El tema añade `comic_balloon_text` y `color_frame` a los datos del ítem y los restaura cuando WooCommerce reconstruye el carrito.



Sanitización

Antes de conservar texto recibido del navegador, el código utiliza funciones de limpieza de WordPress. “Sanitizar” significa convertir la entrada a una forma segura y esperada para ese campo, eliminando contenido que no corresponde a un texto normal.

Control de longitud

La interfaz de Comic Balloon Text limita la entrada visible a 50 caracteres. En el servidor existe además una sanitización que conserva como máximo 160 caracteres. En el uso normal del formulario, el límite que encuentra el comprador es el de la interfaz.

22.1 Guardado al pulsar Update Cart

Cuando el usuario actualiza Cart, WooCommerce dispara `woocommerce_update_cart_action_cart_updated`. La función del tema recorre cada línea mediante su `cart_item_key`, limpia los valores recibidos y los guarda dentro de `cart_contents`.

```
add_action(
    'woocommerce_update_cart_action_cart_updated',
    'mcart_guardar_campos_personalizados_en_carrito'
);

$texto = sanitize_text_field($_POST['comic_balloon_text'][$cart_item_key]);
$texto = mb_substr($texto, 0, 160);
WC()->cart->cart_contents[$cart_item_key]['comic_balloon_text'] = $texto;
```

Frame Color sigue el mismo recorrido. Al final se llama a `WC()->cart->set_session()`, de modo que WooCommerce escribe el estado actualizado de su carrito en la sesión.

22.2 Restauración cuando WooCommerce reconstruye el carrito

Al cambiar de página o reconstruir el carrito desde la sesión, `woocommerce_get_cart_item_from_session` vuelve a copiar `comic_balloon_text` y `color_frame` al ítem activo. Así los valores elegidos no dependen de que la página Cart permanezca abierta.

```
add_filter(
    'woocommerce_get_cart_item_from_session',
    'mcart_restaurar_campos_personalizados_desde_sesion',
    10, 3
);

if (isset($values['color_frame'])) {
    $cart_item['color_frame'] = $values['color_frame'];
}
```

22.3 Valores predeterminados antes de calcular totales

Existe una tercera capa de seguridad. Antes de calcular totales, el tema comprueba cada línea y coloca default cuando Comic Balloon Text o Frame Color todavía no tienen valor. Esto permite que el pedido tenga una estructura consistente incluso si el usuario avanza sin pulsar Update Cart.

```
add_action('woocommerce_before_calculate_totals', 'forzar_valores_por_defecto_en_carrito', 5);

if (!isset($cart_item['color_frame']) || $cart_item['color_frame'] === '') {
    $cart->cart_contents[$cart_item_key]['color_frame'] = 'default';
}
```

22.4 Ejemplo completo de persistencia

```

Alex elige Frame Color = Blue
↓
POST color_frame[cart_item_key] = blue
↓
sanitize_text_field()
↓
cart_contents recibe blue
↓
WC()->cart->set_session()
↓
Alex recarga o pasa a Checkout
↓
woocommerce_get_cart_item_from_session
↓
Frame Color vuelve a ser Blue

```

23. Checkout: formulario final de compra y validaciones

Checkout es la etapa final de la compra, donde el visitante completa los datos necesarios y confirma la operación. WooCommerce normalmente puede solicitar dirección, ciudad, país, teléfono y otros campos. MC Artworks Studio reemplaza esa sección por un formulario mucho más corto porque el flujo está pensado para contenido digital.

Ubicación en el proyecto: wp-content/themes/blocksy-child/functions.php · simplificar_checkout_nombre_y_correo() se registra en woocommerce_checkout_fields alrededor de la línea 7202; limpiar_campos_checkout() alrededor de la 7233; el campo Artworks Name se conecta a woocommerce_before_checkout_billing_form alrededor de la 7953 y su función comienza cerca de la 7961; las tres validaciones de servidor se registran en woocommerce_checkout_process alrededor de las líneas 8179, 8188 y 8195; los maxlength del navegador se imprimen desde wp_footer alrededor de la 8259; Artworks Name se copia al pedido en woocommerce_checkout_create_order alrededor de la 8297.

Orden de ejecución importante. Los límites visuales de JavaScript ayudan mientras Alex escribe, pero el submit vuelve a pasar por las validaciones PHP de woocommerce_checkout_process. Sólo si WooCommerce acepta el formulario continúa la construcción del WC_Order y se disparan los hooks que copian los datos al pedido.

Checkout NOD con formulario reducido y resumen del pedido.

Campo	Regla en el código	Para qué se usa
Artworks Name	Obligatorio; máximo 32 caracteres.	Nombre que se asociará a los artworks del pedido.
First Name	Obligatorio; máximo 50 caracteres.	Nombre del comprador utilizado también en el correo.
Email	Obligatorio; máximo 100 caracteres; formato válido y dominio comprobable.	Destino del correo con instrucciones de pago.

Artworks Name

Es un dato propio del proyecto, no un campo estándar de WooCommerce. Si Alex escribe “Raven”, el pedido guarda Artworks Name = Raven y ese valor vuelve a aparecer en administración, en el log y en los correos.

Validación del dominio mediante DNS

DNS permite localizar los servicios asociados a un nombre de dominio en Internet. El código no sólo revisa que el correo tenga una forma válida: también intenta comprobar que el dominio escrito después de @ tenga registros DNS de tipo MX o A.

Ejemplo

alex@gmail.com usa un dominio resoluble. Una dirección con un dominio inventado puede fallar la comprobación. Esto no demuestra que la casilla “alex” exista; verifica el dominio, no el buzón individual.

23.1 Reducción de los campos estándar

Código fuente: wp-content/themes/blocksy-child/functions.php → filtro woocommerce_checkout_fields registrado alrededor de la línea 7202 → simplificar_checkout_nombre_y_correo() desde la 7212. WooCommerce entrega \$fields; el callback devuelve la estructura billing reducida antes de renderizar Checkout.

woocommerce_checkout_fields reemplaza la sección billing por dos campos: First Name y Email. El sitio vende contenido digital, por eso no solicita dirección, ciudad o teléfono dentro de este flujo.

```
$fields['billing'] = [
    'billing_first_name' => [
        'label' => 'First Name', 'required' => true
    ],
    'billing_email' => [
        'label' => 'Email', 'required' => true
    ],
];
```

23.2 Campos limpios cada vez que se abre Checkout

Código fuente: functions.php → filtro woocommerce_checkout_get_value alrededor de la línea 7233 → limpiar_campos_checkout(\$input, \$key) desde la 7244. WooCommerce consulta este filtro cuando decide qué valor inicial mostrar en cada campo.

El filtro woocommerce_checkout_get_value devuelve una cadena vacía para First Name y Email. De esta manera el formulario no reutiliza automáticamente datos anteriores de la sesión o del navegador.

```
if (in_array($key, ['billing_first_name', 'billing_email'])) {
    return '';
}
```

23.3 Artworks Name como dato propio

Código fuente: functions.php → hook woocommerce_before_checkout_billing_form alrededor de la línea 7953 → wpo_mover_titulo_facturacion_y_agregar_artwork_field(\$checkout) desde la 7961. El callback imprime el campo artwork_name antes de los datos de facturación.

Artworks Name no forma parte del checkout estándar de WooCommerce. El tema lo inserta como un campo propio y, más adelante, lo guarda como metadata general del WC_Order. A diferencia de Frame Color, no pertenece a una línea concreta: describe el pedido completo.

El campo se inserta antes del formulario de facturación mediante un hook de WooCommerce.

woocommerce_form_field() genera el input con el nombre artwork_name; más adelante ese mismo nombre se usa para validar \$_POST y guardar la metadata general del pedido.

```
add_action(
    'woocommerce_before_checkout_billing_form',
    'wpo_mover_titulo_facturacion_y_agregar_artwork_field'
);
```

```
function wpo_mover_titulo_facturacion_y_agregar_artwork_field($checkout) {
    echo '<div id="artwork_name_field">';
    echo '<h3>Name for your artworks</h3>';

    woocommerce_form_field('artwork_name', [
        'type' => 'text',
        'class' => ['artwork-name form-row-wide'],
        'label' => 'Enter the name that will appear on your artworks',
        'required' => true
    ], $checkout->get_value('artwork_name'));

    echo '</div>';
}
```

23.4 Resumen de personalizaciones

Código fuente: functions.php → filtro `woocommerce_get_item_data` alrededor de la línea 7676 → `mostrar_campos_personalizados_en_resumen_pedido($item_data, $cart_item)` desde aproximadamente la 7686. Recibe el cart item ya existente y devuelve las filas adicionales que WooCommerce mostrará en el resumen.

`woocommerce_get_item_data` agrega Frame Color al resumen y sólo muestra Comic Balloon Text cuando el producto tiene `tiene_globo` y no se detecta como bundle. De este modo el comprador puede revisar la personalización antes de confirmar.

```
Cart item
├─ Frame Color → siempre se muestra
├─ Comic Balloon Text
│   └─ tiene_globo y no bundle → se muestra
│   └─ caso contrario → se oculta
```

23.5 Order Notes y elementos estándar que se eliminan

Código fuente: functions.php → filtros `woocommerce_checkout_show_terms` (aprox. línea 7606) y `woocommerce_enable_order_notes_field` (aprox. línea 7993). Son filtros independientes: uno oculta términos y el otro desactiva Order Notes.

El campo estándar Order Notes se desactiva y también se ocultan algunos textos de términos y privacidad del checkout. La intención es dejar una pantalla breve, centrada en los datos necesarios para un pedido digital y en el método manual de pago.

```
add_filter('woocommerce_enable_order_notes_field', '__return_false');
add_filter('woocommerce_checkout_show_terms', '__return_false');
```

23.6 Validación del lado del servidor

Código fuente: functions.php → tres callbacks anónimos registrados en `woocommerce_checkout_process` alrededor de las líneas 8179, 8188 y 8195. No tienen nombre propio: WooCommerce los ejecuta durante el submit antes de crear el `WC_Order`; cada `wc_add_notice(..., 'error')` puede detener el avance.

- Artworks Name: obligatorio y máximo de 32 caracteres.

La validación visible en el navegador es sólo una ayuda. Estas comprobaciones se ejecutan nuevamente en PHP durante `woocommerce_checkout_process`; si alguna regla falla, `wc_add_notice()` agrega el error y WooCommerce detiene el avance normal del checkout.

```
add_action('woocommerce_checkout_process', function() {
    if (empty($_POST['artwork_name'])) {
        wc_add_notice('Please enter a name for your artworks.', 'error');
    } elseif (strlen($_POST['artwork_name']) > 32) {
        wc_add_notice(
            'The name for your artworks cannot exceed 32 characters.',
            'error'
        );
    }
});

add_action('woocommerce_checkout_process', function() {
    if (!empty($_POST['billing_first_name'])
```

```

        && strlen($_POST['billing_first_name']) > 50) {
            wc_add_notice(
                'Billing first name cannot exceed 50 characters.',
                'error'
            );
        }
    });

add_action('woocommerce_checkout_process', function() {
    $email = sanitize_text_field($_POST['billing_email'] ?? '');

    if (strlen($email) > 100) {
        wc_add_notice('Email cannot exceed 100 characters.', 'error');
        return;
    }

    if (empty($email) || !filter_var($email, FILTER_VALIDATE_EMAIL)) {
        wc_add_notice('Please enter a valid email address.', 'error');
        return;
    }

    $domain = substr(strrchr($email, '@'), 1);
    if ($domain === false || strpos($domain, '.') === false) {
        wc_add_notice('The email domain is not valid.', 'error');
        return;
    }
});

```

La comprobación del dominio continúa con registros reales de DNS. El código intenta primero MX, luego MX/A mediante `checkdnsrr()` y finalmente usa `gethostbyname()` como último recurso antes de considerar inválido el dominio.

```

$has_dns = false;

if (function_exists('getmxrr')) {
    $mxhosts = [];
    if (@getmxrr($domain, $mxhosts) && !empty($mxhosts)) {
        $has_dns = true;
    }
}

if (!$has_dns && function_exists('checkdnsrr')) {
    $has_dns = @checkdnsrr($domain, 'MX')
        || @checkdnsrr($domain, 'A');
}

if (!$has_dns) {
    $resolved = @gethostbyname($domain);
    $has_dns = ($resolved !== $domain && $resolved !== false);
}

if (!$has_dns) {
    wc_add_notice(
        'The email domain does not appear to exist (check that it is correctly typed).',
        'error'
    );
}

```

- First Name: máximo de 50 caracteres.
- Email: máximo de 100 caracteres y formato válido.
- Dominio del email: debe contener un punto y resolver mediante registros DNS MX o A, con un último intento de resolución por host.

Estas validaciones se ejecutan con `woocommerce_checkout_process`. Si una condición falla, `wc_add_notice()` agrega un error y WooCommerce no continúa con la creación normal del pedido.

23.7 Validación visible en el navegador

Código fuente: functions.php → JavaScript impreso desde wp_footer alrededor de la línea 8259 → callback DOMContentLoaded que aplica maxlength a artwork_name, billing_first_name y billing_email. Es una ayuda del navegador; la autoridad final sigue siendo la validación PHP de 23.6.

JavaScript agrega maxlength a los tres campos y billing_email se define como type="email". Esto mejora la experiencia del usuario, pero no reemplaza las validaciones del servidor: un usuario puede modificar HTML desde el navegador y por eso PHP vuelve a comprobar los límites.

Estos límites ayudan al usuario mientras escribe, pero no sustituyen la comprobación de PHP de la subsección anterior.

```
document.addEventListener('DOMContentLoaded', function() {
  const art = document.querySelector('#artwork_name');
  if (art) art.setAttribute('maxlength', '32');

  const name = document.querySelector('#billing_first_name');
  if (name) name.setAttribute('maxlength', '50');

  const mail = document.querySelector('#billing_email');
  if (mail) mail.setAttribute('maxlength', '100');
});
```

23.8 Qué ocurre al confirmar

Alex pulsa Pay with PayPal. El navegador envía el formulario; WooCommerce ejecuta las validaciones registradas en woocommerce_checkout_process. Si no hay wc_add_notice() de tipo error, WooCommerce continúa y construye el pedido. En esa construcción se copian Artworks Name y las personalizaciones de cada línea; recién después el gateway seleccionado recibe el order_id.

```
Pay with PayPal
↓
submit del checkout
↓
WooCommerce valida campos
↓
crea WC_Order
↓
manual_pay recibe el order_id
```

24. Método de pago manual mediante PayPal

WooCommerce utiliza el término gateway, o pasarela de pago, para el componente que decide qué ocurre cuando el comprador confirma una forma de pago. El tema registra una pasarela propia con el identificador manual_pay y el título “Pay with PayPal (manual)”.

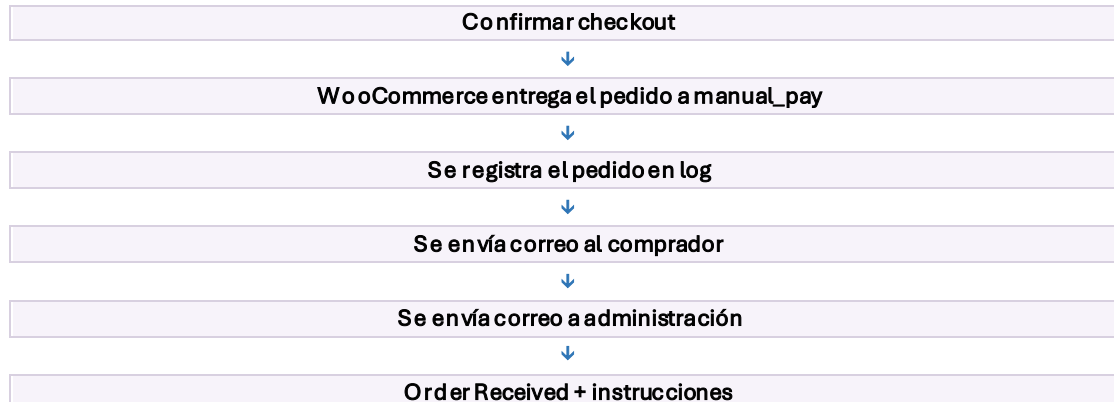
Ubicación en el proyecto: wp-content/themes/blocksy-child/functions.php · add_action('init', 'registrar_metodo_pago_manual') aparece en la línea aproximada 7262; registrar_metodo_pago_manual() comienza cerca de la 7271 y declara la clase WC_Gateway_Manual_Pay. Dentro de esa clase, __construct() configura el gateway, solo_en_checkout() controla disponibilidad, get_log_path()/log_manual_pay() gestionan logs y process_payment(\$order_id) comienza alrededor de la línea 7322. El filtro woocommerce_payment_gateways que incorpora la clase aparece al final del mismo bloque, cerca de la 7593.

Esta distinción evita una confusión frecuente: init no procesa un pago. init hace que WordPress ejecute registrar_metodo_pago_manual(), y esa función declara la clase del gateway si WooCommerce está disponible. El pago ocurre mucho después, cuando WooCommerce selecciona una instancia de esa clase durante Checkout.

Aclaración fundamental

Esta implementación no se conecta a la API de PayPal y no cobra automáticamente. Crea el pedido, registra la operación, envía instrucciones por correo y lleva al comprador a la página de pedido recibido. El pago se completa de forma manual usando los datos indicados en el email.

```
$this->id = 'manual_pay';
$this->title = 'Pay with PayPal (manual)';
...
return ['result' => 'success', 'redirect' => $this->get_return_url($order)];
```



24.1 Registro de la pasarela

Código fuente: functions.php → hook init → registrar_metodo_pago_manual() → declaración de WC_Gateway_Manual_Pay.

La pasarela se registra durante init. El código comprueba primero que WC_Payment_Gateway exista; si WooCommerce no está disponible, no intenta declarar la clase propia.

```
add_action('init', 'registrar_metodo_pago_manual');

if (!class_exists('WC_Payment_Gateway')) return;

class WC_Gateway_Manual_Pay extends WC_Payment_Gateway {
    // ...
}
```

24.2 Configuración de WC_Gateway_Manual_Pay

Código fuente: functions.php → WC_Gateway_Manual_Pay::__construct(). Aquí \$this representa la instancia concreta del gateway que WooCommerce está configurando.

El constructor define la identidad de la pasarela. manual_pay es el identificador interno y “Pay with PayPal (manual)” es el título que representa el método dentro del flujo de compra.

```
$this->id = 'manual_pay';
$this->title = 'Pay with PayPal (manual)';
$this->enabled = 'yes';
$this->supports = ['products'];
```

24.3 Disponibilidad sólo durante Checkout

Código fuente: functions.php → WC_Gateway_Manual_Pay::solo_en_checkout(). El constructor registra este método en woocommerce_available_payment_gateways.

solo_en_checkout() elimina manual_pay de la lista de gateways cuando la petición no corresponde al checkout ni a una actualización AJAX del checkout. Esto evita presentar el método en contextos donde no puede usarse.

La excepción para DOING AJAX es necesaria porque WooCommerce actualiza partes del checkout mediante peticiones AJAX. En esas peticiones el gateway debe seguir disponible aunque is_checkout() no represente una carga convencional de página.

```
public function solo_en_checkout($gateways) {
    if (is_admin()) return $gateways;

    if (!is_checkout()
        && !(defined('DOING AJAX') && DOING AJAX)) {
```

```

        unset($gateways[$this->id]);
    }

    return $gateways;
}

```

24.4 Cómo se agrega a WooCommerce

Código fuente: functions.php → registrar_metodo_pago_manual() → filtro woocommerce_payment_gateways, situado después de la declaración de la clase.

```

add_filter('woocommerce_payment_gateways', function($methods) {
    $methods[] = 'WC_Gateway_Manual_Pay';
    return $methods;
});

```

WooCommerce recibe el nombre de la clase y la incorpora a su lista de métodos de pago disponibles.

24.5 Botón Pay with PayPal

Código fuente: functions.php → hook woocommerce_review_order_before_submit alrededor de la línea 7628 → boton_manual_estilo_paypal() alrededor de la línea 7639. El botón hace submit del Checkout y el input hidden fija payment_method=manual_pay.

Antes del botón estándar de envío, el tema inserta un campo oculto que fuerza payment_method = manual_pay y un botón visible. Ese botón envía el formulario de checkout; no abre PayPal ni realiza un cobro externo.

```

<input type="hidden" name="payment_method" value="manual_pay">
<button type="submit" class="manual-pay-btn">Pay with PayPal</button>

```

24.6 Qué hace process_payment()

Código fuente: functions.php → clase WC_Gateway_Manual_Pay → process_payment(\$order_id), aproximadamente líneas 7322-7590. WooCommerce llama este método cuando manual_pay es el gateway elegido y el WC_Order ya existe.

```

process_payment(order_id)
↓
wc_get_order(order_id)
↓
construye y guarda el log
↓
construye email del comprador
↓
wp_mail(comprador)
↓
construye email de administración
↓
wp_mail(admin)
↓
get_return_url(order)
↓
Order Received

```

24.6.1 Ejemplo guiado: cómo Alex llega al gateway

Alex completa Checkout y pulsa Pay with PayPal. El input oculto deja payment_method=manual_pay. WooCommerce valida el formulario, construye el WC_Order y conserva su ID. Con el gateway ya seleccionado, WooCommerce invoca WC_Gateway_Manual_Pay::process_payment(\$order_id).

```

Alex pulsa Pay with PayPal
↓
submit del Checkout
↓
woocommerce_checkout_process valida
↓
WooCommerce construye WC_Order
↓
payment_method = manual_pay

```

```
↓
WooCommerce invoca WC_Gateway_Manual_Pay::process_payment(order_id)
↓
wc_get_order(order_id) devuelve el objeto WC_Order
```

24.7 Qué NO hace esta pasarela

Aclaración

manual_pay no utiliza la API de PayPal, no consulta el saldo, no confirma el pago y no ejecuta payment_complete(). El pedido queda pendiente hasta que el pago se comprueba de manera manual. El nombre “Pay with PayPal” describe las instrucciones que recibirá el comprador, no una integración automática con PayPal.

25. Creación del pedido

Esta sección explica el momento exacto en que los datos temporales del carrito y del formulario de Checkout pasan a formar parte del WC_Order. El punto importante es temporal: estos hooks se ejecutan durante la construcción del pedido. WC_Gateway_Manual_Pay::process_payment(\$order_id) interviene después, cuando WooCommerce ya dispone de un pedido identificable por su ID.

Ubicación en el proyecto: wp-content/themes/blocksby-child/functions.php. El callback anónimo de woocommerce_checkout_create_order_line_item está en las líneas 7651-7659 y copia Comic Balloon Text y Frame Color a cada WC_Order_Item. El callback anónimo de woocommerce_checkout_create_order está en las líneas 8297-8301 y guarda Artworks Name como metadata del WC_Order. Ambos pertenecen al proceso de creación del pedido; no son métodos de WC_Gateway_Manual_Pay.

No confundir registro con ejecución: add_action() deja el callback asociado a un hook. El código interno no se ejecuta cuando WordPress carga functions.php; se ejecuta más adelante, cuando WooCommerce alcanza ese hook durante Checkout.

25.1 Qué existe justo antes de crear el pedido

Antes de estos hooks, WooCommerce conserva en el carrito cada producto y los datos personalizados agregados por MC Artworks Studio. Para una línea concreta, \$values representa la información del cart item que WooCommerce está transformando en una línea permanente del pedido. En paralelo, Artworks Name llega desde el formulario final de Checkout mediante \$_POST['artwork_name'].

```
CARRITO / CHECKOUT
↓
cart item: product_id + color_frame + comic_balloon_text
formulario: artwork_name
↓
WooCommerce comienza a construir WC_Order y sus WC_Order_Item
```

25.2 Personalizaciones de cada línea: callback completo

Código fuente real: functions.php → callback anónimo registrado en woocommerce_checkout_create_order_line_item → líneas 7651-7659. WooCommerce llama este callback una vez por cada línea que está convirtiendo del carrito al pedido.

```
add_action('woocommerce_checkout_create_order_line_item', function($item, $cart_item_key, $values, $order) {
    if (isset($values['comic_balloon_text']) && !empty($values['comic_balloon_text'])) {
        $item->add_meta_data('Comic Balloon Text', $values['comic_balloon_text']);
    }

    if (isset($values['color_frame']) && !empty($values['color_frame'])) {
        $item->add_meta_data('Frame Color', ucfirst($values['color_frame']));
    }
}, 10, 4);
```

El callback comprueba primero si cada dato existe y no está vacío. Si Comic Balloon Text está disponible, lo agrega al WC_Order_Item con la etiqueta visible “Comic Balloon Text”. Si color_frame está disponible, lo agrega con la etiqueta “Frame Color” y aplica ucfirst() para que el valor comience con mayúscula.

El cambio conceptual es importante: color_frame y comic_balloon_text dejan de ser sólo datos temporales del cart item y pasan a ser metadatos de una línea concreta del pedido. De esta forma, dos productos del mismo WC_Order pueden conservar personalizaciones diferentes.

25.3 Qué significan los cuatro parámetros del callback

Parámetro	Qué representa en este hook
\$item	El WC_Order_Item que WooCommerce está construyendo para esa línea. Aquí se agregan los metadatos permanentes.
\$cart_item_key	La clave interna con la que WooCommerce identifica esa línea dentro del carrito. El callback la recibe, aunque este código no la utiliza.
\$values	Los datos del cart item de origen. MC Artworks lee aquí comic_balloon_text y color_frame.
\$order	El WC_Order que se está construyendo. WooCommerce lo pasa al callback, aunque este fragmento no lo utiliza directamente.

El “10,4” del final de add_action() significa prioridad 10 y cuatro argumentos aceptados. Por eso WordPress/WooCommerce entrega \$item, \$cart_item_key, \$values y \$order al callback anónimo.

25.4 Artworks Name: dato general del pedido

Código fuente real: functions.php → callback anónimo registrado en woocommerce_checkout_create_order → líneas 8297-8301. A diferencia de Frame Color y Comic Balloon Text, Artworks Name pertenece al pedido completo y no a una línea determinada.

```
add_action('woocommerce_checkout_create_order', function($order, $data) {
    if ( isset($_POST['artwork_name']) && $_POST['artwork_name'] !== '' ) {
        $order->update_meta_data('Artworks Name', sanitize_text_field($_POST['artwork_name'])); // Traducido
    }
}, 10, 2);
```

El callback recibe el WC_Order en construcción como \$order. Si artwork_name llegó desde Checkout y no está vacío, sanitize_text_field() limpia el texto recibido y update_meta_data() lo incorpora al pedido con la clave “Artworks Name”. El segundo parámetro del callback, \$data, es entregado por WooCommerce pero no se utiliza en este fragmento.

La diferencia de nivel explica por qué el proyecto utiliza dos APIs distintas: \$item->add_meta_data() para personalizaciones de un producto y \$order->update_meta_data() para un dato que describe el pedido completo.

25.5 Qué queda dentro del WC_Order antes del gateway

```
WC_Order #1530
├─ First Name: Alex
├─ Email: alex@example.com
├─ Artworks Name: Raven
├─ WC_Order_Item: Cyberpunk Portrait
│   └─ Frame Color: Blue
│       └─ Comic Balloon Text: Welcome back
```

En el ejemplo pedagógico, Blue y “Welcome back” pertenecen únicamente a Cyberpunk Portrait. Raven, en cambio, pertenece al pedido completo. Cuando process_payment() reciba el identificador del pedido, esos datos ya estarán disponibles mediante las APIs de WC_Order y WC_Order_Item; el gateway no necesita reconstruirlos desde Cart.

25.6 Ejemplo guiado: Alex antes y después

Antes de crear el pedido, la línea de Alex todavía vive en el carrito y contiene color_frame = blue y comic_balloon_text = Welcome back. Durante la construcción, WooCommerce crea el WC_Order_Item y dispara woocommerce_checkout_create_order_line_item. El callback copia esos valores a las claves visibles “Frame Color” y “Comic Balloon Text”. En el mismo proceso de Checkout, woocommerce_checkout_create_order permite guardar Raven como “Artworks Name” sobre el WC_Order.

```
ANTES
cart item → color_frame = blue · comic_balloon_text = Welcome back
Checkout → artwork name = Raven
↓
DURANTE LA CREACIÓN
WC_Order_Item recibe Frame Color / Comic Balloon Text
WC_Order recibe Artworks Name
↓
```

DESPUÉS

WC_Order #1530 ya contiene los datos que leerá manual_pay

25.7 Entrega del pedido al gateway

Código fuente real que recibe el resultado de esa creación: functions.php → WC_Gateway_Manual_Pay::process_payment(\$order_id) → líneas 7322-7323.

```
public function process_payment($order_id) {
    $order = wc_get_order($order_id);
```

WooCommerce entrega el ID del pedido al gateway seleccionado. `wc_get_order($order_id)` transforma ese identificador en un objeto `WC_Order`. A partir de ese punto, el código trabaja con el pedido ya creado: primero genera el log y después construye los dos correos.

25.8 Estado pendiente y HPOS

`manual_pay` no ejecuta `payment_complete()` dentro de `process_payment()`. Por eso el código del tema no marca el pedido como pagado. En la instalación documentada, los pedidos manuales permanecen en estado `wc-pending` mientras esperan la comprobación manual del pago. Con HPOS activo, el tema sigue utilizando `WC_Order` y `WC_Order_Item` y no necesita escribir SQL directamente sobre las tablas físicas.

26. Logs: registro de pedidos y correos

Un log es un registro cronológico destinado a reconstruir qué ocurrió durante una operación. En MC Artworks Studio, `process_payment()` genera primero un bloque con los datos del pedido y después agrega al mismo archivo el resultado del correo del comprador y del correo de administración. Esta sección separa tres pasos que antes quedaban mezclados: construir el texto, decidir en qué archivo escribirlo y escribirlo.

Ubicación en el proyecto: `wp-content/themes/blocksy-child/functions.php` → clase `WC_Gateway_Manual_Pay`. `process_payment()` empieza en la línea 7322; el bloque que construye `$log` ocupa aproximadamente 7329-7377. `get_log_path()` está en 7298-7308 y `log_manual_pay($message)` en 7310-7320.

26.1 Entrada a process_payment(): de order_id a WC_Order

```
public function process_payment($order_id) {
    $order = wc_get_order($order_id);
```

`$order_id` es el identificador que WooCommerce acaba de entregar al gateway. `wc_get_order()` obtiene el objeto `WC_Order` correspondiente y lo guarda en `$order`. Desde ese objeto se leen el número visible del pedido, los datos de facturación, Artworks Name, las líneas de producto y el total.

```
$order_id = 1530
↓ wc_get_order(1530)
$order = WC_Order #1530
↓
get_order_number() · get_billing_first_name() · get_items() · get_total()
```

26.2 Construcción del encabezado y los datos generales del log

Código fuente real: functions.php → WC_Gateway_Manual_Pay::process_payment() → líneas 7329-7344.

```
$log = "===== \n";
$log .= "🕒 " . date('d-m-Y H:i:s', current_time('timestamp')) . " \n";
$log .= "===== \n";
$log .= "🛒 NEW MANUAL PAYPAL ORDER \n";
$log .= "----- \n";
$log .= "Order #: " . $order->get_order_number() . " \n";
$log .= "Customer: " . $order->get_billing_first_name() . " " . $order->get_billing_last_name() . " \n";
$log .= "Email: " . $order->get_billing_email() . " \n";

// Agregamos Artworks Name si existe
$artwork_name = $order->get_meta('Artworks Name');
if ($artwork_name) {
    $log .= "Name for the artworks: " . $artwork_name . " \n";
}
```

```
$log .= "-----\n";
```

La variable \$log comienza como una cadena de texto. Cada operador .= agrega contenido al final sin borrar lo anterior. La fecha se formatea con la hora que WordPress entrega mediante current_time('timestamp'). Después se incorporan el número del pedido, el nombre de facturación, el email y, si existe, Artworks Name.

En el caso de Alex, \$order->get_meta('Artworks Name') devuelve Raven. Como el valor no está vacío, la condición if (\$artwork_name) agrega "Name for the artworks: Raven" al registro.

26.3 Recorrido de productos y personalizaciones

Código fuente real: functions.php → process_payment() → primer foreach (\$order->get_items()) → líneas 7346-7372.

```
foreach ($order->get_items() as $item) {
    $product_name = $item->get_name();
    $total = number_format($item->get_total(), 2);
    $log .= "• {$product_name} - \${$total}\n";

    $product_id = $item->get_product_id();
    $tiene_globo = get_field('tiene_globo', $product_id); // solo muestra Comic Balloon
    Text si tiene_globo

    $meta_lines = [];
    foreach ($item->get_meta_data() as $meta) {
        $key = strtolower($meta->key);
        $value = $meta->value;

        if ($key === 'frame color') {
            $meta_lines[] = "• Frame Color: {$value}";
        }
        if ($tiene_globo && $key === 'comic balloon text') {
            $meta_lines[] = "• Comic Balloon Text: {$value}";
        }
    }

    if (!empty($meta_lines)) {
        foreach ($meta_lines as $line) {
            $log .= "    {$line}\n";
        }
    }
}
```

\$order->get_items() devuelve las líneas del WC_Order. En cada vuelta, \$item representa un WC_Order_Item. get_name() obtiene el nombre del producto y get_total() el importe de esa línea. Luego get_product_id() permite consultar ACF con get_field('tiene_globo', \$product_id).

El bucle interno recorre los metadatos ya persistidos en la sección 25. Frame Color siempre puede agregarse al log. Comic Balloon Text sólo se agrega cuando tiene_globo está activo para ese producto. \$meta_lines funciona como una lista temporal: primero reúne las líneas que corresponde mostrar y después las incorpora a \$log con sangría.

```
WC_Order_Item: Cyberpunk Portrait
↓ get_name() / get_total()
Cyberpunk Portrait - $18.00
↓ get_meta_data()
Frame Color: Blue
Comic Balloon Text: Welcome back
↓
se agregan esas líneas a $log
```

26.4 Total y primera escritura del registro

Código fuente real: functions.php → final del bloque de log de process_payment() → líneas 7374-7377.

```
$log .= "-----\n";
$log .= "Total: $" . number_format($order->get_total(), 2) . "\n";

$this->log_manual_pay($log);
```

Después de recorrer todos los productos, `get_total()` obtiene el total del `WC_Order`. En ese momento `$log` ya contiene el registro completo del pedido y `$this->log_manual_pay($log)` entrega esa cadena al método encargado de escribirla.

`$this` representa la instancia concreta de `WC_Gateway_Manual_Pay` que está procesando el pedido. `log_manual_pay()` no es una función global: es un método `private` de esa misma clase.

26.5 Cómo se calcula el archivo del día

Código fuente `real: functions.php` → `WC_Gateway_Manual_Pay::get_log_path()` → líneas 7298-7308.

```
private function get_log_path() {
    // Directorio existente donde se guardan los logs
    $uploads_dir = WP_CONTENT_DIR . '/uploads/logs';

    // Nombre del archivo con formato argentino: día-mes-año_orders.log
    $date_str = date('d-m-Y');
    $log_filename = "{$date_str}_orders.log";

    // Retorna el path completo del archivo del día
    return $uploads_dir . '/' . $log_filename;
}
```

`get_log_path()` no escribe contenido. Sólo construye y devuelve la ruta del archivo del día. Para el 2 de septiembre de 2026, el nombre resultante es `02-09-2026_orders.log` dentro de `wp-content/uploads/logs`.

El método presupone que `wp-content/uploads/logs` ya existe y tiene permisos de escritura. No contiene `mkdir()` ni `wp_mkdir_p()`; por eso la existencia y los permisos de esa carpeta forman parte de la instalación del proyecto.

26.6 Cómo escribe `log_manual_pay()`

Código fuente `real: functions.php` → `WC_Gateway_Manual_Pay::log_manual_pay($message)` → líneas 7310-7320.

```
private function log_manual_pay($message) {
    $log_file = $this->get_log_path();

    // Bloque visual para separar cada registro
    $entry = trim($message) . "\n";

    $written = @file_put_contents($log_file, $entry, FILE_APPEND | LOCK_EX);
    if ($written === false) {
        error_log("manual-pay log write FAILED: $entry");
    }
}
```

El método recibe el texto ya construido. `get_log_path()` determina el archivo; `trim($message)` elimina espacios o saltos sobrantes en los extremos y añade un salto final. `file_put_contents()` escribe con `FILE_APPEND` para agregar el nuevo evento al final del archivo existente y con `LOCK_EX` para solicitar un bloqueo exclusivo durante la escritura.

Si `file_put_contents()` devuelve `false`, el código utiliza `error_log()` como salida alternativa. Ese fallback evita que el fallo desaparezca por completo, aunque no crea una segunda copia del archivo de pedidos.

26.7 Ejemplo completo de Alex: del `WC_Order` al archivo

```
WC_Order #1530
↓
fecha/hora de WordPress
↓
Order #: 1530
Customer: Alex
Email: alex@example.com
Name for the artworks: Raven
↓
Cyberpunk Portrait - $18.00
Frame Color: Blue
Comic Balloon Text: Welcome back
↓
Total: $18.00
↓
$this->log_manual_pay($log)
```

↓
02-09-2026_orders.log

El archivo no se genera a partir de una captura de Checkout. Se reconstruye desde el WC_Order que ya contiene la información persistida. Esa decisión permite que el registro refleje el pedido real que WooCommerce acaba de crear.

26.8 Qué más se agrega al mismo log

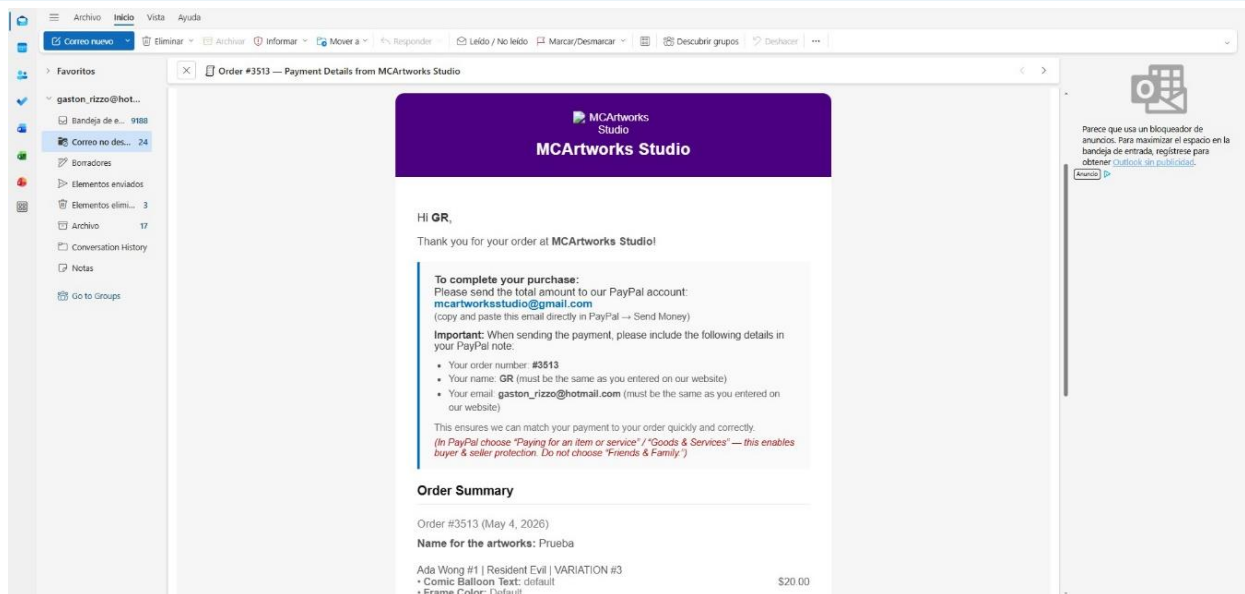
El registro del pedido es sólo el primer evento. Más adelante, el mismo process_payment() vuelve a llamar log_manual_pay() después del wp_mail() del comprador y después del wp_mail() administrativo. Por último agrega una línea separadora. Por eso un mismo archivo diario permite seguir la secuencia completa de cada compra manual.

1. detalle del WC_Order
- ↓
2. resultado del correo del comprador
- ↓
3. resultado del correo de administración
- ↓
4. separador final del pedido

27. Correo al comprador

Después de escribir el detalle del pedido en el log, la misma ejecución de process_payment() construye un email HTML para el comprador. No existe una función independiente que “mande el mail del cliente”: el contenido se arma dentro del propio método y se entrega a WordPress mediante wp_mail().

Ubicación en el proyecto: wp-content/themes/blocksy-child/functions.php → WC_Gateway_Manual_Pay::process_payment(\$order_id) → aproximadamente líneas 7383-7514. \$order ya es un WC_Order y \$artwork_name ya fue recuperado durante la construcción del log.



Ejemplo real de correo recibido por el comprador. La captura ilustra la salida visible; las subsecciones siguientes explican el código que la construye.

27.1 Estado disponible al comenzar el correo

Al llegar a este punto no se vuelve a leer Cart. \$order contiene el pedido ya creado; sus WC_Order_Item conservan Frame Color y Comic Balloon Text; \$artwork_name contiene el dato general Artworks Name. El bloque sólo prepara representaciones HTML de esos datos.

Código fuente real: functions.php → inicio del bloque de correo del comprador → líneas 7383-7385.

```
// === Construir resumen HTML ===
$order_date = $order->get_date_created() ? $order->get_date_created()->date('F j, Y') : '';
$items_html = '';
```

\$order_date se obtiene de la fecha de creación del pedido y se formatea como “F j, Y”. \$items_html comienza vacío porque se irá completando con una fila HTML por cada producto.

27.2 Construcción de \$items_html producto por producto

Código fuente real: functions.php → process_payment() → líneas 7387-7416.

```
// Se arma el listado de productos
foreach ($order->get_items() as $item) {
    $product_name = esc_html($item->get_name());
    $total = number_format($item->get_total(), 2);
    $meta_html = '';

    $product_id = $item->get_product_id();
    $tiene_globo = get_field('tiene_globo', $product_id); // true/false

    foreach ($item->get_meta_data() as $meta) {
        $key = esc_html($meta->key);
        $value = esc_html($meta->value);

        // Solo mostramos "Comic Balloon Text" si tiene_globo es true
        if ($tiene_globo && strtolower($key) === 'comic balloon text') {
            $meta_html .= "<br><span style='color:#777;'>•";
        }

        <strong>{$key}</strong> {$value}</span>";
    }

    // Frame Color siempre se muestra
    if (strtolower($key) === 'frame color') {
        $meta_html .= "<br><span style='color:#777;'>•";
    }

    <strong>{$key}</strong> {$value}</span>";
}

$items_html .= "
<tr>
    <td style='padding:6px 0;border-bottom:1px solid
#eee;'>{$product_name}{$meta_html}</td>
    <td style='padding:6px 0;border-bottom:1px solid #eee;text-
align:right;'>\${$total}</td>
</tr>";
}
```

Este foreach recorre nuevamente el WC_Order, pero ahora el objetivo no es el archivo de log sino una tabla HTML para los correos. esc_html() se aplica a nombre, claves y valores antes de incrustarlos en HTML. \$meta_html reúne las personalizaciones que se mostrarán debajo del nombre del producto.

La condición ACF se mantiene: Frame Color se incorpora siempre que aparezca entre los metadatos; Comic Balloon Text se incorpora sólo si tiene_globo está activo. Finalmente \$items_html .= agrega una fila <tr> con el producto y el precio. Si el pedido tiene varios productos, la cadena termina conteniendo varias filas consecutivas.

```
WC_Order #1530
↓ get_items()
Cyberpunk Portrait
↓
$meta_html = Frame Color: Blue + Comic Balloon Text: Welcome back
↓
$items_html += fila HTML del producto
```

27.3 Artworks Name y URL del logo

Código fuente real: functions.php → process_payment() → líneas 7418-7426.

```
$artwork_html = '';
if ($artwork_name) {
    $artwork_html = '<p style="color:#555;margin:5px 0 15px 0;"><strong>Name for the
artworks:</strong> ' . esc_html($artwork_name) . '</p>';
}

// Path al logo
$uploads_dir = wp_upload_dir();
$logo_url = trailingslashit($uploads_dir['baseurl']) . 'images/logo.png';
```

```
// $logo_url = home_url('/images/logo.png');
```

\$artwork_html comienza vacío y sólo se completa si el pedido tiene Artworks Name. Raven se escapa con esc_html() antes de insertarse en el párrafo HTML. Para el logo, wp_upload_dir() devuelve información del directorio de uploads y trailingslashit() garantiza la barra final antes de concatenar images/logo.png.

Aquí PHP sólo construye una URL y una cadena HTML. La imagen no se incrusta físicamente dentro de functions.php; el email utiliza la URL resultante en el atributo src del .

27.4 Plantilla del comprador: cabecera y saludo

Código fuente real: functions.php → variable \$message → líneas 7428-7443. Este es el comienzo de una única cadena HTML que continúa en las subsecciones siguientes.

```
// === Plantilla HTML elegante ===
$message = '
<html>
<body style="background:#f6f6f6;padding:40px 0;margin:0;font-family:Helvetica,Arial,sans-serif;">
<table align="center" width="600" cellpadding="0" cellspacing="0"
style="background:#ffffff;border-radius:12px;overflow:hidden;">
<td style="background:#4B0082;color:#ffffff;padding:25px 30px;text-align:center;">

<div style="font-size:22px;font-weight:bold;letter-spacing:0.5px;">MCArtworks
Studio</div>
</td>
<tr>
<td style="padding:30px;">
<p style="font-size:16px;color:#333;">Hi <strong>' . esc_html($order-
>get_billing_first_name()) . '</strong></p>
<p style="font-size:15px;color:#555;">Thank you for your order at <strong>MCArtworks
Studio</strong>!</p>
```

La cadena abre <html> y <body>, crea una tabla central de 600 px y utiliza el violeta #4B0082 para la cabecera. esc_url(\$logo_url) coloca la URL del logo. El saludo obtiene el First Name directamente desde \$order y lo protege con esc_html().

27.5 Plantilla del comprador: instrucciones de pago PayPal

Código fuente real: continuación del mismo \$message → líneas 7445-7466. No es una segunda plantilla: estas líneas siguen concatenadas dentro de la cadena iniciada en 7429.

```
<div style="background:#fafafa;border-left:4px solid #0070ba;padding:15px
20px;margin:20px 0;">
<p style="margin:0;color:#333;font-size:15px;">
<strong>To complete your purchase:</strong><br>
Please send the total amount to our PayPal account:<br>
<span style="color:#0070ba;font-
weight:bold;">mcartworksstudio@gmail.com</span>
<br><span style="font-
size:13px;color:#555;">(copy and paste this email directly in PayPal → Send Money)</span>
</p>
<p style="margin:10px 0 0;color:#333;font-size:14px;">
<strong>Important:</strong> When sending the
payment, please include the following details in your PayPal note:
</p>
<ul style="margin:8px 0 0
20px;padding:0;color:#555;font-size:13px;line-height:1.5;">
<li>Your order number: <strong>#</strong>' . $order-
>get_order_number() . '</strong></li>
<li>Your name: <strong>' . esc_html($order-
>get_billing_first_name()) . ' . esc_html($order->get_billing_last_name()) . '</strong> (must be the same
as you entered on our website)</li>
<li>Your email: <strong>' . esc_html($order-
>get_billing_email()) . '</strong> (must be the same as you entered on our website)</li>
</ul>
```

```

                <p style="margin:10px 0 0;color:#777;font-size:13px;">
                    This ensures we can match your payment to your
order quickly and correctly.

                </p>
                <p style="margin:6px 0 0;color:#a00;font-size:13px;font-
style:italic;">
                    (In PayPal choose "Paying for an item or service" /
"Goods & Services" – this enables buyer & seller protection. Do not choose "Friends & Family.")
                </p>
            </div>

```

Este bloque contiene las instrucciones que convierten al gateway en un pago manual. El email indica la cuenta PayPal, pide incluir número de pedido, nombre y email en la nota y advierte que debe utilizarse Goods & Services. El número, nombre y correo se obtienen del WC_Order, por lo que el comprador recibe datos vinculados al pedido que acaba de generar.

27.6 Plantilla del comprador: resumen, total, entrega y cierre

Código fuente real: continuación y cierre de \$message → líneas 7468-7499.

```

                <h3 style="border-bottom:2px solid #eee;padding-bottom:8px;color:#111;">Order
Summary</h3>
                <p style="color:#777;margin:0 0 10px;">Order #' . $order-
>get_order_number() . ' ( ' . $order_date . ' )</p>
                ' . $artwork_html . '
                <table width="100%" cellpadding="0" cellspacing="0" style="font -
size:14px;color:#555;border-collapse:collapse;">
                ' . $items_html . '
                <tr>
                <td style="padding-top:10px;font-weight:bold;">Total to Pay:</td>
                <td align="right" style="padding-top:10px;font-weight:bold;color:#111;">$' .
number_format($order->get_total(), 2) . '</td>
                </tr>
                </table>

                <p style="margin:25px 0 5px;color:#777;font-size:13px;">
                    Once we receive your payment, we'll confirm it via email and deliver your
artworks within 24 hours.
                </p>
                <p style="margin:0;color:#999;font-size:12px;">
                    Order will remain valid for 48 hours.
                </p>

                <p style="margin-top:30px;font-size:14px;color:#555;">
                    Thank you for your trust!<br>
                    <strong>- MC Artworks Studio</strong>
                </p>
            </td>
        </tr>
        <tr>
                <td style="background:#4B0082;color:#ffffff;font-size:12px;text-
align:center;padding:15px;font-weight:600;">
                    © ' . date('Y') . ' MC Artworks Studio – All rights reserved
                </td>
            </tr>
        </table>
    </body>
</html>';

```

El resumen inserta tres piezas que ya estaban preparadas: \$order_date, \$artwork_html y \$items_html. Después get_total() agrega el total del pedido. El texto final informa que los artworks se entregarán dentro de 24 horas después de confirmar el pago y que el pedido permanece válido durante 48 horas. date('Y') completa el año del footer.

La presencia de "' . \$items_html . '" dentro de la plantilla no vuelve a ejecutar el foreach. En ese punto PHP simplemente inserta la cadena HTML que ya fue construida en 27.2.

27.7 Headers, asunto, wp_mail() y registro del resultado

Código fuente real: functions.php → final del correo del comprador → líneas 7501-7514.

```

// === Enviar correo ===
$headers = [
    'Content-Type: text/html; charset=UTF-8',
    'From: MCArtworks Studio <shop@mcartworksstudio.com>'
];

$subject = '📧 Order #' . $order->get_order_number() . ' - Payment Details from
MCArtworks Studio';

$result = wp_mail($order->get_billing_email(), $subject, $message, $headers);

if (!$result) {
    $this->log_manual_pay("⚠ ERROR sending email to {$order->get_billing_email()} for order
#{$order->get_order_number()}");
} else {
    $this->log_manual_pay("✅ Email sent successfully to {$order->get_billing_email()} for order
#{$order->get_order_number()}");
}

```

\$headers indica que el contenido es text/html con UTF-8 y define el remitente visible. \$subject incorpora el número real del pedido. wp_mail() recibe como destinatario el email de facturación del WC_Order, luego asunto, cuerpo HTML y cabeceras. El booleano devuelto se guarda en \$result y se registra inmediatamente mediante log_manual_pay().

La evaluación ocurre antes de comenzar el correo administrativo. Por eso el resultado del comprador queda escrito en el log aunque más adelante la variable \$result se vuelva a utilizar para el segundo wp_mail().

27.8 Ejemplo guiado: qué valores recibe el correo de Alex

```

$order->get_billing_email() → alex@example.com
$subject → Order #1530 - Payment Details from MCArtworks Studio
$artwork_html → Raven
$items_html → Cyberpunk Portrait + Blue + Welcome back
$order->get_total() → 18.00
↓
wp_mail(alex@example.com, $subject, $message, $headers)
↓
$result
↓
log del éxito o del error

```

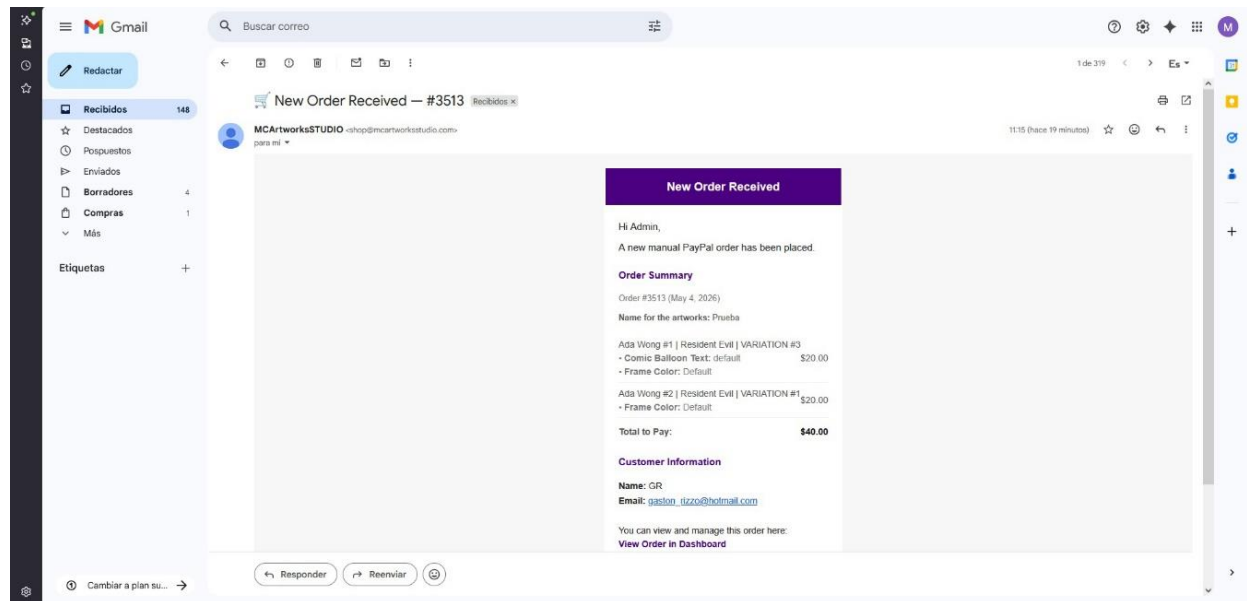
Así, la plantilla no contiene valores ficticios escritos a mano para cada pedido. PHP inserta los datos que lee del WC_Order concreto que está procesando.

28. Correo al administrador

Después de evaluar y registrar el wp_mail() del comprador, process_payment() continúa sin salir del método. El segundo correo tiene una plantilla HTML distinta: está pensado para administración, reutiliza el resumen de productos ya construido y agrega datos internos del cliente junto con un enlace para gestionar el pedido.

Ubicación en el proyecto: wp-content/themes/blocks-child/functions.php → WC_Gateway_Manual_Pay::process_payment() → líneas 7520-7584. No comienza un segundo proceso de WooCommerce ni se vuelve a crear el pedido.

Continuidad temporal: en memoria siguen disponibles \$order, \$order_id, \$artwork_html, \$items_html y \$headers. \$order_date se calcula nuevamente antes de construir \$admin_message.



Ejemplo real del correo administrativo. Debajo se documenta el código fuente que construye esta salida.

28.1 Destinatario, asunto y fecha

Código fuente real: functions.php → inicio del bloque administrativo → líneas 7520-7523.

```
$admin_email = 'mcartworksstudio@gmail.com';
$order_date = $order->get_date_created() ? $order->get_date_created()->date('F j, Y') : '';
```

\$admin_email contiene el destinatario interno fijo configurado en el tema. \$admin_subject incorpora el número del pedido que se está procesando. \$order_date vuelve a obtener la fecha de creación del mismo WC_Order para mostrarla dentro del resumen.

```
WC_Order #1530
↓
$admin_email = mcartworksstudio@gmail.com
$admin_subject = New Order Received — #1530
$order_date = fecha de creación del pedido
```

28.2 Plantilla administrativa: cabecera y resumen del pedido

Código fuente real: functions.php → comienzo de \$admin_message → líneas 7525-7549. Estas líneas forman la primera parte de una única cadena HTML.

```
$admin_message = '
<html>
<body style="font-family: Arial, sans-serif; background-color: #f7f7f7; padding: 20px;
color: #222;">
    <table style="max-width:600px; margin:auto; background:#fff; border-radius:8px;
overflow:hidden;">
        <tr>
            <td style="background:#4B0082; color:#fff; text-align:center; padding:15px;
font-size:18px;">
                <strong>New Order Received</strong>
            </td>
        </tr>
        <tr>
            <td style="padding:20px;">
                <p style="font-size:15px; margin:0 0 10px 0;">Hi Admin,</p>
                <p style="font-size:15px; margin:0 0 15px 0;">A new manual PayPal
order has been placed.</p>

                <h3 style="color:#4B0082; margin-top:20px;">Order Summary</h3>
                <p style="color:#777;margin:0 0 10px 0;">Order #'. $order-
```

```

>get_order_number() . ' (' . $order_date . ')</p>
' . $artwork_html . '

<table width="100%" cellpadding="0" cellspacing="0" style="font-
size:14px;color:#555;border-collapse:collapse;">
' . $items_html . '
<tr>
  |
```

La plantilla abre un documento HTML independiente y crea una tarjeta central. El encabezado violeta muestra “New Order Received”. En Order Summary se insertan el número, la fecha, \$artwork_html y \$items_html. El total se obtiene otra vez desde \$order->get_total().

La reutilización de \$items_html es deliberada: el correo administrativo no vuelve a recorrer Cart ni vuelve a construir los metadatos. Usa exactamente las filas HTML que process_payment() generó antes para el comprador, lo que mantiene el mismo producto, Frame Color y Comic Balloon Text en ambos mensajes.

28.3 Plantilla administrativa: cliente y enlace al Dashboard

Código fuente real: continuación de \$admin_message → líneas 7551-7574. El enlace se genera dentro del atributo href mediante admin_url().

```

<h3 style="color:#4B0082; margin-top:25px;">Customer Information</h3>
<p style="font-size:14px; line-height:1.6em;">
  <strong>Name:</strong> ' . esc_html($order-
>get_billing_first_name()) . ' ' . esc_html($order->get_billing_last_name()) . '<br>
  <strong>Email:</strong> ' . esc_html($order->get_billing_email()) .
'<br>
</p>

<p style="margin-top:25px; font-size:14px;">
  You can view and manage this order here:<br>
  <a href="" . admin_url('post.php?post=' . $order_id .
'&action=edit') . '"
  style="color:#4B0082; font-weight:bold; text-
decoration:none;">
    View Order in Dashboard
  </a>
</p>
</td>
</tr>
<tr>
  |
```

Customer Information lee First Name, Last Name y Email desde el WC_Order y los escapa con esc_html(). Después admin_url() construye una URL del panel de WordPress a partir de post.php?post=ORDER_ID&action=edit. \$order_id es el mismo identificador que WooCommerce entregó al comienzo de process_payment().

```

Email administrativo
↓
View Order in Dashboard
↓
admin_url( post.php?post=1530&action=edit )
↓
pedido #1530 en el panel de WordPress / WooCommerce

```

La documentación describe lo que construye el código fuente actual. El enlace no busca el pedido por nombre ni por email: utiliza directamente \$order_id.

28.4 Envío, resultado y cierre del bloque

Código fuente real: functions.php → líneas 7576-7584.

```
$result = wp_mail($admin_email, $admin_subject, $admin_message, $headers);

if (!$result) {
    $this->log_manual_pay("⚠ ERROR sending email to the admin for order #{ $order->get_order_number() }");
} else {
    $this->log_manual_pay("✅ Email sent successfully to the admin for order #{ $order->get_order_number() }");
}

$this->log_manual_pay("=====\\n\\n");
```

wp_mail() utiliza el destinatario, asunto y cuerpo administrativo, pero reutiliza \$headers del correo del comprador. La asignación vuelve a escribir \$result; inmediatamente después, el código registra éxito o error con un mensaje específico para administración. Finalmente se agrega una línea separadora al archivo de log para cerrar visualmente el registro de ese pedido.

28.5 Ejemplo guiado: Alex atraviesa todo el correo administrativo

```
WC Order #1530 ya existe
↓
$admin_subject = New Order Received - #1530
↓
$admin_message inserta:
Raven
Cyberpunk Portrait
Frame Color: Blue
Comic Balloon Text: Welcome back
Total: $18.00
Alex · alex@example.com
View Order in Dashboard → order_id 1530
↓
wp_mail(mcartworksstudio@gmail.com, ...)
↓
$result
↓
log_manual_pay()
```

Ésta es la diferencia respecto del correo del comprador: ambos comparten el resumen del pedido, pero el mensaje administrativo elimina las instrucciones de pago y agrega información para revisar y gestionar el pedido internamente.

29. Envío de correo y seguimiento de errores

Las secciones 27 y 28 explican cómo el tema construye los dos cuerpos HTML. Esta sección sigue el paso siguiente: qué recibe wp_mail(), qué significa su resultado y cómo distinguir un problema del código del tema, un problema de transporte SMTP y un problema del archivo de log.

Ubicación en el proyecto: functions.php → WC_Gateway_Manual_Pay::process_payment(). El wp_mail() del comprador está en la línea 7508 y el administrativo en la 7576. Los dos resultados se registran mediante WC_Gateway_Manual_Pay::log_manual_pay(). WP Mail SMTP no está implementado dentro del child theme; actúa después, en la infraestructura de correo de WordPress.

29.1 Las dos llamadas reales a wp_mail()

Código fuente real del comprador: functions.php → líneas 7502-7508.

```
$headers = [
    'Content-Type: text/html; charset=UTF-8',
    'From: MC Artworks Studio <shop@mcartworksstudio.com>'
];

$subject = '📦 Order #' . $order->get_order_number() . ' - Payment Details from
```

```
MCArtworks Studio';
```

```
$result = wp_mail($order->get_billing_email(), $subject, $message, $headers);
```

Código fuente real de administración: functions.php → línea 7576. \$headers ya existe porque fue creado antes para el comprador.

```
$result = wp_mail($admin_email, $admin_subject, $admin_message, $headers);
```

Las dos llamadas usan la misma función de WordPress y la misma estructura de cuatro argumentos. Cambian el destinatario, el asunto y el cuerpo HTML.

Argumento de wp_mail()	Comprador	Administración
\$to	\$order->get_billing_email()	\$admin_email
\$subject	\$subject	\$admin_subject
\$message	\$message	\$admin_message
\$headers	\$headers	\$headers

29.2 Qué significa cada argumento

El primer argumento indica el destinatario. El segundo es el asunto. El tercero es el cuerpo completo del mensaje; en este proyecto contiene HTML. El cuarto contiene las cabeceras, entre ellas Content-Type: text/html; charset=UTF-8 y el remitente From. No se pasa un quinto argumento de attachments, por lo que estos correos no adjuntan archivos mediante wp_mail().

wp_mail() no vuelve a leer el WC_Order. Cuando se llama, PHP ya convirtió los datos del pedido en \$message o \$admin_message. wp_mail() recibe cadenas ya preparadas y las entrega al sistema de correo de WordPress.

29.3 Qué ocurre después de wp_mail()

```
functions.php / process_payment()
↓
wp_mail()
↓
WordPress prepara el envío
↓
WP Mail SMTP interviene como infraestructura de transporte
↓
servidor o servicio SMTP configurado
↓
servidor del destinatario
↓
Inbox / Spam / rechazo
```

El tema hijo controla la construcción del destinatario, asunto, HTML y cabeceras. A partir de wp_mail(), la entrega depende también de WordPress, de la configuración del plugin SMTP, del servicio saliente y del servidor receptor. Por eso el estado del pedido y el estado de entrega del correo son problemas relacionados pero distintos.

29.4 Cómo se evalúa el resultado del comprador

Código fuente real: functions.php → líneas 7510-7514.

```
if (!$result) {
    $this->log_manual_pay("⚠ ERROR sending email to {$order->get_billing_email()} for order
#{$order->get_order_number()}");
} else {
    $this->log_manual_pay("✅ Email sent successfully to {$order->get_billing_email()} for order
#{$order->get_order_number()}");
}
```

Si \$result es false, el gateway escribe un ERROR asociado al email del comprador y al número del pedido. Si es true, registra que WordPress aceptó correctamente el intento de envío. El log se escribe antes de comenzar el bloque administrativo.

29.5 Cómo se evalúa el resultado administrativo

Código fuente real: functions.php → líneas 7578-7582.

```
if (!$result) {
    $this->log_manual_pay("⚠ ERROR sending email to the admin for order #{ $order->get_order_number() }");
} else {
    $this->log_manual_pay("✅ Email sent successfully to the admin for order #{ $order->get_order_number() }");
}
```

La lógica es equivalente, pero el mensaje de log identifica explícitamente al administrador. De esta forma el archivo diario puede mostrar que un correo funcionó y el otro falló sin confundir ambos eventos.

29.6 Por qué reutilizar \$result no borra el primer resultado

La variable \$result sí se sobrescribe cuando se ejecuta el segundo wp_mail(). Sin embargo, el valor anterior ya fue evaluado en el if de las líneas 7510-7514 y su mensaje ya fue enviado a log_manual_pay(). Lo que se pierde en memoria es el booleano anterior; no se borra la línea que ya quedó escrita en el archivo.

```
wp_mail(comprador) → $result #1
↓
if/else evalúa $result #1
↓
log_manual_pay() escribe el resultado del comprador
↓
wp_mail(admin) → $result #2 reemplaza la variable
↓
if/else evalúa $result #2
↓
log_manual_pay() escribe el resultado administrativo
```

29.7 Tres niveles de fallo que no deben confundirse

Nivel	Qué puede ocurrir	Dónde mirar
1. wp_mail()	wp_mail() devuelve false y process_payment() registra ERROR.	Archivo diario de manual_pay y código del gateway.
2. Transporte SMTP	wp_mail() devuelve true, pero el mensaje puede terminar en Spam, ser rechazado o fallar después.	Registros/configuración de WP Mail SMTP y servicio SMTP.
3. Archivo de log	El envío puede intentarse correctamente, pero file_put_contents() puede fallar al guardar el registro.	Permisos de uploads/logs y error_log().

No confundir “wp_mail() devolvió true” con “el destinatario leyó el correo” o incluso con “el mensaje llegó a Inbox”. true sólo confirma que WordPress pudo procesar la solicitud de envío en ese punto del flujo.

29.8 Qué ocurre si falla el propio archivo de log

Código fuente real: functions.php → final de log_manual_pay() → líneas 7316-7319.

```
$written = @file_put_contents($log_file, $entry, FILE_APPEND | LOCK_EX);
if ($written === false) {
    error_log("manual-pay log write FAILED: $entry");
}
```

Si la escritura del archivo devuelve false, error_log() recibe el texto como salida alternativa. El proyecto no implementa aquí una cola de reintentos, un segundo archivo de respaldo ni una base de datos propia para estos eventos.

29.9 Por qué el pedido continúa aunque falle un correo

Código fuente real: functions.php → final de process_payment() → líneas 7586-7589. El return se ejecuta después de los dos intentos de correo.

```
return [
    'result' => 'success',
```

```
'redirect' => $this->get_return_url($order),
];
```

Los if de correo sólo registran éxito o error; no hacen return de error ni cancelan el WC_Order. Por eso process_payment() llega igualmente a result = success y devuelve la URL de retorno del pedido. En la siguiente petición, el navegador abrirá Order Received, que se desarrolla en la sección 30.

```
wp_mail() puede fallar
↓
se registra el ERROR
↓
WC_Order se conserva
↓
process_payment() continúa
↓
return result = success + get_return_url($order)
↓
Order Received
```

29.10 Seguimiento operativo paso a paso

Para diagnosticar un pedido manual conviene seguir el mismo orden que el código. Primero comprobar que el WC_Order existe y contiene productos y personalizaciones. Después revisar el archivo diario de manual_pay para saber qué devolvió cada wp_mail(). Si el log indica éxito pero el mensaje no aparece en el buzón, revisar WP Mail SMTP y el servicio SMTP. Si ni siquiera aparece el evento en el archivo, comprobar uploads/logs y el error log del servidor.

```
1. WooCommerce → ¿existe el WC_Order y tiene los datos correctos?
↓
2. uploads/logs/FECHA_orders.log → ¿qué registró process_payment()?
↓
3. WP Mail SMTP / servicio SMTP → ¿el transporte aceptó o rechazó el mensaje?
↓
4. servidor receptor → ¿Inbox, Spam o rechazo?
```

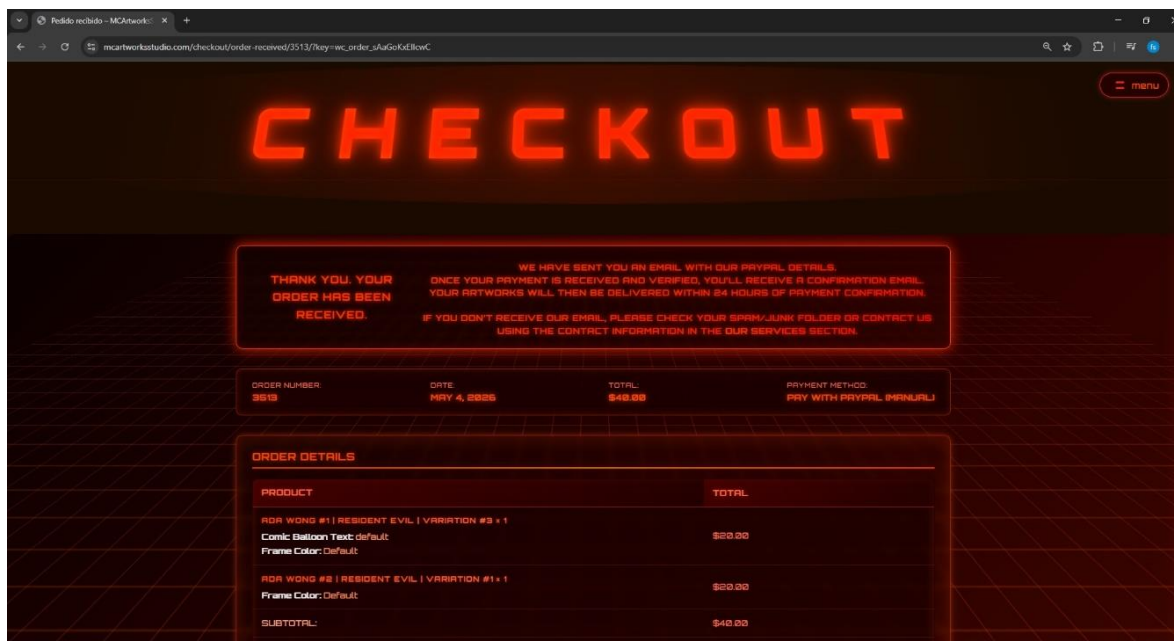
30. Pedido recibido (Order Received) y ejemplo completo

Después de procesar el método manual, WooCommerce abre la página de retorno del pedido. La pantalla Order Received también está personalizada para mantener la apariencia de la facción y mostrar qué debe hacer el comprador a continuación.

Cambio de petición HTTP. Cuando process_payment() devuelve result=success y get_return_url(\$order), ese método termina. El navegador es redirigido y abre la URL de Order Received en una petición nueva. Las funciones de esta sección se ejecutan durante esa nueva carga de WordPress; no continúan dentro de process_payment().

Ubicación en el proyecto: functions.php · el return del gateway está aproximadamente en las líneas 7586-7589; wpo_filtrar_meta_en_pedido() se registra en woocommerce_order_item_get_formatted_meta_data alrededor de la línea 7842 y comienza cerca de la 7861; traducir_textos_pedido_recibido() se registra en wp_footer alrededor de la 8127 y comienza cerca de la 8133; mostrar_mensaje_y_traducir_meses() se registra en wp_footer alrededor de la 8318 y comienza cerca de la 8332.

```
PETICIÓN 1: Checkout
↓
WC_Gateway_Manual_Pay::process_payment()
↓
return success + get_return_url($order)
↓
redirección del navegador
=====
PETICIÓN 2: Order Received
↓
WordPress + WooCommerce reconstruyen la pantalla
↓
wp_footer ejecuta traducciones/mensaje adicional
↓
el navegador ejecuta el JavaScript impreso
```



Pantalla posterior al checkout vido ya creado.

El mismo dato a lo largo de toda la compra

Etapas	Dato visible / conservado
Tienda	Cyberpunk Portrait — USD 18
Cart	Frame Color = Blue · Comic Balloon Text = Welcome back
Checkout	Artworks Name = Raven · First Name = Alex · Email = alex@example.com
Pedido	Producto + metadatos de personalización + Artworks Name
Log	Número de pedido, cliente, correo, producto, opciones y total
Email comprador	Resumen + instrucciones de pago manual
Email admin	Resumen + datos del cliente + enlace al pedido
Order Received	Confirmación del pedido y próximo paso de pago

30.1 Cómo se llega a Order Received

Código fuente: functions.php → final de WC_Gateway_Manual_Pay::process_payment(), aproximadamente líneas 7586-7589. La URL devuelta pertenece a WooCommerce y provoca la navegación posterior.

Al terminar process_payment(), la pasarela no imprime por sí misma una página final. Devuelve a WooCommerce un resultado success y la URL calculada por get_return_url(\$order). WooCommerce realiza entonces la redirección hacia el endpoint de pedido recibido.

```
return [
    'result' => 'success',
    'redirect' => $this->get_return_url($order),
];
```

30.2 Qué aporta WooCommerce y qué aporta el tema

WooCommerce genera la estructura base con número, fecha, email, método de pago, productos y total. El tema hijo mantiene esa información, pero modifica textos, agrega instrucciones y adapta la presentación para que la pantalla final sea coherente con el resto del sitio.

30.3 Traducción dinámica de textos

Código fuente: functions.php → hook wp_footer → traducir_textos_pedido_recibido(), aproximadamente líneas 8127-8172. PHP comprueba is_order_received_page() y, si corresponde, imprime JavaScript; el reemplazo textual ocurre luego en el navegador.

traducir_textos_pedido_recibido() se ejecuta únicamente cuando is_order_received_page() es verdadero. JavaScript busca textos generados por WooCommerce en español y los reemplaza por sus equivalentes en inglés sin editar las plantillas del plugin.

La función no modifica plantillas de WooCommerce. Recorre nodos de texto ya generados en la página y reemplaza únicamente las cadenas conocidas por sus equivalentes en inglés.

```
const traducciones = [
  { find: 'Gracias. Tu pedido ha sido recibido.',
    replace: 'THANK YOU. YOUR ORDER HAS BEEN RECEIVED.' },
  { find: 'Número de pedido:', replace: 'ORDER NUMBER:' },
  { find: 'Fecha:', replace: 'DATE:' },
  { find: 'Correo electrónico:', replace: 'EMAIL:' },
  { find: 'Método de pago:', replace: 'PAYMENT METHOD:' }
];

traducciones.forEach(t => {
  const regex = new RegExp(t.find, 'gi');

  jQuery('body').find('*').contents()
    .filter(function() {
      return this.nodeType === 3 && regex.test(this.nodeValue);
    })
    .each(function() {
      this.nodeValue = this.nodeValue.replace(regex, t.replace);
    });
});
```

“Gracias. Tu pedido ha sido recibido.”

↓

THANK YOU. YOUR ORDER HAS BEEN RECEIVED.

“Número de pedido:” → ORDER NUMBER:

“Fecha:” → DATE:

“Método de pago:” → PAYMENT METHOD:

30.4 Mensaje adicional después del checkout

Código fuente: functions.php → hook wp_footer con prioridad 100 → mostrar_mensaje_y_traducir_meses(), aproximadamente líneas 8318-8399. PHP prepara \$extra_text y entrega el texto al script mediante wp_json_encode().

mostrar_mensaje_y_traducir_meses() agrega dentro del mensaje de agradecimiento una explicación práctica: el detalle PayPal fue enviado por email, el pago debe ser verificado y los artworks se entregan dentro de 24 horas después de confirmar el pago. También recomienda revisar Spam/Junk si el correo no aparece.

El mensaje se crea como un adicional dentro del párrafo de confirmación existente. El contenido procede de \$extra_text y se pasa a JavaScript mediante wp_json_encode() para no insertar manualmente una cadena PHP dentro del script.

```
var p = document.querySelector(
  '.woocommerce-thankyou-order-received'
);

if (p) {
  var span = document.createElement('span');
  span.className = 'woocommerce-thankyou-extra';
  span.style.display = 'block';
  span.style.marginTop = '8px';
  span.innerHTML = <?php echo wp_json_encode($extra_text); ?>;
  p.appendChild(span);
}
```

30.5 Fechas y nombres de meses

La misma función recorre el HTML de la orden y cambia nombres de meses del español al inglés. Esta transformación sólo se aplica en order-received; no modifica la fecha almacenada en WooCommerce.

El reemplazo se realiza sólo sobre el HTML del contenedor del pedido mostrado en Order Received. No cambia la fecha almacenada en WC_Order ni la configuración regional de WordPress.

```

var monthsMap = {
    'enero': 'January',
    'febrero': 'February',
    'marzo': 'March',
    'abril': 'April',
    'mayo': 'May',
    'junio': 'June',
    'julio': 'July',
    'agosto': 'August',
    'septiembre': 'September',
    'octubre': 'October',
    'noviembre': 'November',
    'diciembre': 'December'
};

Object.keys(monthsMap).forEach(function(spanish) {
    var re = new RegExp('\\b' + spanish + '\\b', 'ig');
    html = html.replace(re, monthsMap[spanish]);
});

```

septiembre → September
 octubre → October
 noviembre → November
 diciembre → December

30.6 Metadatos visibles por producto

Código fuente: functions.php → filtro woocommerce_order_item_get_formatted_meta_data → wpo_filtrar_meta_en_pedido(), aproximadamente líneas 7842-7877. Este filtro actúa sobre metadatos ya formateados por WooCommerce antes de mostrarlos.

wpo_filtrar_meta_en_pedido() controla los metadatos que WooCommerce muestra debajo de cada producto. Su objetivo es impedir que Comic Balloon Text aparezca en líneas donde no corresponde.

Este fragmento muestra exactamente de dónde sale la observación sobre es_bundle: la función lo consulta junto con tiene_globo antes de decidir si elimina Comic Balloon Text de los metadatos visibles.

```

function wpo_filtrar_meta_en_pedido($formatted_meta, $item) {
    $product = $item->get_product();
    if (!$product) return $formatted_meta;

    $product_id = $product->get_id();
    $is_bundle = get_field('es_bundle', $product_id);
    $tiene_globo = get_field('tiene_globo', $product_id);

    if ($is_bundle || !$tiene_globo) {
        $formatted_meta = array_filter(
            $formatted_meta,
            function($meta) {
                return stripos(
                    $meta->display_key,
                    'Comic Balloon Text'
                ) === false;
            }
        );
    }

    return $formatted_meta;
}

```

Diferencia entre el código y la configuración ACF

wpo_filtrar_meta_en_pedido() consulta es_bundle además de tiene_globo. El código espera ese campo, pero el relevamiento actual de ACF no lo incluye entre los cuatro campos confirmados. Cuando es_bundle no está configurado, esa comprobación devuelve un valor vacío y el filtro sólo puede apoyarse en tiene_globo para decidir si oculta Comic Balloon Text.

30.7 Ejemplo completo de Alex

Alex ya no está en la misma respuesta de Checkout. Su navegador cargó una nueva URL de WooCommerce. En esa nueva petición WordPress obtiene el pedido correspondiente, WooCommerce genera su resumen y recién entonces los callbacks del tema modifican los textos visibles y agregan las instrucciones posteriores al pago.

```
SHOP
Cyberpunk Portrait – USD 18
↓
CART
Frame Color = Blue
Comic Balloon Text = Welcome back
↓
CHECKOUT
Artworks Name = Raven
First Name = Alex
Email = alex@example.com
↓
WC_ORDER #1530
producto + metadatos + Artworks Name
↓
LOG DIARIO
detalle del pedido
↓
EMAIL COMPRADOR + EMAIL ADMIN
↓
get_return_url()
↓
ORDER RECEIVED
confirmación + instrucciones para completar el pago
```

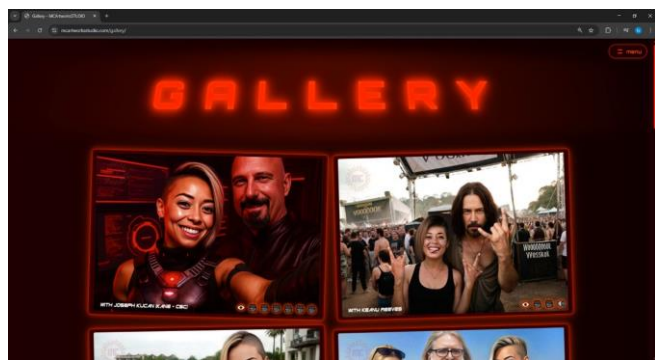
PARTE V — GALERÍA, SERVICIOS E INTERFAZ

31. Galería (Gallery)

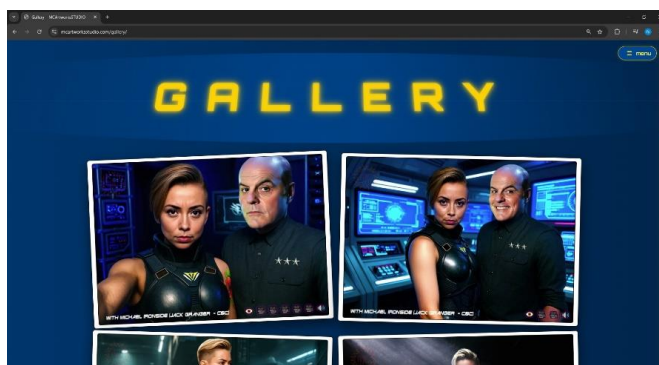
La Galería es una pantalla personalizada del tema hijo. No obtiene sus videos desde WooCommerce, Advanced Custom Fields (ACF) ni una consulta a la base de datos. La lista de contenidos se define directamente en `functions.php`, dentro de `cnc_gallery_parallax()`. Esa función decide qué archivos forman cada marco, qué variantes alternativas tiene y si debe mostrarse un control de sonido.

Ubicación en el proyecto: `wp-content/themes/blocksy-child/functions.php · add_action('wp_head', 'cnc_gallery_parallax')` alrededor de la línea 8410 y `cnc_gallery_parallax()` desde aproximadamente la 8418 hasta antes del bloque `Our Services`. La función reúne configuración PHP, HTML y JavaScript de Gallery. La integración con el overlay del menú pertenece a `wp-content/themes/blocksy-child/header.php`; los estilos visuales dependen de `gdi-faction-min.css` / `nod-faction-min.css`. El contenido técnico existente de 31.1-31.16 se conserva y el ejemplo de Alex de 31.17 sólo lo recorre desde la experiencia del usuario.

Los MP4 existen físicamente en el servidor, pero la carpeta no se recorre automáticamente. Que un archivo esté guardado dentro de uploads no hace que aparezca en la Galería: sólo participa si su URL fue incluida de forma explícita en el array `$videos` de la facción correspondiente.



Galería de la facción NOD



Galería de la facción GDI

31.1 De dónde sale cada parte de la Galería

La pantalla se construye combinando cuatro piezas distintas. Separarlas permite entender de dónde sale cada elemento que finalmente ve el visitante.

- `functions.php`: contiene `cnc_gallery_parallax()`, los arrays de videos GDI/NOD, el HTML de los 12 marcos y el JavaScript que controla loaders, variantes, sonido y parallax.
- `gdi-faction-min.css` / `nod-faction-min.css`: aportan el aspecto visual de la facción activa: colores, marcos, título, grilla, controles, loaders y comportamiento responsive.
- `wp-content/uploads/videos/`: contiene los archivos MP4 físicos. En el caso NOD proporcionado, los archivos de Gallery están en `wp-content/uploads/videos/NOD-GALLERY/`.
- `header.php`: integra Gallery con el menú principal; al abrir el overlay sobre `/gallery`, pausa los elementos `<audio>` externos, silencia los `<video>` externos y guarda su estado para restaurarlo al cerrar el menú.

```
/gallery/
↓
WordPress carga el tema hijo
↓
la cookie faction determina GDI o NOD
↓
se carga el CSS de esa facción
↓
wp_head ejecuta cnc_gallery_parallax()
↓
functions.php selecciona el array $videos
↓
PHP genera 12 frame-container
↓
```

```

el navegador carga los 12 videos main
↓
JavaScript conecta loaders y controles

```

31.2 Dónde se activa y dónde está implementada

No existe un archivo `page-gallery.php` dentro del tema hijo. La implementación actual está agrupada en `functions.php`. WordPress ejecuta `cnc_gallery_parallax()` mediante `wp_head`, y la función se detiene inmediatamente si la página solicitada no es Gallery.

```

add_action('wp_head', 'cnc_gallery_parallax');

function cnc_gallery_parallax() {
    if (!is_page('gallery')) return;

    // configuración, HTML y JavaScript de Gallery
}

```

Por lo tanto, la misma función reúne tres capas: primero prepara los datos en PHP, después imprime el HTML de la Galería y finalmente agrega el JavaScript que controla la interacción.

31.3 Cómo se determina la facción y de dónde sale el aspecto visual

Gallery lee la misma cookie `faction` utilizada por el resto del sitio. Si no existe, toma GDI como valor predeterminado. Esa elección afecta tanto los archivos de video como el fondo y el loader.

```

$faction = $_COOKIE['faction'] ?? 'gdi';

if ($faction === 'gdi') {
    $body_bg = 'linear-gradient(270deg, #001f3f, #003f7f, #005bbb, #001f3f)';
    $loader_class = 'video-loader gdi';
    $videos = [ /* configuración GDI */ ];
} elseif ($faction === 'nod') {
    $body_bg = 'linear-gradient(270deg, #2a0000, #550000, #2a0000)';
    $loader_class = 'video-loader nod';
    $videos = [ /* configuración NOD */ ];
}

```

La carga del CSS de facción también se decide desde la cookie, pero la implementación actual contiene dos bloques `wp_enqueue_scripts` activos con prioridad 999. El bloque marcado como **PRODUCCIÓN** elimina estilos previos y utiliza `filemtime()` como versión; el bloque marcado como **VERSIÓN DE DESARROLLO** intenta encolar la misma hoja con un parámetro `?v=` generado mediante `time()` para invalidar la caché durante el desarrollo.

```

/* PRODUCCIÓN */
add_action('wp_enqueue_scripts', function() {
    $faction = $_COOKIE['faction'] ?? 'gdi';
    if (empty($faction)) return;

    wp_dequeue_style('gdi-faction');
    wp_dequeue_style('nod-faction');

    if ($faction === 'gdi') {
        $css_file = get_stylesheet_directory() . '/gdi-faction-min.css';
        $version = file_exists($css_file) ? filemtime($css_file) : '1.0';
        wp_enqueue_style('gdi-faction', get_stylesheet_directory_uri() . '/gdi-faction-min.css', [],
        $version, 'all');
    }
    // NOD aplica el mismo esquema con nod-faction-min.css.
}, 999);

/* VERSIÓN DE DESARROLLO */
add_action('wp_enqueue_scripts', function() {
    $faction = $_COOKIE['faction'] ?? '';
    if (empty($faction)) return;

    $theme_uri = get_stylesheet_directory_uri();

```

```

$timestamp = time();

if ($faction === 'gdi') {
    $url = $theme_uri . '/gdi-faction-min.css?v=' . $timestamp;
    wp_enqueue_style('gdi-faction', $url, [], null, 'all');
}
// NOD aplica el mismo esquema con nod-faction-min.css.
}, 999);

```

Los dos bloques aparecen activos en functions.php. Por eso se documentan por separado: uno está planteado para producción y el otro para desarrollo. La selección GDI/NOD sigue dependiendo de la cookie faction, pero la estrategia de versionado y caché no es la misma en ambos bloques.

Un detalle que puede resultar confuso al leer el código es que el contenedor conserva la clase gdi-gallery-wrapper incluso cuando la facción es NOD. Esto no significa que NOD use el diseño GDI: ambas hojas de estilos definen ese mismo selector con reglas distintas y la cookie decide cuál de ellas se carga.

El fondo también se aplica en dos niveles. cnc_gallery_parallax() imprime un background para body mediante \$body_bg, mientras la hoja de estilos de la facción define el fondo y los efectos propios de .gdi-gallery-wrapper. El loader funciona de manera similar: existe una base .video-loader y luego una variante .video-loader.gdi o .video-loader.nod.

31.4 La fuente real de los videos: el array \$videos

Código fuente: wp-content/themes/blocksy-child/functions.php → cnc_gallery_parallax() → selección de facción y arrays \$videos aproximadamente entre las líneas 8421-8454. Cada entrada se define manualmente en PHP; Gallery no recorre la carpeta para descubrir archivos.

El punto central de Gallery es \$videos. No se usa glob(), scandir(), wp_upload_dir(), ACF ni una consulta a WooCommerce para descubrir archivos. Cada video que puede aparecer está escrito manualmente en este array.

Cada entrada representa un marco de la Galería y contiene exactamente tres datos:

main: video principal del marco. Se crea desde PHP y comienza a cargarse al abrir Gallery.

alt: array de uno o más videos alternativos. No se crean inicialmente; JavaScript los genera bajo demanda.

sound: bandera booleana que indica si PHP debe imprimir el botón de sonido para ese marco.

```

[
    'main' => '/videos/nod-gallery/video2_main.mp4',
    'alt' => [
        '/videos/nod-gallery/video2_alt_one.mp4',
        '/videos/nod-gallery/video2_alt_two.mp4'
    ],
    'sound' => true
]

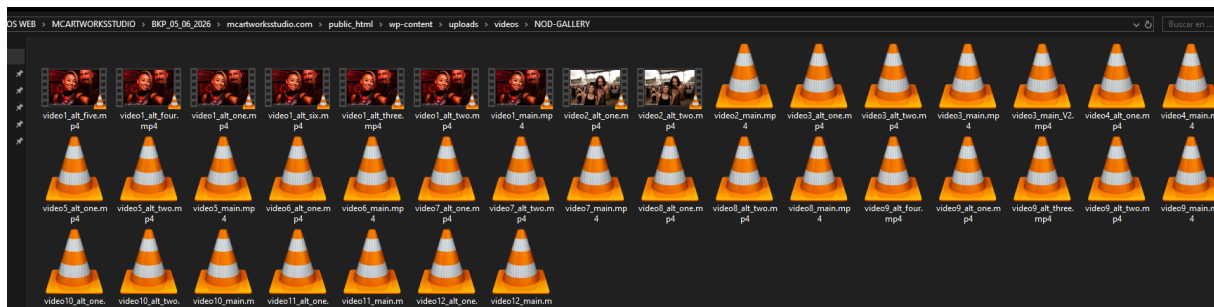
```

Tanto GDI como NOD tienen 12 entradas, por lo que la pantalla genera 12 marcos en cualquiera de las dos facciones. En el código actual cada facción referencia 12 archivos main y 26 archivos alt: 38 URLs de video por configuración.

El número de alternativas no es igual para todos los marcos. Por ejemplo, NOD video1 posee seis variantes, video2 posee dos y video4 posee una. El número de botones  se obtiene directamente de esa cantidad.

31.5 Archivos físicos y URLs públicas: no son la misma cosa

En la captura proporcionada, los archivos NOD están físicamente en public_html/wp-content/uploads/videos/NOD-GALLERY/. Allí se observan los archivos video1_main.mp4, video1_alt_one.mp4, video1_alt_two.mp4 y el resto de la serie.



Archivos físicos de NOD Gallery dentro de wp-content/uploads/videos/NOD-GALLERY/.

Sin embargo, functions.php no utiliza esa ruta física al construir el HTML. Las URLs escritas en \$videos son del tipo /videos/nod-gallery/video1_main.mp4. El child theme no contiene la regla que relaciona /videos/nod-gallery/ con wp-content/uploads/videos/NOD-GALLERY/; esa correspondencia pertenece a la configuración del servidor y debe conservarse al desplegar el sitio.

Ubicación física confirmada:

public_html/wp-content/uploads/videos/NOD-GALLERY/video1_main.mp4

URL utilizada por functions.php:

/videos/nod-gallery/video1_main.mp4

Esto también permite distinguir almacenamiento de configuración. La captura de NOD muestra 39 elementos, mientras el array NOD de functions.php referencia 38 archivos. Entre los archivos físicos aparece video3_main_V2.mp4, pero ese nombre no está incluido en el array \$videos del tema. Por lo tanto, ese archivo existe en el servidor pero Gallery no lo utiliza.

La regla práctica es simple: un archivo extra dentro de la carpeta se ignora si no está en \$videos; en cambio, si \$videos referencia una URL inexistente o mal mapeada, el navegador intentará cargarla y la Galería no tiene un mecanismo propio para sustituirla por otro archivo.

31.6 Cómo PHP convierte \$videos en los 12 marcos

Código fuente: functions.php → cnc_gallery_parallax() → foreach(\$videos as \$v) aproximadamente entre las líneas 8494-8510. Este tramo se ejecuta en el servidor y deja main, data-alt y data-sound escritos en el HTML que recibirá el navegador.

Una vez elegido el array de la facción, PHP lo recorre con foreach(). Cada entrada produce un frame-container. El video main se escribe directamente en el HTML; las rutas alt se guardan dentro del atributo data-alt en formato JSON, y sound se convierte en 1 o 0 dentro de data-sound.

```
<?php foreach ($videos as $v): ?>

<div class="frame-container"
  data-alt='<?=json_encode($v['alt']) ?>'
  data-sound='<?=$v['sound'] ? 1 : 0 ?>'>

  <div class="video-wrap">
    <video class="vid-main"
      src="<?=$v['main'] ?>"
      autoplay
      muted
      loop
      playsinline
      preload="auto">
    </video>
  </div>

  <div class="frame"></div>

  <div class="video-controls">
    <span class="icon-eye">👁</span>

    <?php foreach ($v['alt'] as $index => $alt): ?>
      <span class="icon-video">
```

```

        data-alt="<?= $index ?>"><img alt="icon" data-bbox="412 58 428 74"/></span>
    <?php endforeach; ?>

    <?php if ($v['sound']): ?>
        <span class="icon-sound"><img alt="icon" data-bbox="362 111 378 127"/></span>
    <?php endif; ?>
</div>
</div>

<?php endforeach; ?>

```

Este bloque explica de dónde sale cada control. Todos los marcos reciben un botón . El foreach interno crea un botón  por cada posición de alt. El botón  sólo aparece cuando sound es true.

31.7 Cómo pasan las rutas de PHP a JavaScript

Código fuente: functions.php → cnc_gallery_parallax() → script de Gallery iniciado alrededor de la línea 8514; cada control localiza su .frame-container y reconstruye altSources desde dataset.alt. Éste es el puente entre los datos que imprimió PHP y la lógica del navegador.

PHP y JavaScript se conectan a través de los atributos data-* del frame-container. PHP no crea los videos alternativos: sólo deja sus URLs serializadas en data-alt. Cuando JavaScript inicializa los controles, recupera ese atributo y vuelve a convertirlo en un array.

```

const container = ctrl.closest('.frame-container');

const altSources = JSON.parse(container.dataset.alt);
const altHasSound = parseInt(container.dataset.sound);

```

Cada botón  contiene únicamente un índice numérico en data-alt. Por ejemplo, data-alt="0" significa usar altSources[0], data-alt="1" significa usar altSources[1] y así sucesivamente. El botón no necesita repetir la URL completa porque la lista ya quedó guardada en el frame-container.

```

<span class="icon-video" data-alt="0"><img alt="icon" data-bbox="372 491 388 507"/></span>
<span class="icon-video" data-alt="1"><img alt="icon" data-bbox="372 508 388 524"/></span>

JavaScript:
idx = 0 → altSources[0]
idx = 1 → altSources[1]

```

31.8 Qué se carga al abrir la página y qué se crea sólo al hacer clic

Código fuente: functions.php → cnc_gallery_parallax() → JavaScript de .video-controls alrededor de las líneas 8580-8643. altCache vive dentro de la inicialización de cada control; los <video class="vid-alt"> se crean sólo cuando el usuario pide una variante.

Al abrir Gallery ya existen los 12 elementos vid-main porque PHP los imprimió en el HTML. Todos usan autoplay, muted, loop, playsinline y preload="auto". Por eso el navegador puede comenzar a preparar los videos principales desde la carga inicial.

Las variantes alt funcionan de otra manera. No hay 26 reproductores alternativos creados desde el inicio. Sólo existen sus URLs dentro de data-alt. El elemento <video class="vid-alt"> se crea cuando el visitante pulsa el botón  correspondiente.

```

const altCache = {};

let v = altCache[idx];

if (!v) {
    v = document.createElement('video');
    v.src = altSources[idx];
    v.loop = true;
    v.playsInline = true;
    v.classList.add('vid-alt');

    wrap.appendChild(v);
}

```

```
altCache[idx] = v;
}
```

altCache conserva el reproductor que ya fue creado. Si el usuario vuelve más tarde a la misma variante, JavaScript reutiliza ese elemento en lugar de crear otro.

31.9 Loader inicial y espera de loadeddata

Código fuente: functions.php → cnc_gallery_parallax() → bloque “Loader inicial” aproximadamente entre las líneas 8546-8568; las variantes crean un loader equivalente alrededor de las líneas 8628-8640. Ambos caminos esperan readyState/loadeddata antes de revelar el video.

Cada video principal comienza oculto mediante CSS. JavaScript agrega un loader encima del video-wrap y espera a que el navegador tenga datos suficientes para mostrar el MP4. La clase del loader viene de PHP: video-loader gdi o video-loader nod.

```
const loader = document.createElement('div');
loader.className = '<?php echo $loader_class; ?>';
loader.innerHTML =
  '<div>Loading...</div><div class="spinner"></div>';

wrap.appendChild(loader);

function showMain() {
  loader.style.opacity = 0;
  setTimeout(() => loader.remove(), 500);
  main.classList.add('visible');
}

if (main.readyState >= 2) {
  showMain();
} else {
  main.addEventListener('loadeddata', showMain, { once: true });
}
```

readyState >= 2 significa que el navegador ya posee datos del recurso suficientes para mostrar el frame actual. Las variantes alt utilizan el mismo criterio: si todavía no alcanzaron ese estado, se crea un loader temporal y finalize() se ejecuta después de loadeddata.

31.10 Cómo funciona cada botón

Código fuente: functions.php → cnc_gallery_parallax() → showVideo() desde la línea aproximada 8570 y listeners de .icon-eye/.icon-video/.icon-sound entre 8595-8652. Todos actúan sobre el marco ya creado; no generan una petición WordPress nueva.

Botón  — volver al video principal. Si había una variante activa, la pausa y la oculta. Después showVideo() muestra vid-main, reinicia currentTime a 0 y comienza nuevamente la reproducción.

```
function showVideo(wrap, video) {
  wrap.querySelectorAll('video').forEach(v => {
    v.classList.remove('visible');
    v.pause();
  });

  video.classList.add('visible');
  video.currentTime = 0;
  video.play();
}

ctrl.querySelector('.icon-eye').addEventListener('click', () => {
  if (currentAlt) {
    currentAlt.pause();
    currentAlt.classList.remove('visible');
  }

  showVideo(wrap, main);
  setCurrent(main);
});
```

Botón  — abrir una variante. El botón lee su índice data-alt. Si altCache ya contiene ese índice, reutiliza el reproductor. Si no existe, crea un nuevo <video>, le asigna la URL correspondiente de altSources y lo guarda en la caché.

```
ctrl.querySelectorAll('.icon-video').forEach(icon => {
  const idx = parseInt(icon.dataset.alt);

  icon.addEventListener('click', () => {
    let v = altCache[idx];

    if (!v) {
      v = document.createElement('video');
      v.src = altSources[idx];
      v.loop = true;
      v.playsInline = true;
      v.muted = !altHasSound;
      v.classList.add('vid-alt');

      wrap.appendChild(v);
      altCache[idx] = v;
    }

    // si ya está cargado → finalize()
    // si no → loader + loadeddata → finalize()
  });
});
```

Botón  /  — controlar el video visible. El botón existe sólo cuando sound es true. Si hay una variante activa actúa sobre currentAlt; si no, actúa sobre main. Al activar sonido, muteAllGlobal() silencia todos los demás videos de la Galería.

```
soundBtn.addEventListener('click', () => {
  const video = currentAlt || main;

  video.muted = !video.muted;

  if (!video.muted) {
    muteAllGlobal(video);
  }

  updateAllSoundIcons();
});
```

31.11 Un solo video con sonido en toda la Galería

El control de audio es global, no sólo local al marco. muteAllGlobal() recorre todos los videos que existen dentro de .frame-container y aplica muted = true a todos excepto al que acaba de habilitarse. updateAllSoundIcons() revisa después cada marco para mostrar  o  según el video visible.

```
function muteAllGlobal(exceptVideo) {
  document.querySelectorAll('.frame-container video').forEach(v => {
    if (v !== exceptVideo) {
      v.muted = true;
    }
  });

  updateAllSoundIcons();
}
```

31.11.1 Estado inicial del icono de sonido

Existe otra diferencia entre el estado real del reproductor y el icono que se ve al cargar Gallery. Cuando sound es true, PHP imprime el botón con el texto inicial ; sin embargo, el video principal se crea con el atributo muted. showMain() sólo elimina el loader y agrega la clase visible: no ejecuta updateAllSoundIcons().

```

<!-- PHP imprime el botón así cuando sound = true -->
<span class="icon-sound" title="Activar/Desactivar sonido"><img alt="icono de sonido" data-bbox="535 75 555 85"/></span>

<!-- pero el main nace muteado -->
<video class="vid-main" ... autoplay muted loop playsinline></video>

function showMain(){
  loader.style.opacity = 0;
  setTimeout(()=>loader.remove(), 500);
  main.classList.add('visible');
  // aquí no se llama a updateAllSoundIcons()
}

```

En consecuencia, al abrir por primera vez un marco con `sound = true`, el botón muestra  aunque el video principal esté realmente silenciado. El texto del icono se sincroniza recién cuando alguna interacción posterior llama a `updateAllSoundIcons()`, por ejemplo al cambiar de video o utilizar el propio control de sonido.

31.12 Movimiento visual: flotación del marco y parallax del video

Código fuente: `functions.php` → `cnc_gallery_parallax()` → ángulo `--rot-angle` alrededor de las líneas 8541-8544 y bloque `Parallax` desde aproximadamente la 8656 hasta la 8696. `requestAnimationFrame()` sólo anima el `.vid-main` visible en el navegador.

Gallery combina dos efectos diferentes. El primero afecta al `frame-container` completo: JavaScript genera un ángulo aleatorio entre aproximadamente -6° y $+6^\circ$ y lo guarda en la variable CSS `--rot-angle`. Las hojas de estilo utilizan ese valor dentro de la animación `gdiFloat` para producir la flotación del marco.

```

const angle = (Math.random() * 12 - 6).toFixed(2) + 'deg';
container.style.setProperty('--rot-angle', angle);

```

El segundo efecto es el parallax. Se aplica únicamente al video principal `.vid-main`. La posición del cursor dentro de `video-wrap` se transforma en un desplazamiento objetivo y `requestAnimationFrame()` aproxima gradualmente la posición actual a ese objetivo.

```

const maxX = 18, maxY = 12, ease = 0.12;

const onMove = e => {
  const r = videoWrap.getBoundingClientRect();

  tX = -(((e.clientX - r.left) / r.width) - 0.5) * maxX;
  tY = -(((e.clientY - r.top) / r.height) - 0.5) * maxY;
};

function step() {
  cX += (tX - cX) * ease;
  cY += (tY - cY) * ease;

  main.style.transform =
    `translate3d(calc(-50% + ${cX}px),
      calc(-50% + ${cY}px), 0)`;

  raf = requestAnimationFrame(step);
}

```

Las variantes `alt` no reciben este parallax porque el código selecciona expresamente `.vid-main`. La grilla también es responsive: las hojas de estilo muestran dos columnas en escritorio y una sola columna por debajo de 900 px.

31.13 Integración con el menú principal

Código fuente: `wp-content/themes/blocksy-child/header.php` → `stopAllOtherAudio()` alrededor de la línea 514 y `restorePreviousAudio()` cerca de la 533. Esta lógica pertenece al menú, no a `cnc_gallery_parallax()`: coordina los medios de Gallery cuando el HUD se abre o se cierra.

Gallery también se conecta con `header.php`. Esta integración no trabaja solamente con videos. Al abrir el menú principal sobre `/gallery`, `stopAllOtherAudio()` revisa todos los elementos `<audio>` y `<video>` que están fuera de `#zajOverlay`. Los audios que estaban reproduciéndose se marcan con `data-was-playing` y se pausan; los videos que estaban con sonido se marcan con `data-was-unmuted` y se silencian. Al cerrar el menú, `restorePreviousAudio()` restaura únicamente los estados que habían quedado registrados.

```
function stopAllOtherAudio() {
    document.querySelectorAll('audio').forEach(a => {
        if (!a.closest('#zajOverlay')) {
            a.dataset.wasPlaying = !a.paused ? 'true' : 'false';
            a.pause();
        }
    });

    document.querySelectorAll('video').forEach(v => {
        if (!v.closest('#zajOverlay')) {
            v.dataset.wasUnmuted = !v.muted ? 'true' : 'false';
            v.muted = true;
        }
    });
}

function restorePreviousAudio() {
    document.querySelectorAll('audio').forEach(a => {
        if (!a.closest('#zajOverlay') && a.dataset.wasPlaying === 'true') {
            a.play().catch(() => {});
        }
    });

    document.querySelectorAll('video').forEach(v => {
        if (!v.closest('#zajOverlay') && v.dataset.wasUnmuted === 'true') {
            v.muted = false;
        }
    });
}
```

De esta forma, el overlay del menú no se mezcla con audios o videos que ya estaban activos fuera del propio menú. Al cerrarlo, header.php intenta recuperar el estado previo en lugar de dejar permanentemente pausados o silenciados esos recursos.

31.14 Qué ocurre si un archivo no carga

Código fuente: functions.php → cnc_gallery_parallax() → el script termina con el listener contextmenu alrededor de la línea 8698. En los bloques de carga no hay un listener error equivalente ni un fallback propio para una URL de Gallery que falle.

El código de Gallery espera readyState >= 2 o el evento loadeddata, pero no registra un listener específico para error ni define un MP4 de respaldo. Por eso, si una URL de \$videos no existe, el mapeo /videos/... está mal configurado o el archivo no puede descargarse, el loader de ese recurso puede permanecer visible.

El bloqueo de clic derecho tampoco constituye un sistema de protección de archivos. Gallery cancela contextmenu como decisión de interfaz, pero un MP4 que ya fue entregado al navegador sigue siendo un recurso accesible desde las herramientas de red o desde su URL.

```
document.addEventListener(
    'contextmenu',
    e => e.preventDefault()
);
```

31.15 Comportamiento actual del campo sound

Código fuente: functions.php → cnc_gallery_parallax() → creación de variantes alrededor de las líneas 8609-8640 y control de sonido alrededor de 8645-8652. La inconsistencia surge dentro del mismo flujo: v.muted nace de altHasSound, pero finalize() fuerza después muted = false.

El comportamiento actual de sound tiene una inconsistencia más concreta de lo que sugiere el nombre del campo. Cuando sound = false, PHP no imprime el botón .icon-sound y JavaScript crea inicialmente la variante con muted = true. Sin embargo, finalize() cambia inmediatamente esa variante a muted = false antes de mostrarla. Después setCurrent(v) detecta que se trata de una alternativa y ejecuta muteAllGlobal(v), por lo que esa variante queda como el único video no silenciado de la Galería.

```
<!-- Con sound = false, PHP NO imprime este control -->
<?php if ($v['sound']): ?>
```

```

    <span class="icon-sound"><img alt="sound icon" data-bbox="300 58 315 70"/></span>
<?php endif; ?>

// La variante nace muteada cuando altHasSound = 0:
v.muted = !altHasSound;

function finalize(){
    v.muted = false;
    showVideo(wrap, v);
    setCurrent(v);
    updateAllSoundIcons();
}

function setCurrent(video){
    currentAlt = (video === main) ? null : video;
    if (currentAlt) muteAllGlobal(video);
    updateAllSoundIcons();
}

```

La consecuencia es importante: si una variante perteneciente a un marco con sound = false contiene una pista de audio, el código actual puede reproducirla con sonido y silenciar los demás videos, pero ese marco no tiene botón  porque PHP no lo generó. El usuario, por lo tanto, no dispone del control propio de Gallery para volver a silenciar esa variante. Si el MP4 no contiene pista de audio, el problema no se percibe, pero la lógica JavaScript sigue siendo la misma. Esta documentación describe el comportamiento existente; no lo presenta como funcionamiento deseado.

31.16 Ejemplo completo: NOD, marco 2

El segundo marco NOD permite seguir el flujo completo porque tiene un video principal, dos variantes y sound = true.

```

[
  'main' => '/videos/nod-gallery/video2_main.mp4',
  'alt' => [
    '/videos/nod-gallery/video2_alt_one.mp4',
    '/videos/nod-gallery/video2_alt_two.mp4'
  ],
  'sound' => true
]

```

PHP genera un frame-container con un , dos botones  y un botón . Al abrir Gallery, video2_main.mp4 ya existe como vid-main y comienza muteado. Si el usuario pulsa el primer , JavaScript lee data-alt="0", busca altCache[0] y, como todavía no existe, crea un vid-alt con video2_alt_one.mp4. Cuando loadeddata confirma que el archivo está listo, lo muestra y lo guarda en la caché. Si se activa su sonido, muteAllGlobal() silencia los videos de los demás marcos. Al pulsar , la variante se pausa y el marco vuelve a video2_main.mp4 desde el segundo 0.

```

NOD video2
↓
main = video2_main.mp4
alt[0] = video2_alt_one.mp4
alt[1] = video2_alt_two.mp4
sound = true
↓
PHP genera:    
↓
clic en primer 
↓
idx = 0
↓
altCache[0] no existe
↓
crear vid-alt + cargar video2_alt_one.mp4
↓
loadeddata
↓
mostrar variante + guardar en altCache
↓

```

```

clic ⓘ
↓
volver a video2_main.mp4

```

31.17 Ejemplo guiado: Alex utiliza Gallery

La sección anterior ya siguió técnicamente el marco NOD 2. Este ejemplo recorre el mismo mecanismo desde la experiencia de una persona, sin reemplazar ninguna de las reglas, inconsistencias o detalles de implementación documentados en 31.1-31.16.

Alex entra en Gallery con faction=nod. Antes de que vea la pantalla, PHP lee la facción, `cnc_gallery_parallax()` selecciona el array NOD y genera los doce frame-container. Los videos principales ya forman parte del HTML; las variantes alternativas todavía existen sólo como URLs dentro de data-alt.

Alex mueve el cursor sobre el segundo marco. El frame conserva su flotación y el video principal `.vid-main` recibe el parallax calculado a partir de la posición del puntero. Las variantes alt no reciben ese efecto porque el código de parallax selecciona expresamente el video principal.

Después Alex pulsa el primer botón de variante. JavaScript lee `data-alt="0"` y consulta `altCache[0]`. Como es la primera vez, no encuentra un reproductor existente: crea un nuevo `<video class="vid-alt">`, asigna `video2_alt_one.mp4`, lo inserta dentro del marco y espera `loadeddata`. Cuando hay datos suficientes, muestra la variante y guarda ese elemento en `altCache`.

Alex vuelve al video principal con el botón de regreso y, más tarde, pulsa otra vez la misma variante. Esta vez `altCache[0]` ya contiene el `<video>` creado anteriormente, por lo que JavaScript lo reutiliza en lugar de construir un segundo reproductor. Si Alex activa sonido en un marco que dispone del control, `muteAllGlobal()` silencia los demás videos para evitar varias pistas simultáneas.

```

Alex abre Gallery con NOD
↓
PHP genera 12 marcos desde $videos
↓
Alex mueve el cursor → parallax sobre vid-main
↓
Alex pulsa una variante
↓
leer data-alt
↓
¿existe altCache[idx]?
├ no → crear vid-alt + esperar loadeddata + guardar
└ sí → reutilizar el reproductor
↓
mostrar variante
↓
botón de regreso → video principal desde el inicio

```

Si Alex abre MENU mientras Gallery tiene audio o videos activos, `header.php` registra qué recursos estaban reproduciéndose o desmuteados, los pausa/silencia fuera del overlay y, al cerrar el menú, intenta restaurar sólo los estados que había guardado. De esta forma el HUD no mezcla su audio con el contenido que Alex estaba viendo en Gallery.

Relación con la implementación existente

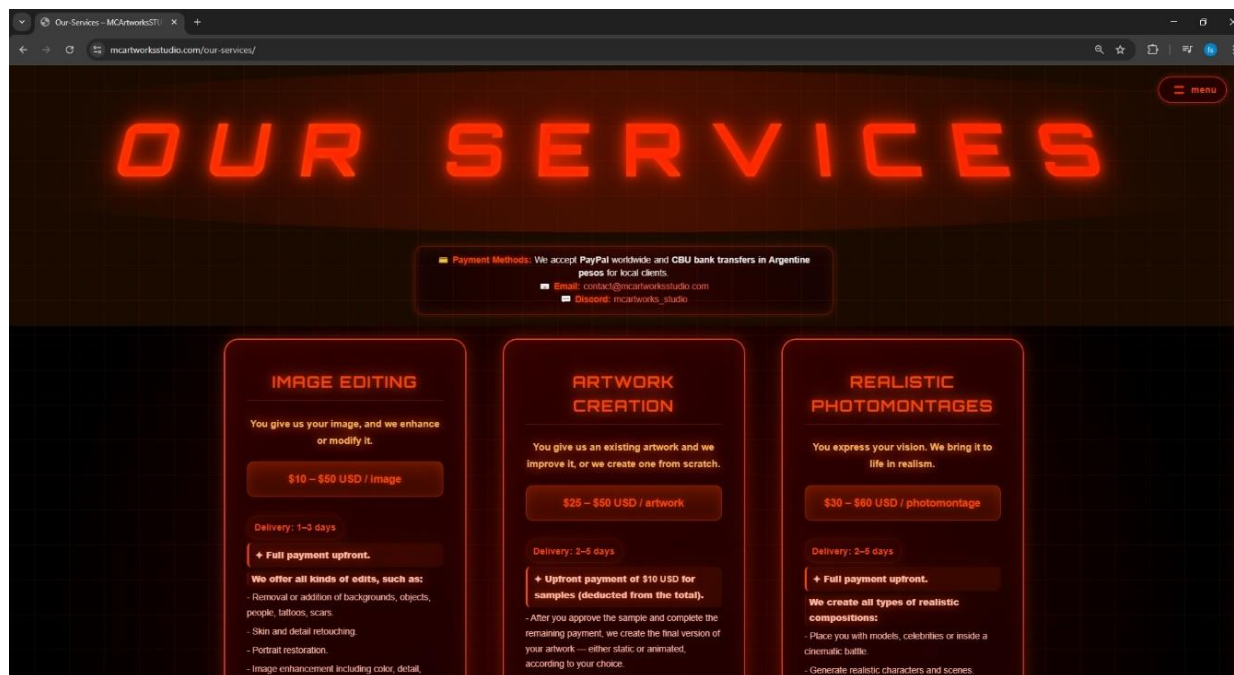
Este recorrido de Alex es una lectura pedagógica de la lógica ya documentada. No sustituye los apartados sobre `altCache`, `loaders`, `sound`, `parallax`, fallos de carga o integración con `header.php`; todos esos comportamientos siguen siendo los que definen el funcionamiento real de Gallery.

32. Sección de servicios (Our Services)

Our Services es una pantalla informativa generada directamente por el tema hijo. No consulta `WC_Product` ni crea un flujo comercial de WooCommerce: PHP imprime el panel de contacto, tres tarjetas de servicio y un bloque final de condiciones, mientras las hojas de estilo de la facción determinan su apariencia.

Ubicación en el proyecto: `wp-content/themes/blocksy-child/functions.php` → `add_action('wp_head', 'mostrar_servicios_personalizados')` en la línea 8713 → `mostrar_servicios_personalizados()` desde la 8718 hasta la 8809. Los estilos de `.your-services`, `.your-services-card`, `.payment-info-panel` e `.info-panel` están definidos en `gdi-faction.css` / `nod-faction.css` y sus versiones minificadas.

Detalle de implementación actual: el callback está registrado en wp_head, pero imprime HTML visible del body. La documentación describe esa arquitectura tal como existe; una reorganización futura podría mover ese contenido a un punto semánticamente más apropiado del body.



Our Services NOD: panel de contacto, tres tarjetas y apariencia aplicada por la facción.

32.1 Activación: de la URL al callback

```
add_action('wp_head', 'mostrar_servicios_personalizados');

// =====
// Función que agrega contenido HTML personalizado en el <head> para la página "Our Services".
// =====
function mostrar_servicios_personalizados() {

    if (is_page('our-services')) : ?>

        <style>
```

WordPress carga functions.php y registra el callback. En una petición normal, wp_head dispara mostrar_servicios_personalizados(); la función comprueba is_page('our-services'). Si la página actual no coincide, el bloque no imprime la sección. Si coincide, comienza a generar CSS inline y HTML.

```
/our-services/
↓
WordPress resuelve la página
↓
wp_head
↓
mostrar_servicios_personalizados()
↓
is_page('our-services') = true
↓
PHP imprime contenido visible
```

32.2 Panel de contacto y medios de pago

Código fuente real: functions.php → mostrar_servicios_personalizados() → líneas 8736-8749.

```
<div class="custom-shop-title">OUR SERVICES</div>

<div class="payment-info-panel">
  <p>
    <span class="highlight">Payment Methods:</span> We accept <b>PayPal</b> worldwide and
```

```

        <b>CBU bank transfers in Argentine pesos</b> for local clients.
    <br>
    ✉ <span class="highlight">Email:</span>
    <a href="mailto:contact@mcartworksstudio.com">contact@mcartworksstudio.com</a>
    <br>
    💬 <span class="highlight">Discord:</span>
    <a href="https://discordapp.com/users/1432397605471649905" target="_blank">mcartworks_studio</a>
</p>
</div>

```

El panel es contenido estático del tema: PayPal CBU se muestran como medios informativos; el enlace de email usa mailto: y Discord abre una URL externa. Nada de este bloque llama al gateway manual_pay ni genera un WC_Order.

32.3 Las tres tarjetas salen del HTML del tema

Código fuente real: functions.php → .your-services → líneas 8751-8795. No hay consulta a WooCommerce ni a ACF para obtener estas tarjetas.

```

<section class="your-services">
    <div class="your-services-card">
        <h3>IMAGE EDITING</h3>
        <p>You give us your image, and we enhance or modify it.</p>
        <div class="price-box">$10 - $50 USD / image</div>
        <p><span class="highlight-delivery">Delivery: 1-3 days</span></p>
        <p class="highlight-upfront">♦ Full payment upfront.</p>
        <p class="highlight-makes">We offer all kinds of edits, such as:</p>
        <p>- Removal or addition of backgrounds, objects, people, tattoos, scars.</p>
        <p>- Skin and detail retouching.</p>
        <p>- Portrait restoration.</p>
        <p>- Image enhancement including color, detail, sharpness.</p>
        <p>- Gender, face, hair, or clothing changes.</p>
        <p>📺 Want your edited image turned into a video?</p>
        <p>We can create up to three unique short videos from it.</p>
    </div>

    <div class="your-services-card">
        <h3>ARTWORK CREATION</h3>
        <p>You give us an existing artwork and we improve it, or we create one from scratch.</p>
        <div class="price-box">$25 - $50 USD / artwork</div>
        <p><span class="highlight-delivery">Delivery: 2-5 days</span></p>
        <p class="highlight-upfront">♦ Upfront payment of <b>$10 USD</b> for samples (deducted from
the total).</p>
        <p>- After you approve the sample and complete the remaining payment, we create the final
version of your artwork – either static or animated, according to your choice.</p>
        <p>- Artworks can be made for posters, social media, wallpapers, Steam, and more.</p>
        <p>- If you need us to make any adjustments or corrections, we will do so.</p>
        <p>- If it's for Steam, we deliver in your preferred format: Full, Frame, or Workshop.</p>
    </div>

```

La segunda y la tercera tarjeta continúan con la misma estructura: título, descripción, price-box, plazo y condiciones. Los valores visibles están escritos literalmente en functions.php, por lo que cambiar un precio o texto requiere modificar ese archivo o migrar esos datos a una configuración administrable.

Servicio	Precio escrito en el código	Entrega escrita en el código
Image Editing	USD 10–50 / imagen	1–3 días
Artwork Creation	USD 25–50 / artwork	2–5 días
Realistic Photomontages	USD 30–60 / photomontage	2–5 días

32.4 Important Information y dependencia de Google Fonts

```

<div class="section-divider"></div>

<div class="info-panel">

```

```

<h4>Important Information</h4>
<p>⚠ We do not work with explicit content (nudity or extreme gore).</p>
<p>💰 Prices may vary depending on project complexity.</p>
<p>🕒 Delivery times may vary according to complexity and workload.</p>
</div>

<link href="https://fonts.googleapis.com/css2?family=Orbitron:wght@400;700&display=swap"
rel="stylesheet">

<?php endif;

```

El bloque final informa restricciones de contenido y aclara que precios y tiempos pueden variar. Inmediatamente después se carga Orbitron desde Google Fonts. Esa dependencia es visual: si la fuente externa no está disponible, el contenido sigue existiendo, pero el navegador utilizará una tipografía de respaldo.

32.5 Apariencia NOD/GDI: misma estructura, distinto CSS

La estructura HTML no cambia entre facciones. La diferencia visual proviene de los CSS de NOD y GDI, que definen los mismos selectores con paletas, sombras, fondos y bordes distintos. Entre los selectores principales están .your-services, .your-services-card, .payment-info-panel, .info-panel, .price-box y .section-divider.

```

mismo HTML de Our Services
├─ faction = gdi → gdi-faction-min.css
├─ faction = nod → nod-faction-min.css
↓
misma información, identidad visual diferente

```

32.6 Ejemplo guiado: Alex consulta un servicio

Alex abre /our-services/ con faction=nod. WordPress llega a wp_head, la función confirma el slug y PHP imprime el panel de contacto y las tres tarjetas. El CSS NOD convierte esa estructura en la interfaz roja de la captura. Si Alex pulsa el correo o Discord, sale hacia un canal de contacto; no se agrega ningún artículo al carrito y no se crea un pedido de WooCommerce.

No confundir: Our Services documenta servicios ofrecidos por el estudio, pero sus tarjetas no son productos WooCommerce. Shop/Bundles y Our Services son dos mecanismos distintos.

33. JavaScript y cambios dinámicos en la página

Esta sección ya no funciona como un glosario de APIs. Sigue cuatro mecanismos reales del proyecto y muestra cuándo actúan: DOMContentLoaded para trabajar con un DOM inicial existente, IntersectionObserver para activar previews al acercarse al viewport, MutationObserver para reaccionar a HTML que un plugin vuelve a crear y eventos de WooCommerce/jQuery para continuar después de una operación AJAX. requestAnimationFrame() se utiliza cuando el cambio está ligado al ciclo de pintura del navegador.

Frontera importante: PHP puede imprimir un <script>, pero no ejecuta document.addEventListener(), MutationObserver ni requestAnimationFrame(). Ese JavaScript viaja en la respuesta HTTP y se ejecuta después en el navegador.

33.1 DOMContentLoaded: Checkout espera que existan los campos

Código fuente real: functions.php → callback anónimo de wp_footer condicionado por is_checkout() → líneas 8259-8271.

```

add_action('wp_footer', function() {
    if (is_checkout()) {
        ?>
        <script>
        document.addEventListener('DOMContentLoaded', function(){
            const art = document.querySelector('#artwork_name');
            if(art) art.setAttribute('maxlength', '32');

            const name = document.querySelector('#billing_first_name');
            if(name) name.setAttribute('maxlength', '50');

            const mail = document.querySelector('#billing_email');
            if(mail) mail.setAttribute('maxlength', '100');
        });
        </script>
    }
});

```

El listener se registra cuando el navegador interpreta el script. La función interna se ejecuta cuando el DOM inicial ya fue construido. Sólo entonces `querySelector()` busca los inputs y `setAttribute()` agrega límites visibles de caracteres. Estas restricciones mejoran la interfaz, pero la validación PHP del servidor sigue siendo la autoridad final.

```
PHP imprime <script>
↓ respuesta HTTP
navegador crea DOM
↓ DOMContentLoaded
querySelector() encuentra inputs
↓
maxlength cambia el HTML en memoria
```

33.2 IntersectionObserver: Shop activa videos al acercarse

Código fuente `real: functions.php → video_lazy_loop_shop()` → líneas 2979-3033. El observer principal está en 3015-3022 y utiliza `rootMargin: 200px`.

```
const observer = new IntersectionObserver((entries, obs) => {
  entries.forEach(entry => {
    if(entry.isIntersecting){
      reproducirVideo(entry.target, false); // scroll: sin delay
      obs.unobserve(entry.target);
    }
  });
}, { rootMargin: "200px" });

videos.forEach(video => {
  const rect = video.getBoundingClientRect();
  const inViewport = rect.top < window.innerHeight && rect.bottom > 0;
  if(inViewport){
    reproducirVideo(video, true); // primeros visibles: 850ms
  } else {
    observer.observe(video); // scroll: inmediato
  }
})
```

`entries` contiene los elementos cuya relación con el viewport cambió. Cuando `entry.isIntersecting` es verdadero, `entry.target` es el video correspondiente. `reproducirVideo()` copia `data-src` a `src` si todavía no existe y comienza la reproducción. `obs.unobserve()` evita seguir observando ese elemento después de activarlo.

El `rootMargin` de 200 px hace que la activación pueda ocurrir antes de que el video esté completamente visible. Así el navegador puede empezar a preparar el MP4 mientras Alex se acerca con el scroll.

33.3 MutationObserver: HUSKY reemplaza nodos después de AJAX

Código fuente `real: functions.php → agregar_boton_lupa_buscar()` → `MutationObserver` alrededor de las líneas 1591-1615.

```
const observer = new MutationObserver((mutations) => {
  for (const m of mutations) {
    // Revisar nodes añadidos
    for (const node of m.addedNodes) {
      if (!(node instanceof HTMLElement)) continue;
      if (node.matches && node.matches('.woof_husky_txt-input')) {
        agregarLupaSiInputListoEl(node);
        return;
      }
      const found = node.querySelector && node.querySelector('.woof_husky_txt-input');
      if (found) {
        agregarLupaSiInputListoEl(found);
        return;
      }
    }
  }
  // Opcional: si detecta que #btn_search fue removido, reintentar re-agregar
  for (const node of m.removedNodes) {
    if (!(node instanceof HTMLElement)) continue;
    if (node.id === 'btn_search') {
      // pequeño timeout para dar tiempo a que Husky re-renderice
      setTimeout(() => {
        const input = findInput();
        if (input) agregarLupaSiInputListoEl(input);
      }, 100);
    }
  }
})
```

```
    }, 20);
  }
```

HUSKY puede reconstruir su input después de aplicar filtros. El observer no consulta al servidor: observa mutaciones del DOM que ya ocurrieron. Si entre los `addedNodes` aparece `.woof_husky_txt-input`, el tema vuelve a ejecutar `agregarLupaSiInputListoEl()`. Si detecta que `#btn_search` fue removido, espera brevemente y busca el nuevo input generado por el plugin.

```
HUSKY hace AJAX
↓
reemplaza parte del DOM
↓
MutationObserver recibe mutations
↓
encuentra el nuevo input
↓
MC Artworks vuelve a agregar la lupa
```

33.4 requestAnimationFrame(): Gallery sincroniza el parallax con la pintura

Código fuente `real: functions.php → cnc_gallery_parallax() → bloque Parallax`, aproximadamente líneas 8662-8695.

```
const maxX = 18, maxY = 12, ease = 0.12;
let tX=0,tY=0,cX=0,cY=0,raf=null,inSide=false;

const onMove = e => {
  const r = videoWrap.getBoundingClientRect();
  tX = -(((e.clientX-r.left)/r.width)-0.5)*maxX;
  tY = -(((e.clientY-r.top)/r.height)-0.5)*maxY;
};

function step(){
  cX += (tX-cX)*ease;
  cY += (tY-cY)*ease;
  main.style.transform = `translate3d(calc(-50% + ${cX}px), calc(-50% + ${cY}px),0)`;
  if(!inSide && Math.abs(cX)<0.25 && Math.abs(cY)<0.25){
    main.style.transform='translate3d(-50%,-50%,0)';
    cancelAnimationFrame(raf);
    raf=null;
    cX=0;cY=0;
    return;
  }
  raf=requestAnimationFrame(step);
}

videoWrap.addEventListener('mouseenter',()=>{
  inSide=true;
  videoWrap.addEventListener('mousemove',onMove);
  if(!raf) step();
});

videoWrap.addEventListener('mouseleave',()=>{
  inSide=false;
  videoWrap.removeEventListener('mousemove',onMove);
  tX=0;tY=0;
});
```

`onMove()` no mueve directamente el video: calcula una posición objetivo `tX/tY`. `step()` aproxima `cX/cY` hacia ese objetivo mediante `ease` y aplica `translate3d()`. Al final solicita el siguiente ciclo con `requestAnimationFrame(step)`. Cuando el cursor sale y la posición vuelve casi a cero, `cancelAnimationFrame()` detiene el ciclo.

33.5 Evento `added_to_cart`: continuar después del AJAX de WooCommerce

Código fuente `real: functions.php → script de sincronización de Shop → listener added_to_cart` alrededor de las líneas 3664-3685.

```
$(document.body).on('added_to_cart', function(event, fragments, cart_hash, $button) {
  if (!$button || !$button.length) return;

  const pid = $button.data('product_id');
  if (!pid) return;
```

```

// Oculta el botón 'Añadir al carrito' que se acaba de usar
$button.hide();

// Elimina instancias duplicadas de 'Ver carrito'
$button.siblings('a.added_to_cart.wc-forward, a.button.wc-forward').remove();

// Inserta solo un botón 'Ver carrito' limpio
const verCarrito = $('<a>', {
  href: wc_cart_url,
  class: 'added_to_cart wc-forward',
  'data-product_id': pid,
  text: 'Ver carrito'
});

$button.after(verCarrito);
});

```

added_to_cart no es un evento DOM nativo: WooCommerce lo dispara después de agregar el producto mediante AJAX. El callback recibe el botón que originó la operación, obtiene product_id, oculta Add to Cart, elimina botones duplicados y crea un enlace View Cart. De esta forma el tema reacciona después de que WooCommerce ya confirmó la operación de carrito.

Hallazgo relacionado: en el código activo, wc_cart_url se localiza como un objeto JavaScript con la propiedad url. Existe además un bloque anterior que la habría localizado como string, pero está comentado y no se ejecuta. El problema real es que algunas rutinas posteriores usan wc_cart_url directamente como href en lugar de wc_cart_url.url. La inconsistencia se consolida en la sección 39.

33.6 Cuándo usar cada mecanismo en este proyecto

Mecanismo	Qué espera	Caso real de MC Artworks
DOMContentLoaded	DOM inicial creado	Checkout aplica maxlength a campos
IntersectionObserver	cambio de visibilidad/proximidad	Shop activa previews desde data-src
MutationObserver	cambio en la estructura del DOM	HUSKY recrea el buscador tras AJAX
added_to_cart	fin de una operación AJAX WooCommerce	Shop cambia Add to Cart por View Cart
requestAnimationFrame	siguiente ciclo de pintura	Gallery interpola parallax

33.7 Ejemplo guiado: Alex atraviesa varios mecanismos

```

Alex abre Shop
↓ DOMContentLoaded
se inicializa la lógica
↓ scroll
IntersectionObserver activa un preview
↓ aplica filtro HUSKY
AJAX reemplaza DOM
↓ MutationObserver repara la lupa
↓ Add to Cart
WooCommerce dispara added_to_cart
↓
se muestra View Cart
↓ abre Gallery
requestAnimationFrame anima el parallax

```

34. Adaptación entre navegadores

La compatibilidad multimedia no se resuelve con una única rama. La Intro detecta Firefox y navegadores Chromium, elige rutas HLS distintas y configura HLS.js de manera diferente. Shop, por otro lado, contiene una corrección específica para Chrome/Opera que toma snapshots de video antes de una navegación para evitar cuadros negros.

34.1 Detección utilizada por la Intro

Código fuente real: wp-content/themes/blocksy-child/hud-intro.php → script de sincronización HLS alrededor de la línea 1032. El fuente está minificado en una sola línea; aquí se muestra reindentado sin cambiar su lógica.

```
const isFirefox = typeof InstallTrigger !== 'undefined';
const isChromium = !!window.chrome;

const videos = isFirefox
  ? {
    GDIvideo: '/videos/gdi-intro-firefox/GDI-INTRO.m3u8',
    NODvideo: '/videos/nod-intro-firefox/NOD-INTRO.m3u8'
  }
  : {
    GDIvideo: '/wp-content/uploads/videos/GDI-INTRO/GDI-INTRO.m3u8',
    NODvideo: '/wp-content/uploads/videos/NOD-INTRO/NOD-INTRO.m3u8'
  };
```

La detección de navegador decide primero qué rutas utilizar. Firefox apunta a /videos/...-firefox/, mientras el resto de esa rama utiliza rutas dentro de wp-content/uploads/videos. Esta es detección por navegador; más adelante el código también hace detección de capacidad mediante Hls.isSupported() y canPlayType().

34.2 HLS.js frente a soporte HLS nativo

```
if (window.Hls && Hls.isSupported()) {
  const hls = new Hls(/* configuración según navegador */);
  hls.loadSource(src);
  hls.attachMedia(vid);
  hls.on(Hls.Events.MANIFEST_PARSED, () => resolve());
} else if (vid.canPlayType('application/vnd.apple.mpegurl')) {
  vid.src = src;
  vid.addEventListener('canplaythrough', ready);
} else {
  resolve();
}
```

Hls.isSupported() pregunta si HLS.js puede trabajar en ese navegador. Si puede, loadSource() carga el manifiesto .m3u8 y attachMedia() lo conecta con el elemento <video>. Si HLS.js no es el camino disponible pero el elemento video declara soporte nativo para application/vnd.apple.mpegurl, el manifiesto se asigna directamente a src.

34.3 Configuraciones diferentes para Firefox y Chromium

Rama	Configuración HLS.js comprobada	Objetivo observable
Firefox	enableWorker=true · lowLatencyMode=true · backBufferLength=60	Camino específico con mayor back buffer
Chromium	enableWorker=false · lowLatencyMode=false · backBufferLength=1 · maxBufferLength=10 · maxMaxBufferLength=20 · timeouts y retries	Camino más acotado y con timeouts explícitos

Después de cargar los dos feeds, el script espera Promise.all(), lleva ambos videos al tiempo 0, los pausa, espera un requestAnimationFrame() y vuelve a reproducirlos. Un setInterval compara sus currentTime y corrige diferencias superiores a 0,05 segundos. El intervalo es de 1500 ms para Chromium y 2000 ms para la otra rama.

34.4 autoplay, muted y playsinline: una sola explicación

Los videos de la interfaz suelen combinar autoplay, muted y playsinline. muted reduce las restricciones que muchos navegadores aplican al autoplay con sonido; playsinline solicita que el recurso permanezca integrado en la página en lugar de forzar un reproductor fullscreen. Estas banderas no garantizan por sí solas que play() tenga éxito: el código utiliza .play().catch(() => {}) en varios puntos porque el navegador puede rechazar la reproducción.

34.5 Corrección específica de Chrome/Opera en Shop

Código fuente real: functions.php → fix_videos_chrome_shop_universal() → hook wp_footer en 3039; detección y snapshot general desde 3054.

```
function fix_videos_chrome_shop_universal() {
    if (!is_shop()) return;
    ?>
    <script>
    (function(){
        const ua = navigator.userAgent;
        const isChrome = /Chrome/.test(ua) && /Google Inc/.test(navigator.vendor);
        const isOpera = /OPR\//.test(ua);
        if (!(isChrome || isOpera)) return;

        // === Snapshot general (para galería principal) ===
        function snapshotVideo(video){
            try {
                // Ignorar los videos del menú principal
                if (video.classList.contains('zaj-card-video')) return;

                const w = video.offsetWidth || video.clientWidth;
                const h = video.offsetHeight || video.clientHeight;
                const canvas = document.createElement('canvas');
                canvas.width = video.videoWidth || w;
                canvas.height = video.videoHeight || h;
                canvas.style.width = w + 'px';
                canvas.style.height = h + 'px';
                canvas.style.position = 'absolute';
                canvas.style.top = video.offsetTop + 'px';
                canvas.style.left = video.offsetLeft + 'px';
                canvas.style.zIndex = '9999';
                canvas.style.pointerEvents = 'none';
                const ctx = canvas.getContext('2d');
                ctx.drawImage(video, 0, 0, canvas.width, canvas.height);
                const parent = video.parentElement;
                if (getComputedStyle(parent).position === 'static') {
                    parent.style.position = 'relative';
                }
                parent.insertBefore(canvas, video);
                return true;
            }
        }
    })();
}
```

La función sólo continúa en Shop y sólo para Chrome u Opera según userAgent/vendor. snapshotVideo() crea un Canvas del tamaño del video, dibuja el fotograma actual mediante drawImage() y coloca ese Canvas delante del reproductor. Si la captura falla, intenta utilizar el poster como imagen de respaldo.

34.6 Cuándo se toma el snapshot

```
// 1 Enlaces internos (paginación, View Cart, etc.)
document.addEventListener('click', function(e){
    const link = e.target.closest && e.target.closest('a[href]');
    if (!link) return;

    // Ignorar enlaces que abren videos con Fancybox o clics dentro de <video>
    if (link.hasAttribute('data-fancybox') || e.target.closest('.video-loop-link') ||
    e.target.tagName === 'VIDEO') return;

    // Ignorar enlaces de "Add to Cart" o "View Cart"
    if (link.classList.contains('added_to_cart') || link.classList.contains('add_to_cart_button'))
    return;

    if (link.target === '_blank') return;

    const href = link.getAttribute('href');
    if (!href || href.startsWith('#')) return;

    e.preventDefault();
});
```

```
// Tomar snapshot de todo
snapshotAllVideos();
snapshotCartVideos();

setTimeout(() => window.location.href = href, 80);
}, true);

// [2] Formularios (Husky / WOOF / AJAX)
document.addEventListener('submit', function(){
  snapshotAllVideos();
});
```

El código intercepta enlaces internos y submit de formularios. Antes de navegar, snapshotAllVideos() y snapshotCartVideos() congelan visualmente los reproductores y la navegación se retrasa 80 ms. El objetivo es evitar el frame negro que puede aparecer cuando el navegador desmonta el video antes de que la siguiente página se muestre.

34.7 Límites de esta estrategia

- La detección por userAgent/vendor puede necesitar mantenimiento si cambian los navegadores.
- Canvas depende de que el fotograma pueda dibujarse; el código contempla poster como fallback en snapshotVideo().
- HLS.js se carga desde CDN con @latest en distintos bloques del proyecto, por lo que la versión efectiva puede cambiar sin modificar el child theme.
- El resultado buscado es una experiencia equivalente, no ejecutar exactamente el mismo código en todos los navegadores.

34.8 Ejemplo guiado: Alex abre Intro en dos navegadores

```
Alex abre Intro
├─ Firefox
│   └─ rutas /videos/*-firefox/
│       └─ Hls config: worker + low latency + backBuffer 60
├─ Chromium
│   └─ rutas /wp-content/uploads/videos/...
│       └─ Hls config: buffers/timeouts más acotados
↓
Hls.isSupported() o HLS nativo
↓
se cargan GDI + NOD
↓
Promise.all()
↓
reproducción + sincronización
```

35. Recursos multimedia y servicios externos

Esta sección sigue recursos reales desde su fuente hasta el navegador. La regla es siempre la misma: identificar dónde vive el dato o archivo, quién construye la URL, cuándo el navegador la solicita y qué dependencia externa participa.

35.1 Caso A: video_destacado de un producto de Shop

Código fuente real: functions.php → tarjeta audiovisual de Shop → líneas 2902-2924.

```
$faction = $_COOKIE['faction'] ?? 'gdi';
$uploads_dir = wp_upload_dir();

if ($faction == 'gdi')
    $poster_path = trailingslashit($uploads_dir['baseurl']) . 'images/irina_poster.jpg';
else
    $poster_path = trailingslashit($uploads_dir['baseurl']) . 'images/poster.jpg';

$video_url = get_field('video_destacado', $product->get_id());
if (!$video_url) return;

?>

<a href="<?php echo esc_url($video_url); ?>"
```

```

data-fancybox="videos-tienda"
class="video-loop-link video-loop-wrapper">
<div class="video-container" style="position:relative;">
  <video
    width="630"
    height="870"
    poster="<?php echo esc_url($poster_path); ?>"
    data-src="<?php echo esc_url($video_url); ?>"
    preload="metadata"

```

`get_field('video_destacado', $product->get_id())` obtiene desde ACF la URL asociada al producto. PHP no activa inmediatamente la descarga como `src`: la escribe en `data-src`. Más tarde, el `IntersectionObserver` de la sección 33 copia ese valor a `src`. El poster sí se construye a partir de `wp_upload_dir()['baseurl']` y se escapa con `esc_url()`.

```

WC_Product
↓ product_id
ACF video_destacado
↓ URL
HTML data-src
↓ IntersectionObserver
src
↓ petición HTTP
MP4

```

35.2 Caso B: URL pública de Gallery frente a archivo físico

Gallery no usa Media Library para descubrir sus archivos: `$videos` contiene URLs como `/videos/nod-gallery/video2_main.mp4`. El relevamiento físico confirma archivos dentro de `wp-content/uploads/videos/NOD-GALLERY/`. El child theme no contiene la regla que une ambas rutas, por lo que ese mapeo pertenece al servidor y debe conservarse al migrar.

Concepto	Ejemplo
URL que recibe el navegador	<code>/videos/nod-gallery/video2_main.mp4</code>
Ubicación física confirmada	<code>wp-content/uploads/videos/NOD-GALLERY/video2_main.mp4</code>
Responsable del mapeo	Configuración del servidor fuera del child theme

No confundir: una URL pública no demuestra por sí sola la ruta física del archivo. Son dos niveles diferentes.

35.3 Caso C: logo del email construido con `wp_upload_dir()`

Código fuente real: `functions.php` → `WC_Gateway_Manual_Pay::process_payment()` → líneas 7423-7425.

```

// Path al logo
$uploads_dir = wp_upload_dir();
$logo_url = trailingslashit($uploads_dir['baseurl']) . 'images/logo.png';

```

`wp_upload_dir()` devuelve información del directorio de uploads. El código utiliza `baseurl`, no la ruta física del disco, porque el HTML del email necesita una URL que el cliente de correo pueda solicitar.

35.4 Caso D: Fancybox y HLS.js llegan desde jsDelivr

Código fuente real: `functions.php` → `cargar_fancybox_tienda()` → líneas 30-43. HLS.js también se carga desde jsDelivr con `@latest` en `functions.php` y `hud-intro.php`.

```

add_action('wp_enqueue_scripts', 'cargar_fancybox_tienda');

// =====
// Función que carga las librerías para iniciar fancybox.
// =====
function cargar_fancybox_tienda() {

    // Cargar jQuery (ya viene en WordPress)
    wp_enqueue_script('jquery');

    if (is_shop() || is_product_category() || is_product_tag()) {

```

```
wp_enqueue_style('fancybox-css', 'https://cdn.jsdelivr.net/npm/@fancyapps/ui/dist/fancybox.css', [],
'4.0', 'all');
wp_enqueue_script('fancybox-js', 'https://cdn.jsdelivr.net/npm/@fancyapps/ui/dist/fancybox.umd.js',
[], '4.0', true);
}
```

Fancybox se carga desde jsDelivr mediante URLs que no fijan explícitamente la versión del paquete en la propia ruta. El valor 4.0 pasado a `wp_enqueue_style()`/`wp_enqueue_script()` funciona como versión del recurso para WordPress y puede aparecer como parámetro de consulta, pero no equivale a solicitar `@fancyapps/ui@4.0` desde jsDelivr. HLS.js, por su parte, aparece explícitamente como `hls.js@latest`. En ambos casos, fijar la versión del paquete en la URL CDN mejoraría la reproducibilidad del despliegue.

35.5 Caso E: HLS no es un único archivo MP4

Un manifiesto .m3u8 describe un stream compuesto por fragmentos. HLS.js o el soporte nativo leen ese manifiesto y solicitan segmentos según la reproducción y el buffer. Por eso el flujo correcto es manifiesto → segmentos → buffer → video, y no “descargar un MP4 completo”.

35.6 Música y sonidos

Las carpetas `wp-content/uploads/songs` y `wp-content/uploads/sounds` contienen recursos usados por Intro, menú y selector de facción. Igual que con los videos, PHP/HTML imprime URLs y el navegador crea los objetos de audio. Su existencia en uploads es independiente de las cookies o de la sesión de WooCommerce.

35.7 Matriz final de recursos

Recurso	Fuente de verdad / origen	Quién construye o expone la URL	Consumidor
video_destacado	ACF + attachment/uploads	<code>get_field()</code> / WordPress	Shop/Bundles
Gallery MP4	array \$videos + archivos físicos	<code>functions.php</code> + mapeo servidor	Gallery
Logo email	<code>uploads/images/logo.png</code>	<code>wp_upload_dir()</code>	cliente de correo
HLS Intro	manifiestos .m3u8	<code>hud-intro.php</code>	HLS.js / video nativo
Fancybox	jsDelivr	<code>wp_enqueue_script/style</code>	navegador
SMTP	configuración WordPress/plugin	<code>wp_mail()</code> → WP Mail SMTP	servidor de correo

36. Caché y rendimiento

En el proyecto conviven mecanismos distintos: caché HTTP, versionado de CSS, lazy loading, buffering HLS, sesión WooCommerce y cachés lógicas de JavaScript. Esta sección conserva esa distinción y analiza además los costos de rendimiento que surgen de la implementación actual.

36.1 Versionado de CSS: `filemtime()` y `time()`

Código fuente real: `functions.php` → dos callbacks `wp_enqueue_scripts` con prioridad 999 → líneas 92-185.

```
if ($faction === 'gdi') {
    $css_file = get_stylesheet_directory() . '/gdi-faction-min.css';
    $version = file_exists($css_file) ? filemtime($css_file) : '1.0';

    wp_enqueue_style(
        'gdi-faction',
        get_stylesheet_directory_uri() . '/gdi-faction-min.css',
        [],
        $version,
        'all'
    );
} elseif ($faction === 'nod') {
    $css_file = get_stylesheet_directory() . '/nod-faction-min.css';
```

```

    $version = file_exists($css_file) ? filemtime($css_file) : '1.0';

    wp_enqueue_style(
        'nod-faction',
        get_stylesheet_directory_uri() . '/nod-faction-min.css',
        [],
        $version,
        'all'
    );
}
}, 999);
add_action('wp_enqueue_scripts', function() {
    // Detectar facción según cookie
    $faction = $_COOKIE['faction'] ?? '';

    // Si no existe la cookie o está vacía, no cargamos ningún CSS
    if (empty($faction)) return;

    // Directorios del tema
    $theme_dir = get_stylesheet_directory();
    $theme_uri = get_stylesheet_directory_uri();

    // Generar parámetro dinámico basado en la hora actual
    // Esto fuerza la recarga del archivo en cada refresh
    $timestamp = time();

    // Cargar el CSS correspondiente a la facción
    if ($faction === 'gdi') {
        $url = $theme_uri . '/gdi-faction-min.css?v=' . $timestamp;
        wp_enqueue_style('gdi-faction', $url, [], null, 'all');
    } elseif ($faction === 'nod') {
        $url = $theme_uri . '/nod-faction-min.css?v=' . $timestamp;
        wp_enqueue_style('nod-faction', $url, [], null, 'all');
    }
}, 999);

```

El primer bloque usa `filemtime()`: mientras el archivo no cambie, la versión puede permanecer estable. El segundo genera `time()` en cada petición y construye una URL distinta. Los dos callbacks están activos y utilizan los mismos handles `gdi-faction/nod-faction`. No debe suponerse que el segundo “reemplaza” de manera limpia al primero; la coexistencia es una estrategia redundante que conviene unificar.

36.2 Lazy loading de Shop como optimización

Las tarjetas se imprimen con `data-src` y el video se activa cerca del viewport. Esto evita iniciar todos los MP4 de Shop en la primera carga. El observer utiliza `rootMargin=200px`, de modo que puede precargar un recurso poco antes de que entre en pantalla.

36.3 Gallery: 12 videos principales con preload=auto

Código fuente [real: functions.php](#) → [cnc_gallery_parallax\(\)](#) → generación de cada `vid-main` en líneas 8494-8498.

```

<?php foreach($videos as $v): ?>
    <div class="frame-container" data-alt='<?=json_encode($v['alt']) ?>' data-sound='<?=$v['sound'] ? 1 : 0 ?>'>
        <div class="video-wrap">
            <video class="vid-main" src="<?=$v['main'] ?>" autoplay muted loop
playsinline preload="auto"></video>
        </div>
    </div>
</?>

```

Gallery genera 12 elementos `vid-main` al abrir la página y todos incluyen `preload="auto"`. Las variantes `alt` sí se crean bajo demanda, lo que reduce su costo inicial, pero los `main` pueden comenzar a solicitar datos desde el inicio. Esto contrasta con Shop, donde `data-src` difiere la activación.

36.4 Bundles: consulta completa y filtrado en PHP

Código fuente [real: functions.php](#) → [bloque de Bundles](#) → [wc_get_products\(\)](#) en 4504-4510 y filtrado manual desde 4517.

```

    $all_bundles = wc_get_products([
        'status'      => 'publish',
        'limit'       => -1,
        'orderby'     => 'title',
        'order'       => 'ASC',
        'tax_query'   => $tax_query,
    ]);

    // === PAGINACIÓN ===
    $paged = max(1, absint(get_query_var('paged') ?: get_query_var('page') ?: (isset($_GET['paged']) ?
    intval($_GET['paged']) : 1)));
    $per_page = 12;
    $offset = ($paged - 1) * $per_page;

    // === FILTRADO MANUAL ===
    $filtered_products = [];
    foreach ($all_bundles as $bundle) {
        $price = floatval($bundle->get_price());
        $name = $bundle->get_name();
        if ($price >= $min_price && $price <= $max_price && ($q === '' || stripos($name, $q) !== false)) {
            $filtered_products[] = $bundle;
        }
    }

```

limit=-1 solicita todos los bundles compatibles con tax_query y luego el código recorre el resultado para filtrar precio/texto y paginar. Con el catálogo actual puede funcionar, pero el costo crece con la cantidad de bundles porque la paginación ocurre después de obtener el conjunto completo.

36.5 HLS: buffering frente a descarga completa

Intro/Home trabajan con manifiestos y segmentos. El consumo depende del buffer y de la reproducción. La configuración Chromium limita maxBufferLength/maxMaxBufferLength, mientras Firefox usa un backBufferLength diferente. Esto es un costo dinámico, no una descarga monolítica.

36.6 Qué ya está optimizado

- Shop difiere previews mediante data-src + IntersectionObserver.
- Gallery crea variantes alternativas sólo cuando el usuario las solicita y luego las reutiliza mediante altCache.
- Se utilizan hojas CSS minificadas para las facciones en la carga normal.
- HLS permite trabajar por segmentos en lugar de requerir un MP4 completo para Intro/Home.

36.7 Costos y puntos de revisión comprobados

Punto	Comportamiento actual	Posible mejora
CSS de facción	filemtime() y time() activos con mismo handle	Conservar una estrategia según entorno
Gallery	12 main con preload=auto	Revisar preload/activación inicial
Bundles	wc_get_products(limit=-1) + filtrado manual	Mover más filtrado/paginación a la consulta
HLS.js	CDN con @latest	Fijar una versión conocida
Shop	lazy loading por observer	Mantener; es una optimización efectiva
Fancybox	URL jsDelivr sin versión de paquete fijada; ver=4.0 es versionado de WordPress, no pin del paquete	Fijar una versión conocida dentro de la URL CDN

No confundir rendimiento con caché: reducir descargas iniciales, evitar consultas completas y controlar buffers son problemas diferentes de decidir si una respuesta HTTP puede reutilizarse.

37. Tratamiento de datos y seguridad

La seguridad se analiza siguiendo entradas reales del proyecto. Validar, sanitizar y escapar son etapas distintas: validar decide si un valor cumple una regla; sanitizar lo normaliza o limpia antes de usarlo; escapar lo prepara para un contexto de salida como HTML o una URL.

37.1 Caso A: Artworks Name desde Checkout hasta WC_Order

Código fuente real: functions.php → callback de woocommerce_checkout_create_order → líneas 8297-8301.

```
add_action('woocommerce_checkout_create_order', function($order, $data) {
    if ( !isset($_POST['artwork_name']) && $_POST['artwork_name'] !== '' ) {
        $order->update_meta_data('Artworks Name', sanitize_text_field($_POST['artwork_name'])); // Traducido
    }
}, 10, 2);
```

```
input artwork_name
↓ POST
sanitize_text_field()
↓
$order->update_meta_data()
↓ WC_Order
$order->get_meta()
↓
esc_html() cuando se incorpora a HTML
```

La sanitización no sustituye la validación de longitud de Checkout. Tampoco sustituye el escape posterior cuando el mismo valor se imprime dentro del email o de una página.

37.2 Caso B: validación adicional del email

Código fuente real: functions.php → callbacks de woocommerce_checkout_process alrededor de 8195-8256.

```
add_action('woocommerce_checkout_process', function() {
    $email = '';

    // Capturar el email del checkout
    if (!empty($_POST['billing_email'])) {
        $email = sanitize_text_field($_POST['billing_email']);
    } elseif (!empty($_POST['billing']['email'])) {
        $email = sanitize_text_field($_POST['billing']['email']);
    }

    // 1) Longitud máxima
    if (!empty($email) && strlen($email) > 100) {
        wc_add_notice('Email cannot exceed 100 characters.', 'error'); // Traducido
        return;
    }

    // 2) Formato básico con filter_var
    if (empty($email) || !filter_var($email, FILTER_VALIDATE_EMAIL)) {
```

El código limpia el valor, valida formato y después separa el dominio. Más adelante intenta comprobar registros DNS mediante resolución MX/A y un fallback de host. Esa prueba puede detectar algunos errores de dominio, pero no demuestra que el buzón individual exista ni que vaya a aceptar correo.

37.3 Caso C: endpoint AJAX público obtener_id_desde_slug

Código fuente real: functions.php → wp_ajax_obtener_id_desde_slug / wp_ajax_nopriv_obtener_id_desde_slug → líneas 3693-3725.

```
add_action('wp_ajax_obtener_id_desde_slug', 'obtener_id_desde_slug_callback');
add_action('wp_ajax_nopriv_obtener_id_desde_slug', 'obtener_id_desde_slug_callback');

/* =====
 * Función que responde a la petición AJAX para obtener el ID de un producto
 * dado su slug (parte final de la URL).
 * Esta función está registrada para usuarios logueados y no logueados.
 * Proceso:
 * - Verifica que se envió el parámetro 'slug' vía POST.
```

```

* - Sanitiza el slug recibido para evitar inyección o caracteres inválidos.
* - Busca el producto en la base de datos usando el slug.
* - Si encuentra el producto, devuelve un JSON con el ID.
* - Si no encuentra el producto o falta el slug, termina la ejecución sin respuesta.
* ===== */
function obtener_id_desde_slug_callback() {
    // Verifica que se haya recibido el parámetro 'slug' vía POST
    if (!isset($_POST['slug'])) {
        wp_die(); // Termina sin respuesta si falta slug
    }

    // Limpia y sanitiza el valor recibido para evitar inyecciones o caracteres inválidos
    $slug = sanitize_title($_POST['slug']);

    // Busca el producto en la base de datos usando su slug (nombre amigable en la URL)
    $producto = get_page_by_path($slug, OBJECT, 'product');

    if ($producto) {
        // Si el producto existe, responde con éxito y devuelve un array con el ID del producto
        wp_send_json_success(['id' => $producto->ID]);
    } else {
        wp_die(); // Termina sin respuesta si no encuentra el producto
    }
}

```

El endpoint está disponible para usuarios autenticados y no autenticados. Comprueba que exista slug, aplica `sanitize_title()`, busca un product mediante `get_page_by_path()` y, si lo encuentra, devuelve únicamente su ID con `wp_send_json_success()`.

Control no presente en el callback actual: este código no contiene `check_ajax_referer()` ni una comprobación de capability. Su alcance debe interpretarse con precisión: es un endpoint de lectura que devuelve un ID de producto; la ausencia de nonce no equivale por sí sola a afirmar una vulnerabilidad crítica o capacidad de modificación.

37.4 Caso D: cookie faction

Código fuente real: `faction-selection.php` → al confirmar la facción se escribe `document.cookie`; `functions.php` y `header.php` leen `$_COOKIE['faction']`.

```
document.cookie = "faction=" + faction.dataset.faccion + "; path=/; max-age=" + (60 * 60 * 24 * 365);
```

La cookie se usa para decidir apariencia y recursos. En muchos puntos del PHP se aplica un valor por defecto `gdi`, pero no todos los lectores validan explícitamente que el contenido sea sólo `gdi` o `nod`. Una cookie es entrada controlable por el navegador; no debe tratarse conceptualmente como una credencial ni como una prueba de identidad.

37.5 Escape de salida

Función	Uso comprobado	Objetivo
<code>esc_html()</code>	nombre, email, metadatos en HTML/email	evitar interpretar texto como marcado
<code>esc_url()</code>	videos, posters, logo, enlaces	preparar una URL para salida HTML
<code>sanitize_text_field()</code>	Artworks Name / valores POST	limpiar texto de entrada
<code>sanitize_title()</code>	slug del AJAX	normalizar un slug
<code>wc_clean()</code>	datos WooCommerce	normalizar valores del dominio WooCommerce

37.6 Logs: permiso de escritura no equivale a privacidad

Los logs contienen nombre, email, productos, personalizaciones y total. PHP necesita permiso de escritura sobre `wp-content/uploads/logs`, pero ese permiso no demuestra que el directorio esté bloqueado frente a peticiones HTTP. La protección depende de la configuración del servidor o de una estrategia de logging diferente. `get_log_path()` tampoco crea la carpeta si falta.

No confundir: “PHP puede escribir” y “un visitante no puede descargar el archivo” son propiedades distintas.

37.7 Bloqueos visuales frente a seguridad real

El child theme bloquea selección, arrastre o contextmenu en distintas pantallas. Esas decisiones pueden dificultar acciones accidentales, pero no son DRM ni impiden inspeccionar recursos que el navegador ya descargó. La seguridad real depende de validación, permisos, configuración del servidor, controles de acceso y tratamiento correcto de entradas/salidas.

37.8 AIOS y alcance de la información disponible

All-In-One Security forma parte del entorno relevado, pero la configuración concreta de sus módulos no está definida en el código del tema hijo. Por eso esta documentación no le atribuye reglas específicas que no estén demostradas por la información disponible.

37.9 Matriz de entradas principales

Entrada	Origen	Tratamiento comprobado	Persistencia/salida
Artworks Name	Checkout POST	validación + sanitize_text_field	WC_Order meta → email/log
Email	Checkout POST	formato + DNS + límites	billing data → wp_mail/log
Frame Color / Balloon	Cart	limpieza/validaciones de flujo	WC_Order_Item meta
slug AJAX	POST público	sanitize_title	ID de producto JSON
faction	cookie navegador	default y comparaciones	selección de CSS/recursos

38. Instalación, migración y verificación del proyecto

38.1 Qué trae el proyecto y qué debe reconstruirse

Elemento	¿Está dentro de blocksy-child?	Qué hacer al migrar
Código PHP/CSS del child theme	Sí	Copiar a wp-content/themes/blocksy-child y activar junto a Blocksy.
WordPress Core	No	Instalar/configurar WordPress.
Blocksy	No	Instalar el tema padre compatible.
WooCommerce/ACF/HUSKY/Xootix/WP Mail SMTP	No	Instalar y configurar cada plugin.
Base de datos	No forma parte del tema hijo	Restaurar/importar la instalación correspondiente.
wp-content/uploads	No forma parte del tema hijo	Restaurar imágenes, MP4, audio, HLS y logs/directorios necesarios.
Configuración /videos/	No confirmada dentro del child theme	Recrear regla o mapeo equivalente en el servidor.

La instalación debe comprobar cada capa antes de continuar. Copiar sólo el tema hijo no reproduce el sitio, porque productos, ACF, páginas, plugins, uploads y reglas del servidor participan en el comportamiento real.

38.2 Qué debe reconstruirse fuera del child theme

La documentación técnica sólo es reproducible si se distinguen código, base de datos, uploads y configuración de plugins/servidor.

38.3 Capas que deben quedar disponibles

Pieza	Ubicación / origen	Responsabilidad
WordPress + PHP + servidor web	entorno	ejecuta la plataforma
Blocksy	tema padre	requisito del child theme
blocksy-child	código propio	debe copiarse y activarse
Base de datos	sitio WordPress/WooCommerce	productos, pedidos, opciones
wp-content/uploads	archivos físicos	media, sonidos, videos, logs
Plugins	WordPress Admin	WooCommerce, ACF, HUSKY, Xootix, WP Mail SMTP, etc.
Reglas servidor	servidor web	rutas públicas como /videos/...

No confundir

Copiar únicamente el tema hijo no reproduce el sitio completo.

El objetivo de cada paso es dejar una condición comprobable antes de continuar.

38.4 Orden de montaje y comprobación

```

instalar WordPress y comprobar acceso al admin
↓
instalar/activar Blocksy
↓
copiar blocksy-child a wp-content/themes
↓
activar child theme
↓
instalar WooCommerce, ACF, HUSKY/WOOF, Xootix y WP Mail SMTP
↓
restaurar base y uploads
↓
crear/verificar páginas con slugs requeridos
↓
comprobar campos ACF
↓
comprobar catálogo/carrito/checkout

```

No confundir

La configuración exacta de HUSKY/Xootix no está completamente expresada en el código PHP; debe verificarse también en WordPress Admin.

El código consulta páginas concretas con `is_page()` y espera recursos en rutas determinadas. Una migración debe conservar esas expectativas.

38.5 Dependencias externas al tema hijo

Pieza	Ubicación / origen	Responsabilidad
/intro/	WordPress Page	activa hud-intro
/faction-selection/	WordPress Page	activa selector NOD/GDI
/bundles/	WordPress Page	activa contenido de Bundles
/gallery/	WordPress Page	activa Gallery
/our-services/	WordPress Page	activa Services
ACF	grupo de campos	video_destacado, productos_del_bundle, tiene_globo, video_workshop
uploads	sistema de archivos	medios y sonidos

No confundir

El código también consulta `es_bundle`, pero el relevamiento ACF anterior no confirmó ese campo dentro de la configuración relevante; debe verificarse en la instalación real.

MC Artworks Studio funciona sobre una instalación de WordPress con el tema padre Blocksy, el tema hijo blocksy-child, WooCommerce y los plugins utilizados por el proyecto. Para reproducir correctamente el sitio también deben estar disponibles la base de datos, la estructura de wp-content/uploads y los recursos multimedia asociados.

38.6 Orden recomendado de instalación

1. Instalar WordPress en un entorno compatible con WooCommerce.
2. Instalar y activar el tema padre Blocksy.
3. Copiar blocksy-child a wp-content/themes/ y activar el tema hijo.
4. Instalar WooCommerce.
5. Instalar Advanced Custom Fields y recrear o importar los campos video_destacado, video_workshop, productos_del_bundle y tiene_globo con sus reglas de ubicación.
6. Instalar HUSKY/WOOF y Xootix Side Cart y verificar su configuración de acuerdo con la integración documentada en el tema hijo.
7. Configurar las páginas intro, faction-selection, bundles, gallery, our-services, cart y checkout según las rutas usadas por el código.
8. Restaurar wp-content/uploads, incluyendo categories, images, videos, songs, sounds y los archivos administrados por Media Library, además de cualquier recurso HLS utilizado por el tema.
9. Crear o verificar wp-content/uploads/logs con permisos de escritura y protección adecuada.
10. Configurar el envío de correo mediante WP Mail SMTP o el servicio SMTP elegido; el tema solicita los envíos mediante wp_mail().
11. Crear o importar productos y categorías de WooCommerce. Los bundles son productos simples de la categoría Bundles y utilizan productos_del_bundle para relacionar los artworks.
12. Configurar el dominio y las rutas del entorno donde se despliegue el proyecto.

Comprobación por capas. Después de WordPress + Blocksy + blocksy-child, la portada debe cargar sin errores PHP. Después de WooCommerce, Shop/Cart/Checkout deben existir. Después de ACF, un producto de prueba debe devolver sus campos. Después de restaurar uploads, los posters y videos deben resolver por HTTP. Después de SMTP, debe enviarse un mensaje de prueba antes de validar manual_pay.

Si una capa falla, conviene detener la instalación allí: corregir un problema de rutas, permisos o plugin después de cargar todo el sistema vuelve más difícil identificar la causa. Las subsecciones 38.14 y 38.15 convierten estas comprobaciones en una prueba funcional y un diagnóstico por síntoma.

Plugin auxiliar de seguridad

AIOS forma parte del entorno relevado como plugin auxiliar de seguridad. No es un requisito funcional del flujo Shop → Cart → Checkout → Order Received y su configuración se administra fuera del tema hijo.

Configuración de plugins

La documentación describe la integración que espera blocksy-child, pero no dispone de un export completo de la configuración administrativa de HUSKY/WOOF y Xootix Side Cart. Al migrar el proyecto, esos ajustes deben verificarse en la instalación original antes de considerar reproducido su comportamiento.

Contenido y recursos

La base de datos conserva productos, configuración, sesiones y pedidos, mientras wp-content/uploads contiene los archivos multimedia y logs. Para reproducir el sitio completo deben estar disponibles ambos componentes.

Configuración del entorno

Al desplegar el proyecto en otro dominio deben ajustarse las URLs y rutas que dependan del entorno. WordPress ofrece funciones como home_url(), get_permalink() y wp_upload_dir() para obtener estas direcciones a partir de la configuración del sitio, evitando escribirlas manualmente en distintos lugares del código.

38.7 Páginas y slugs que espera el código

```
/intro/
/faction-selection/
/bundles/
/gallery/
/our-services/
/cart/
/checkout/
```

Varias funciones utilizan `is_page()` o URLs concretas. Si se cambia un slug sin modificar el código correspondiente, esa lógica puede dejar de ejecutarse.

38.8 Configuración mínima de ACF

- `video_destacado`: recurso principal del producto utilizado en Shop y Bundles.
- `video_workshop`: recurso Workshop opcional.
- `productos_del_bundle`: relación entre el bundle y los artworks que representa.
- `tiene_globo`: determina si el producto admite Comic Balloon Text.

`functions.php` también consulta `es_bundle` dentro de `wpo_filtrar_meta_en_pedido()`. El código espera ese campo, pero el relevamiento actual de ACF no lo incluye entre los cuatro campos confirmados. Si `es_bundle` no está configurado, `get_field()` devuelve un valor vacío y esa función no reconoce bundles por esa vía; el filtrado queda apoyado en `tiene_globo`.

38.9 WooCommerce

Para reproducir el entorno actual deben existir Shop, Cart y Checkout, la moneda debe ser USD y HPOS debe estar disponible si se desea conservar el mismo modelo de almacenamiento de pedidos. La categoría con slug bundles es especialmente importante porque Shop la excluye mediante `pre_get_posts`.

38.10 Recursos multimedia

No alcanza con copiar `blocksy-child`. También debe restaurarse `wp-content/uploads` con imágenes, videos, canciones, sonidos, attachments y streams HLS. En Gallery, por ejemplo, los archivos NOD son MP4 físicos dentro de `uploads/videos/NOD-GALLERY/`.

38.11 Logs y correo

Debe existir `wp-content/uploads/logs` con permisos para que PHP pueda escribir. WP Mail SMTP o el servicio SMTP elegido debe configurarse y probarse antes de considerar terminado el despliegue, porque `manual_pay` depende del correo para entregar las instrucciones de pago.

38.12 Cómo comprobar el sitio por capas

Dos dependencias críticas no se validan sólo viendo que WordPress carga: correo y rutas multimedia.

38.13 De una prueba simple al recorrido completo

```
configurar WP Mail SMTP y enviar prueba
↓
verificar que wp_mail() del gateway llegue por el transporte configurado
↓
comprobar que URLs /videos/... respondan en el servidor
↓
abrir una primera visita sin cookie faction
↓
elegir facción
↓
abrir Shop y reproducir previews
↓
agregar a Cart y personalizar
↓
completar Checkout
```

↓
confirmar WC_Order + log + emails + Order Received

No confundir

Un sitio puede “abrir” y aun así estar incompleto si fallan SMTP, HLS, rutas /videos/ o configuraciones de plugins.

Instalar archivos no demuestra que el entorno esté completo. La comprobación final debe recorrer el sistema por capas para localizar rápidamente qué pieza falla.

38.14 Prueba funcional después de instalar

1. Seleccionar GDI o NOD y comprobar que la elección se conserve.
2. Abrir Home y verificar que cargue el contenido correspondiente a la facción elegida.
3. Entrar en Shop y comprobar que las vistas previas de video se reproduzcan correctamente.
4. Usar ADD TO CART y verificar que WooCommerce agregue el producto y Xootix actualice el carrito lateral.
5. Abrir Cart, elegir Frame Color y, cuando corresponda, completar Comic Balloon Text; después actualizar el carrito.
6. Continuar a Checkout y comprobar la validación de Artworks Name, First Name y Email.
7. Confirmar Pay with PayPal y verificar que WooCommerce cree el WC_Order y conserve sus datos y personalizaciones.
8. Comprobar que el archivo diario de logs reciba la información del pedido y el resultado de los intentos de correo.
9. Verificar el intento de envío al comprador y a administración mediante WP Mail SMTP o el servicio SMTP configurado.
10. Abrir Order Received y confirmar que muestre la información del pedido y las instrucciones para completar el pago manual.

Criterio de comprobación

Si alguno de estos pasos falla, conviene revisar la capa correspondiente antes de modificar otras: página/slug, plugin, base de datos, archivo físico, JavaScript, sesión de WooCommerce, permisos del log o configuración SMTP.

38.15 Diagnóstico rápido por síntoma

Prueba/ síntoma	Qué confirma	Revisar primero si falla
WordPress abre el sitio	Core, servidor, DB básica y tema activo.	configuración de WordPress, servidor, conexión DB.
Intro reproduce GDI/NOD	hud-intro.php, HLS/MP4 y rutas multimedia.	uploads, /videos/, HLS.js, consola del navegador.
Faction Selection conserva la elección	cookie faction + lectura PHP.	faction-selection.php, cookies del navegador.
Shop muestra productos	WooCommerce + consulta + tema.	productos, categoría Bundles, hooks Shop.
Preview se activa al hacer scroll	data-src + observer + archivo MP4.	video_destacado, URL, consola/red.
Cart conserva Blue tras F5	sesión WooCommerce + callbacks de restauración.	cart item, sesión, cookies.
Checkout crea pedido	validaciones + WC_Order/HPOS.	avisos WooCommerce, logs PHP, configuración Checkout.
Se crea *_orders.log	permisos de uploads/logs + gateway.	carpeta, permisos, error_log.
Llega el email	wp_mail + WP Mail SMTP + servicio SMTP.	prueba SMTP, logs del plugin/servidor.

38.16 Prueba completa con Alex

```
primera visita
-> Intro
-> elegir NOD/GDI
-> Home
-> Shop
-> Cyberpunk Portrait
-> Add to Cart
```

```
-> Welcome back + Blue
-> Checkout + Raven
-> crear pedido
-> log
-> emails
-> Order Received
```

38.17 Checklist final de entorno

Comprobación	Estado esperado antes de dar el despliegue por válido
WordPress + Blocksy + child theme	La portada y las páginas base cargan sin errores PHP
Plugins	WooCommerce, ACF, HUSKY/WOOF, Xootix y WP Mail SMTP activos/configurados
Base de datos	Productos, categorías, páginas, opciones y pedidos disponibles
Uploads	Imágenes, MP4, HLS, songs y sounds responden por HTTP
/videos/	El mapeo externo resuelve las URLs usadas por Intro/Gallery
ACF	video_destacado, video_workshop, productos_del_bundle y tiene_globo disponibles
Logs	uploads/logs existe, PHP puede escribir y el acceso público está controlado
SMTP	Mensaje de prueba y wp_mail() del gateway atraviesan el transporte configurado
Flujo funcional	Intro → facción → Shop → Cart → Checkout → Order Received funciona de punta a punta

39. Posibles líneas de evolución

Las mejoras de esta sección se derivan de comportamientos presentes en el código actual del proyecto. No describen funciones que el proyecto ya tenga: son una hoja de mantenimiento que relaciona cada hallazgo con su ubicación, consecuencia técnica y una posible línea de mejora.

39.1 Matriz de hallazgos priorizados

Prioridad	Comportamiento actual	Ubicación / evidencia	Posible evolución y objetivo
Alta	functions.php supera 8.800 líneas y concentra muchas responsabilidades	functions.php	Separar progresivamente la lógica por áreas como Shop, Cart, Checkout, correos, Gallery y Services para facilitar la lectura, el mantenimiento y la localización de errores sin cambiar el comportamiento actual.
Alta	Dos callbacks activos versionan el mismo CSS con filemtime() y time()	functions.php 92-185	Usar una sola estrategia de versionado para evitar que dos mecanismos distintos intenten controlar la caché del mismo CSS. En producción, filemtime() permite renovar la versión cuando cambia el archivo.
Alta	Callback de bloqueo Add to Cart registrado dos veces	functions.php: 2685 y 2690	Conservar un solo add_action() para evitar que el mismo callback pueda ejecutarse dos veces durante wp_footer.

Prioridad	Comportamiento actual	Ubicación / evidencia	Posible evolución y objetivo
Alta	wc_cart_url se localiza activamente como objeto {url: ...}, pero algunos consumidores lo usan directamente como href	functions.php: localización activa 3567-3574; consumidores posteriores usan href: wc_cart_url	Unificar la representación de wc_cart_url: usar wc_cart_url.url si se conserva el objeto, o exponer directamente una cadena. Así se evita entregar un objeto donde el código espera una URL.
Alta	Logs con datos personales; get_log_path() no crea la carpeta logs	WC_Gateway_Manual_Pay	Asegurar que la carpeta exista antes de escribir, impedir su acceso público y definir cuánto tiempo se conservarán los registros. Como alternativa, utilizar el sistema de logs de WooCommerce para centralizar su gestión.
Media	HLS.js se carga mediante @latest	functions.php / hud-intro.php	Usar una versión específica de HLS.js en lugar de @latest, para que una nueva publicación de la librería no pueda cambiar Intro o Home sin haber sido probada previamente.
Media	Gallery genera 12 main con preload=auto	cnc_gallery_parallax()	Reducir o escalonar la precarga de los videos principales para evitar que el navegador intente preparar los doce recursos desde el inicio cuando el usuario todavía no los necesita.
Media	Gallery: sin error/fallback MP4 y sound=false inconsistente	cnc_gallery_parallax()	Agregar un estado de error o fallback cuando un video no pueda cargarse y corregir la lógica de sound para que un recurso marcado sin sonido no termine reproduciendo audio sin un control disponible para silenciarlo.
Media	Bundles obtiene limit=-1 y filtra/pagina después en PHP	functions.php 4504+	Trasladar a la consulta la mayor cantidad posible de filtros y paginación para evitar cargar y recorrer todos los bundles cuando el catálogo aumente de tamaño.
Media	/videos/... depende de mapeo externo al child theme	Intro / Gallery	Definir las rutas multimedia en un único lugar y documentar la regla del servidor que relaciona /videos/... con los archivos físicos, para facilitar migraciones a otro dominio o servidor.
Media	AJAX público de slug sin nonce/capability	callback AJAX 3693-3725	Determinar si el endpoint necesita verificar la procedencia de la petición mediante un nonce. Como sólo devuelve el ID de un producto, el control debe ajustarse al riesgo real y no agregar permisos innecesarios.

Prioridad	Comportamiento actual	Ubicación / evidencia	Posible evolución y objetivo
Media	Chequeo DNS durante Checkout	callbacks de validación de Checkout	Valorar si la comprobación DNS aporta suficiente beneficio frente al tiempo adicional de resolución y a posibles falsos negativos, ya que un fallo DNS temporal podría impedir continuar con un email cuyo formato es válido.
Baja	Bloqueos de clic derecho/selección se presentan como UX, no seguridad	varios scripts	Conservarlos sólo si aportan a la experiencia visual, dejando claro que no impiden acceder a recursos ya descargados por el navegador ni sustituyen controles de seguridad del servidor.
Media	Fancybox se carga desde jsDelivr sin fijar la versión del paquete en la URL	cargar_fancybox_tienda(): @fancyapps/ui/dist/...; ver=4.0 es parámetro de WordPress	Indicar una versión concreta de Fancybox en la URL de jsDelivr para que el proyecto siga utilizando la misma versión hasta que una actualización sea probada deliberadamente.

39.2 Doble registro del mismo callback

Código fuente real: functions.php → add_action('wp_footer', 'deshabilitar_boton_add_to_cart_mientras_procesa') aparece en las líneas 2685 y 2690 antes de una única definición de la función.

```
add_action('wp_footer', 'deshabilitar_boton_add_to_cart_mientras_procesa');

/* =====
// Función que deshabilita el boton "Add to Cart" mientras se procesa el producto
/* ===== */
add_action('wp_footer', 'deshabilitar_boton_add_to_cart_mientras_procesa');
function deshabilitar_boton_add_to_cart_mientras_procesa() {
```

WordPress conserva cada registro de action. Mantener dos add_action() idénticos puede provocar que el callback se invoque dos veces en el mismo hook. La mejora es conservar una sola inscripción y dejar la definición de la función separada.

39.3 wc_cart_url: objeto localizado y consumidores que esperan una URL

Código fuente real: functions.php → wp_localize_script alrededor de 3567-3574. El bloque activo expone wc_cart_url como { url: wc_get_cart_url() }; un bloque anterior que la exponía como string está comentado. Consumidores posteriores usan href: wc_cart_url, por lo que esperan una cadena donde actualmente reciben un objeto.

```
// Agrega la URL del carrito disponible para JavaScript
/*
add_action('wp_enqueue_scripts', function() {
    wp_localize_script('jquery', 'wc_cart_url', wc_get_cart_url());
});
*/

add_action('wp_enqueue_scripts', function() {
    wp_localize_script('jquery', 'wc_cart_url', array(
        'url' => wc_get_cart_url()
    ));
});

const verCarrito = $('<a>', {
    href: wc_cart_url,
    class: 'added_to_cart wc-forward'
});
```

Con el código activo, `wc_cart_url` tiene forma `{url: ...}`. Utilizar ese objeto directamente como href no equivale a usar la URL contenida en `wc_cart_url.url`. La corrección consiste en elegir una única representación: mantener el objeto y usar `wc_cart_url.url` en todos los consumidores, o localizar una cadena simple y conservar href: `wc_cart_url` de forma coherente.

39.4 Rendimiento y dependencias externas

El análisis de las implementaciones descritas en las secciones 34-36 muestra tres frentes: Gallery puede iniciar 12 main con `preload=auto`; Bundles obtiene todos los resultados antes de filtrar/paginar; HLS.js se resuelve con `@latest`. Ninguno de estos puntos significa que el sitio no funcione, pero sí afecta previsibilidad, consumo o escalabilidad a medida que crece el catálogo y el tráfico.

39.5 Rutas y configuración de entorno

Las URLs `/videos/...` y algunos datos escritos directamente en `functions.php` hacen que una migración dependa de configuración externa. Una evolución razonable es agrupar rutas y valores de entorno en funciones/configuración de WordPress, conservar URLs construidas con `home_url()`, `get_permalink()` o `wp_upload_dir()` cuando corresponda y documentar explícitamente la regla del servidor que expone `/videos/`.

39.6 Seguridad operativa y logs

La mejora prioritaria no es “poner más validaciones” de manera indiscriminada. Es alinear cada control con su riesgo real: proteger logs con datos personales, asegurar creación/retención del directorio, revisar el endpoint AJAX según su uso y mantener separadas las medidas de interfaz de los controles de acceso del servidor.

39.7 Criterio de prioridad

Alta: puede provocar ejecución duplicada, comportamiento inconsistente, problemas de mantenimiento o exposición operativa relevante. Media: afecta previsibilidad, rendimiento, migración o endurecimiento. Baja: mejora claridad/UX sin modificar el núcleo comercial.

La prioridad es de mantenimiento técnico, no una afirmación de que el sitio esté roto. La documentación registra el comportamiento existente para que las mejoras puedan hacerse sin perder el flujo NOD/GDI, la experiencia multimedia ni el recorrido comercial.

40. Alcance técnico y conclusión

40.1 Mapa final de responsabilidades

Capa	Responsabilidad principal en MC Artworks Studio
WordPress Core	Recibe peticiones, carga temas/plugins, hooks, páginas, AJAX y APIs base
Blocksy	Tema padre y estructura sobre la que trabaja blocksy-child
blocksy-child	HUD, facciones, multimedia, personalización de Shop/Cart/Checkout, gateway, logs, correos, Gallery y Services
WooCommerce	WC_Product, carrito, sesión, Checkout, WC_Order, gateways y HPOS
ACF	Metadatos propios de productos como video_destacado, tiene_globo y productos_del_bundle
MariaDB	Productos, opciones, sesiones, pedidos y metadatos persistentes
wp-content/uploads + servidor	Archivos físicos, logs y rutas públicas/mapeos multimedia
WP Mail SMTP / SMTP	Transporte posterior a las llamadas wp_mail()

40.2 Dos recorridos que resumen el proyecto

PRIMERA VISITA

URL → WordPress → template_redirect → presentación/Intro → selección NOD/GDI → cookie faction → CSS/recursos de faction → menú/Home

COMPRA

WC_Product → Cart item → personalizaciones → Checkout → WC_Order/WC_Order_Item → manual_pay → log → email comprador → email admin → return URL → Order Received

Si un lector puede seguir ambos recorridos y, en cada transición, identificar archivo, callback, dato de entrada y resultado, la documentación cumple su objetivo pedagógico: explicar no sólo qué ve el usuario, sino cómo cooperan WordPress, WooCommerce, el tema hijo, los plugins, la base de datos y los recursos físicos.

40.3 Fuentes de información utilizadas

Fuente	Uso en este documento
Código fuente del tema hijo blocksy-child	Implementación de PHP, JavaScript, CSS, hooks, rutas y comportamiento propio del sitio.
Configuración y datos de WordPress, WooCommerce y ACF	Estructura del catálogo, pedidos, campos personalizados y configuración funcional del entorno.
Capturas de funcionamiento del proyecto	Recorrido visible de la experiencia del visitante y comprobación de las pantallas principales.
Documentación técnica base del proyecto	Información previa sobre dependencias, base de datos, instalación y características del entorno.

40.4 Conclusión

MC Artworks Studio combina WordPress y WooCommerce con una capa audiovisual propia desarrollada sobre blocksy-child. El proyecto no reemplaza los sistemas de WordPress: se apoya en sus hooks, páginas, plugins, APIs y almacenamiento para construir una experiencia NOD/GDI, un catálogo multimedia y un flujo de compra personalizado.

La complejidad principal está en las conexiones entre capas: PHP genera estructura y scripts; el navegador ejecuta eventos y observers; ACF relaciona productos con recursos; WooCommerce conserva carrito y pedidos; HPOS persiste WC_Order; el gateway manual registra y comunica la compra; uploads y el servidor entregan los recursos multimedia; SMTP transporta los correos.

Los principales puntos de mantenimiento identificados son la concentración de functions.php, los registros duplicados, las estrategias de caché superpuestas, las dependencias externas no fijadas, las rutas ligadas al entorno, los costos de carga multimedia y los controles operativos de logs. Documentarlos permite evolucionar el proyecto sin confundir comportamiento actual con comportamiento deseado.